DLSU

Kyle Edward Ong <kyle_edward_ong@dlsu.edu.ph>

Submission of Revised Thesis Documents

2 messages

Kyle Edward Ong <kyle_edward_ong@dlsu.edu.ph> Sun, Mar 15, 2026 at 4:32 PM
To: Gregory Cu <gregory.cu@dlsu.edu.ph>

Hello Sir, ito po yung updated thesis document with the revisions list. Thank you po!

2 attachments

Revisions List.pdf
94K

NIS 16 - Cross-Layer Dataset Design and Exploratory Analysis of ESP32-Based ESP-WIFI-MESH Network.pdf
7262K

Gregory Cu <gregory.cu@dlsu.edu.ph> Sun, Mar 15, 2026 at 11:37 PM
To: Kyle Edward Ong <kyle_edward_ong@dlsu.edu.ph>

Hello,

I am approving the document.

Greg
[Quoted text hidden]

Cross-Layer Dataset Design and Exploratory Analysis of ESP32-Based ESP-WIFI-MESH Network

A Thesis Proposal Presented to

The Faculty of the College of Computer Studies

De La Salle University

In Partial Fulfillment

of the Requirements for the Degree of

Bachelor of Science in Computer Science

by

Calpoporo, Jose Angelo C.

Carlos, Miguel Sebastian C.

Ong, Kyle Edward M.

Reinante, Christian Victor G.

Mr. Gregory G. Cu

Faculty Adviser

March 15, 2026

Abstract

Wireless Mesh Networks (WMNs) are widely used in Internet of Things (IoT) systems due to their decentralized, multi-hop communication capabilities. The ESP32 microcontroller, with built-in Wi-Fi support and ESP-WIFI-MESH functionality, enables low-cost deployment of infrastructure-less mesh networks. However, most existing intrusion detection research relies on simulation-based datasets that do not accurately capture the cross-layer interactions and hardware-level behaviors observed in real ESP32 mesh deployments.

This study addresses this gap by designing and generating a hardware-derived cross-layer dataset from an ESP32-based ESP-WIFI-MESH testbed deployed in a controlled indoor environment. A network of 5-10 nodes is configured to collect telemetry under normal operation and controlled routing-based attack scenarios, including blackhole and wormhole attacks. Metrics are gathered across the Physical (PHY), Medium Access Control (MAC), and Network layers, including RSSI values, retransmission counts, hop transitions, forwarding ratios, and parent-switching events.

Rather than implementing a real-time intrusion detection system, the study focuses on documenting cross-layer behavioral patterns and applying exploratory data analysis and unsupervised clustering to examine separability between normal and attack conditions. The resulting dataset provides a structured foundation for future research on ESP32-based wireless mesh security.

TABLE OF CONTENTS

TABLE OF TABLES

TABLE OF FIGURES

TABLE OF EQUATIONS

1. Introduction

1.1. Overview

Wireless Mesh Networks (WMNs) have become a critical communication architecture for Internet of Things (IoT) systems due to their self-organizing, infrastructure-less, and multi-hop characteristics [1]. These networks are widely used in smart cities, environmental monitoring, disaster recovery, and rural connectivity, where centralized infrastructure is impractical or unavailable [2]. By allowing each node to function as both a host and a router, WMNs provide scalability and fault tolerance in dynamic environments [1].

Despite these advantages, the decentralized nature of WMNs introduces significant security and reliability challenges [5]. Routing-based misbehavior, such as blackhole and wormhole attacks, can severely degrade network performance, disrupt communication paths, and reduce data delivery reliability [3, 4, 18, 20]. These behaviors are difficult to analyze because malicious nodes often exploit normal routing mechanisms and remain protocol-compliant at the physical and link layers [3, 4].

The ESP32 microcontroller is widely adopted in low-cost IoT deployments and academic research due to its integrated Wi-Fi capabilities, low power consumption, and support for ESP-WIFI-MESH networking [21]. ESP32-based mesh networks are commonly used in smart homes, environmental sensing, and agricultural monitoring [22, 23]. However, empirical studies examining routing behavior and attack-induced anomalies in real ESP32 mesh hardware remain limited [24].

Most existing network security datasets are generated from simulations or traditional IP-based networks and fail to capture the cross-layer behavior, routing dynamics, and hardware-level characteristics of microcontroller-based wireless mesh systems [29, 30, 31]. As noted by Sommer and Paxson [30], the "closed-world assumption" in dataset construction—where controlled laboratory conditions fail to capture operational network complexity—undermines subsequent analysis, regardless of the sophistication of the technique. Similarly, Ring et al. [31] identified common issues in traffic generation, attack diversity, and cross-layer observability in existing datasets, validating the need for modern, hardware-derived mesh datasets. As a result, there is limited understanding of how routing-based misbehavior manifests in real ESP32 mesh deployments [24, 28].

To address this gap, this research focuses on the design and generation of a cross-layer dataset collected from physical ESP32-based wireless mesh networks. The dataset captures observable network behavior under both normal operation and controlled routing-based attack scenarios [25, 26]. Using this dataset, the study performs exploratory data analysis and unsupervised clustering to examine whether distinct behavioral patterns emerge and whether normal and malicious behaviors are separable based on observable network features [38, 41].

1.2. Problem Statement

Wireless Mesh Networks (WMNs) built using ESP32 microcontrollers are increasingly deployed in real-world IoT systems. However, their decentralized and multi-hop nature makes them vulnerable to

routing-based misbehavior such as blackhole and wormhole attacks, which can significantly impact communication reliability and topology integrity.

While prior research has explored attack detection in simulated environments, there is a lack of hardware-generated datasets that capture how such behaviors manifest in real ESP32 wireless mesh networks. Existing datasets are largely simulation-based or designed for conventional networks, and therefore do not represent the cross-layer interactions, routing dynamics, and wireless characteristics inherent to ESP32-based mesh systems.

This lack of realistic datasets limits empirical analysis, reproducibility, and understanding of routing-based misbehavior in microcontroller-based mesh networks. There is a need for a dedicated, hardware-derived dataset that enables exploratory analysis and unsupervised examination of network behavior under normal and adversarial conditions.

1.3. Research Objectives

1.3.1. General Objective

To design, generate, and analyze a comprehensive cross-layer dataset derived from an ESP32-based wireless mesh network to explore and evaluate behavioral differences between normal operation and routing-based attack scenarios.

1.3.2. Specific Objectives

Specifically, this study aims to:

- To establish an ESP32-based wireless mesh testbed capable of generating realistic multi-hop network traffic under both normal operation and controlled routing-based attack scenarios.
- To collect and structure cross-layer features from the Physical (PHY), Medium Access Control (MAC), and Network layers into a labeled and well-documented dataset that captures observable mesh behavior.
- To evaluate the usability and quality of the generated dataset through exploratory data analysis (EDA) and unsupervised or baseline offline machine-learning techniques, to assess whether normal and attack-induced behaviors are separable based on the collected features.

1.4. Project Scope and Limitation

This study focuses on the design, generation, and analysis of a realistic, hardware-based dataset derived from physical ESP32 wireless mesh networks using the ESP-WIFI-MESH protocol. The dataset captures observable network behavior, including signal strength variations, packet-forwarding dynamics, and routing-related inconsistencies that naturally arise in decentralized, multi-hop mesh networks.

Data collection is restricted to two primary network states: normal operation and controlled routing-based attack scenarios. Specifically, the study considers two attack types: blackhole (forwarding suppression)

and wormhole (topology distortion), as these represent fundamental and distinct categories of routing-layer threats—one targeting forwarding compliance and the other targeting topology integrity. A cross-layer data collection strategy is employed to gather indicators from the Physical (PHY), Medium Access Control (MAC), and Network layers, including Received Signal Strength Indication (RSSI), retransmission counts, packet forwarding ratios, changes in hop count, and parent-switching events.

The collected features are intended to support exploratory analysis, clustering, and behavioral separability studies, rather than the development or deployment of any intrusion detection system. The scope of the study is limited to observing and documenting how routing-based disruptions manifest across protocol layers in a real ESP32 mesh network.

1.4.1. Limitations of the Study

Several limitations are acknowledged regarding the dataset's scope and applicability. First, the experimental testbed is limited to a small- to medium-scale deployment consisting of approximately 5 to 10 ESP32 nodes. As a result, the dataset may not fully represent the behavior of large-scale or high-density mesh networks.

Second, the threat model is strictly limited to two specific routing-layer behaviors: blackhole and wormhole attacks. The dataset does not include other routing-layer threats (such as grayhole, Sybil, or selective forwarding), application-layer attacks, cryptographic exploits, physical tampering, firmware compromise, or adversarial radio interference beyond controlled routing manipulation.

Third, the study emphasizes exploratory and unsupervised analysis, rather than predictive performance or real-time deployment. Consequently, the dataset is not evaluated for detection accuracy, online response time, or operational security guarantees.

Finally, all experiments are conducted in a controlled indoor environment to ensure repeatability and consistent labeling. While this improves experimental reliability, it may not capture environmental variability such as outdoor interference, mobility-induced topology changes, or weather-related signal effects.

1.5. Significance of the Study

This study contributes to the growing body of research on wireless mesh networking using low-cost microcontroller platforms by providing a structured, hardware-derived dataset that captures cross-layer behavior in ESP32-based ESP-WIFI-MESH systems.

For researchers and students, the study offers a practical reference for building and instrumenting real ESP32 mesh testbeds, demonstrating how observable routing behavior can be captured beyond purely theoretical models or simulation-based environments. The dataset supports exploratory investigations into how normal and disrupted mesh behaviors differ across protocol layers.

More broadly, the study highlights methodological considerations in cross-layer data collection, including synchronization, logging constraints, and interpretation challenges when working with decentralized mesh

protocols. Rather than presenting definitive security solutions, the findings aim to support future analytical and methodological research on wireless mesh behavior.

In alignment with United Nations Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure), this work supports the advancement of resilient, low-cost communication infrastructure by improving the empirical understanding of wireless mesh networks deployed in resource-constrained environments.

1.6. Resources

1.6.1. Hardware

- **ESP32 Development Boards (10 units)**
    - The ESP32 microcontroller serves as the core hardware platform for constructing the experimental ESP-WIFI-MESH testbed and generating a realistic, hardware-based dataset. Its integrated Wi-Fi capability supports multi-hop mesh communication via the ESP-MESH protocol, enabling the observation of routing behavior, packet-forwarding dynamics, and cross-layer interactions under both normal and adversarial conditions. A network of up to 10 ESP32 nodes forms small- to medium-scale mesh topologies for controlled experimentation, traffic generation, and network metric logging.

      The ESP32 devices are used solely for data generation and behavior observation; no real-time intrusion detection is deployed on the nodes in this study.
- **Wi-Fi Router or Access Point (for configuration and baseline testing)**
    - While the final system operates without an infrastructure, a temporary Wi-Fi router will facilitate initial firmware deployment, synchronization, and debugging before full mesh operation is activated.
- **Laptop/Workstation (2–3 units)**
    - High-performance laptops equipped with multicore processors and at least 16 GB of RAM serve as the primary environment for log aggregation, dataset consolidation, exploratory data analysis (EDA), and baseline offline machine learning experiments. These systems are also used for experiment orchestration, visualization, and result documentation.
- **Monitoring and Diagnostic Tools**
    - Serial monitors and diagnostic logging interfaces are used to observe mesh events, node state transitions, packet counters, and timing behavior during experimental runs. These tools support validation of network stability, attack execution timing, and ground-truth labeling rather than real-time detection. Serial monitors and diagnostic logs will be used to measure latency and node responsiveness during real-time operation.

1.6.2. Software

This section describes the software environment used to implement, monitor, and validate the ESP32-based ESP-WIFI-MESH testbed. The selected tools support firmware development, cross-layer telemetry extraction, controlled experimentation with routing behavior, and dataset preprocessing for subsequent exploratory analysis. The software stack is designed to balance detailed metric collection with the processing constraints of resource-limited ESP32 devices.

The mesh network is implemented using the ESP-IDF framework with ESP-WIFI-MESH functionality enabled. Core mesh functions such as initialization, packet transmission, packet reception, and event monitoring are integrated into the application layer. Logging routines are refined to capture selected cross-layer metrics—such as forwarding behavior, RSSI observations, and routing events—while minimizing processing overhead to preserve normal mesh operation.

Additional tools include Wireshark for packet-level verification and traffic validation, and Python or MATLAB for preprocessing, feature extraction, and exploratory statistical analysis of the generated dataset.

- **Arduino IDE / ESP-IDF SDK**
    - The Arduino IDE and ESP-IDF SDK are used to develop and deploy firmware for ESP32 nodes implementing ESP-WIFI-MESH functionality, traffic generation, controlled attack behaviors, and cross-layer metric logging. The ESP-IDF SDK is also used for accessing low-level mesh APIs, event handlers, and diagnostic counters required for dataset generation.

- **Wireshark**
    - Wireshark is used as a supplemental packet-capture and verification tool to observe wireless traffic patterns, confirm packet-forwarding behavior, and support validation of attack effects. It is employed for traffic inspection and ground-truth verification rather than for evaluating detection accuracy.

- **C / Python / MATLAB**
    - C is used for implementing embedded logic on ESP32 nodes, including mesh operation, attack behavior control, and metric logging. Python scripts are used for dataset preprocessing, feature extraction, correlation analysis, and statistical evaluation during EDA and baseline offline ML experiments. MATLAB may be used for preliminary visualization and exploratory analysis of cross-layer feature relationships before finalizing the dataset.

2. Review of Related References

2.1. Wireless Mesh Networks (WMNs)

Wireless Mesh Networks (WMNs) represent an important evolution in wireless communication, designed to provide high adaptability, reliability, and scalability in dynamic environments. Akyildiz et al [1] describe WMNs as a self-organizing and self-configuring system where nodes automatically establish and maintain mesh connectivity with one another. This decentralized architecture eliminates the need for a central authority, allowing the network to operate autonomously. Mesh routers within WMNs form a self-healing web of connections, enabling the system to maintain communication even when certain nodes or links fail. This self-healing property ensures fault tolerance, as data can automatically reroute through alternate paths whenever disruptions occur.

WMNs are characterized as multi-hop networks in which information can travel through multiple intermediate nodes before reaching its destination. This feature allows them to cover large geographic areas with minimal infrastructure, reducing deployment costs while maintaining flexibility and scalability. As noted by both Akyildiz et al. [1] and Cilfone et al. [2], this multi-hop capability makes WMNs particularly suitable for applications requiring robust and dynamic connectivity, such as Internet of Things (IoT) systems, vehicular ad hoc networks (VANETs), disaster recovery communication, broadband home networking, and community or metropolitan area networks. Because they can dynamically reconfigure, WMNs are ideal for environments where reliable connectivity and rapid deployment are critical, including smart cities, industrial IoT, and remote monitoring systems.

According to Cilfone et al. [2], WMNs have become a key enabler of IoT due to their scalability, cost-effectiveness, and flexibility. Each node can act as both a host and a router, allowing seamless multi-hop communication without fixed infrastructure. These networks can self-configure, self-heal, and adapt to changing topologies, maintaining connectivity even when nodes move or fail. Such capabilities make WMNs an attractive communication backbone for large-scale, heterogeneous IoT environments and mobile systems.

Despite their advantages, both studies acknowledge that WMNs face significant challenges, particularly in security and network management. The open and distributed nature of WMNs exposes them to security vulnerabilities, as the lack of centralized control and reliance on shared wireless channels make them susceptible to data interception, unauthorized access, and routing attacks. Cilfone et al. [2] emphasized that these vulnerabilities are worsened by interoperability issues, limited node resources, and the heterogeneity of communication protocols in IoT systems. Akyildiz et al. [1] further noted that while WMNs exhibit strong self-organization and self-healing properties, achieving full automation in real-world deployments remains difficult due to constraints in routing algorithms, scalability, and resource management.

Given these limitations, there is a clear need for robust security mechanisms to protect WMNs from malicious activities. Although Cilfone et al. [2] do not provide a detailed discussion of Intrusion Detection Systems (IDSs), their identification of security weaknesses underscores the importance of IDS implementation in WMNs. IDSs play a crucial role in detecting, analyzing, and responding to network intrusions, thereby enhancing the trustworthiness and resilience of these decentralized, multi-hop systems

Author and Title	Description	Approach	Contribution
Akyildiz, Wang & Wang (2005). Wireless mesh networks: A survey (Computer Networks)	Comprehensive survey of WMN architectures, routing, scalability, and fundamental design challenges.	Literature review and conceptual analysis of WMN architecture, self-healing features, and multi-hop capabilities	Defines WMN architecture & roles (mesh routers/clients), and characterizes multi-hop/self-healing properties.
Cilfone, Davoli, Belli & Ferrari (2019). Wireless mesh networking: An IoT-oriented perspective, Survey on relevant technologies (Future Internet)	Updated IoT-oriented survey covering WMN use in heterogeneous IoT deployments: protocols, interoperability, and application scenarios.	Systematic analysis of IoT integration with WMNs; evaluation of node behavior, network resilience, and self-configuration mechanisms	Positions WMNs as enablers for IoT and smart-city applications, also highlighting interoperability, resource-constraint, and heterogeneity issues in IoT contexts.
Al-Anzi (2022). Design and analysis of intrusion detection systems for wireless mesh networks (Digital Communications & Networks)	Examines IDS design choices and limitations for WMNs, particularly the constraints of signature-based systems and the centralization issues.	Analysis and proposal of lightweight signature-based IDS optimizations for WMNs.	Shows limitations of signature-based IDS for adaptive/zero-day threats, and exposes centralization bottlenecks and scalability problems.
Hanif et al. (2022). AI-Based Wormhole Attack Detection Techniques in WSNs (Electronics)	Systematic review of ML techniques used to detect wormhole attacks (relevant to WMN routing-layer threats).	Systematic review comparing ML classifiers (SVM, DT, DNN) and evaluation datasets.	Shows ML methods often outperform classical approaches for wormhole detection.
Bhatti, Saleem, Imran et al. (2024). Detection and isolation of wormhole nodes in wireless ad hoc networks based on post-wormhole actions (Scientific Reports)	Proposes a behavior-based framework to detect and isolate wormhole nodes using observed post-attack behaviors.	Behavior analysis method with post-wormhole metrics; prototype evaluation on ad hoc networks.	Demonstrates high accuracy with low overhead using behavior-based features.

Table 2.1 Summary of Overview of Wireless Mesh Networks

2.2. Security Challenges and Common Attacks in WMNs

Wireless Mesh Networks (WMNs) are widely used for decentralized communication but remain vulnerable to various routing-layer attacks such as blackhole, grayhole, Sybil, and wormhole. In these attacks, malicious nodes manipulate routing paths by absorbing, selectively dropping, or tunneling packets to disrupt data transmission. They are particularly difficult to detect because they often mimic normal node behavior, rendering traditional signature- or rule-based defenses unreliable. This challenge highlights the need for adaptive, learning-based intrusion detection systems (IDSs) capable of dynamically analyzing network patterns.

Blackhole attacks represent a fundamental routing-layer threat in which a malicious node first positions itself on active data paths by advertising favorable routing metrics, such as low hop count or strong signal quality, to attract traffic from neighboring nodes. Once established as an intermediate forwarder, the node

silently drops all received packets rather than relaying them to their destinations. Bhonsle et al. [20] demonstrated that deep learning approaches using Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks can effectively detect blackhole attacks in WMNs by analyzing network traffic patterns, achieving significant improvements in detection rates compared to traditional rule-based systems. Their work on the NSL-KDD dataset showed reduced false positives and adaptable detection capabilities suitable for dynamic mesh environments. More recently, Vanlalhruaia et al [18] proposed a detection mechanism for AODV-based WMNs that utilizes destination sequence number comparison and hop count verification to identify inconsistencies caused by blackhole nodes, demonstrating effective mitigation of attack impacts through simulation-based analysis.

Wormhole attacks pose an equally serious threat through a distinct mechanism: two colluding malicious nodes establish a low-latency out-of-band tunnel to relay packets between distant parts of the network, creating the illusion that physically remote nodes are immediate neighbors. Hanif et al. [3] conducted a systematic review of AI-based wormhole detection methods in Wireless Sensor Networks, showing that machine learning approaches such as SVMs, decision trees, and deep neural networks achieved higher detection accuracy than traditional cryptographic or hop-count methods. However, they noted that many models rely on simulated datasets and are not fully optimized for real-world deployment. Alshehri [13] proposed a hybrid SVM–DNN model for IoT and WSN environments that efficiently detects wormhole attacks without requiring additional hardware, achieving high accuracy and low latency while highlighting the need for adaptive learning mechanisms to maintain performance under dynamic network conditions. Bhatti et al [4] introduced a behavior-based wormhole detection framework that identifies malicious nodes based on their post-attack actions, such as abnormal forwarding delays or altered routes, rather than tunnel characteristics. This distributed method achieved high accuracy with minimal overhead, making it suitable for real-time mesh networks.

Recent advances in cross-layer defense mechanisms show promise for simultaneously addressing both attack types. Srinivasan et al. [19] proposed an innovative cross-layer framework that integrates knowledge from the physical, MAC, and network layers to effectively mitigate wormhole and blackhole attacks in wireless ad hoc networks. Their Enhanced Support Vector Machine (E-SVM) algorithm, implemented in NS3, demonstrated superior performance across multiple metrics, including packet delivery ratio (89.28%), dropping ratio (12.5%), and energy consumption, outperforming traditional SVM techniques across various network sizes. Collectively, these studies underline that machine learning, behavioral analysis, and cross-layer approaches offer promising solutions for detecting complex routing-layer attacks in WMNs. By enabling nodes to learn and adapt, these approaches pave the way for intelligent, energy-efficient, and resilient network defense systems. However, the persistent reliance on simulated datasets underscores the need for hardware-generated data from real mesh deployments.

Author and Title	**Description**	**Approach**	**Contribution**
Hanif et al. (2022). AI-Based Wormhole Attack Detection Techniques in WSNs (Electronics)	Analyzing low-powered network detection methods	Comparison of hop count, Machine learning, and cryptographic approaches	Shows Machine Learning, which generally outperforms traditional methods

Alshehri (2024). Wormhole detection and mitigation model for IoT/Wireless Sensor Network using Machine Learning	A Hybrid Machine Learning model that spots tunneling behaviors	SVM + DNN pipeline	High accuracy and low latency stress are needed for adaptive learning without the need for extra hardware
Bhatti et al. (2024). Detection & isolation of wormhole nodes based on post-attack actions	Detects the behavior of malicious relays after the attack	Distributed based on behavior features	Accurate and low-overhead detection that should be applicable for mesh; also, isolation of attackers
Bhonlse, M, et al. (2025). Deep Learning-Based Defense Mechanisms against black hole attacks in wireless mesh networks	Investigates deep learning approaches for detecting blackhole attacks through network traffic pattern analysis	RNN and LSTM networks evaluated on the NSL-KDD dataset	Demonstrates significant improvements in detection rates compared to traditional rule-based systems; reduces false positives in dynamic mesh environments.
Vanlalhruaia, et al (2025). Securing AODV against black hole attack for wireless mesh network	Proposes a detection mechanism specifically for AODV-based WMNs	Destination sequence number comparison and hop count verification to identify routing inconsistencies	Demonstrates effective mitigation of blackhole attack impacts through simulation-based analysis
Srinivasan, J. (2025). Innovative cross-layer defense mechanisms for blackhole and wormhole attacks in wireless ad-hoc networks	Proposes a cross-layer framework integrating knowledge from physical, MAC, and network layers	Enhanced Support Vector Machine (E-VSM) algorithm implemented in NS3	Achieves superior performance across delivery ratio (89.28%), dropping ratio (12.5%), and energy consumption; outperforms traditional SVM across various network sizes.

Table 2.2 Summary of Security Challenges and Common Attacks in WMNs

2.3. Infrastructure-less and Decentralized Network Environments

According to Al-Anzi [5], such autonomy and flexibility make WMNs highly adaptable for real-time applications such as disaster recovery, vehicular networks, and remote monitoring. However, the same characteristics that promote flexibility also introduce significant security challenges. Without a central authority to validate identities or enforce policies, malicious nodes can more easily infiltrate the network, manipulate routing information, or disrupt data transmission.

The decentralized nature of these environments complicates the implementation of traditional Intrusion Detection Systems (IDS). Conventional IDS models often depend on centralized monitoring nodes that aggregate traffic data and analyze it for malicious behavior. As Al-Anzi [5] notes, this centralized approach creates bottlenecks and single points of failure, which are unsuitable for large-scale, dynamically changing WMNs. To overcome these limitations, security mechanisms must be distributed and cooperative, where individual nodes participate in both monitoring and decision-making.

Ioulianou et al. [6] emphasized this need in their study on collaborative frameworks for IoT networks using the IPv6 Routing Protocol for Low-Power and Lossy Networks (RPL). Their results showed that cooperative and reputation-based systems outperform centralized detection methods when dealing with combined Rank and Blackhole attacks. By assigning scores to each node and allowing nearby peers to validate these scores, the system achieved improved detection accuracy and reduced false alarms. Nonetheless, their framework also revealed the cost of decentralized cooperation: increased communication overhead and energy consumption due to frequent metric exchanges.

Pedroso et al. [9] proposed a collaborative IDS architecture for IoT integration, in which nodes communicate summarized anomaly indicators rather than raw data. This distributed learning and consensus process maintains the decentralized structure while limiting communication overhead.

Collectively, these studies underscore a paradigm shift in how security is maintained in infrastructure-less environments. Instead of relying on centralized servers, emerging approaches distribute intelligence and evaluation across participating nodes. Such designs ensure scalability, fault tolerance, and adaptability—core requirements for WMNs and IoT systems operating under limited energy and computational resources.

Author and Title	Description	Approach	Contribution
Al-Anzi (2022). Design & analysis of IDS for WMNs	Wireless mesh networks security & IDS constraints	IDS analysis focused on Signatures	Identifies resource centralization limits
Pedroso et al. (2024). A direct collaborative network intrusion detection system for IoT network integration	Cooperative detection across multiple nodes	Lightweight summaries and consensus	Improving accuracy and latency also limits raw data exchange overhead

Table 2.3 Summary of Literature on Infrastructure-less/Decentralized Environments

2.4. Intrusion Detection Systems (IDS) for WMNs

The development of Intrusion Detection Systems (IDS) has been a primary focus for securing Wireless Mesh Networks (WMNs) against routing-based attacks. However, the effectiveness of any IDS—whether signature-based, anomaly-based, or distributed—is fundamentally constrained by the quality and realism of the data used to build and validate it.

Al-Anzi [5] provides a foundational analysis of IDS design in WMNs, demonstrating that while signature-based detection achieves high accuracy against known threats, it exhibits critical limitations. Specifically, such systems fail to detect novel or adaptive routing misbehavior, including variations of blackhole and selective forwarding attacks, because they rely on pre-defined attack databases. This "closed-world" assumption renders signature-based IDS ineffective against zero-day exploits in dynamic mesh environments.

Furthermore, Al-Anzi highlights that resource constraints inherent to mesh nodes—limited processing power, memory, and energy—complicate the deployment of comprehensive monitoring. Centralized IDS

architectures introduce single points of failure and detection latency, while distributed collaborative designs, though more resilient, increase communication overhead through frequent metric exchanges (Ioulianou et al., 2022; Pedroso et al., 2024).

These findings underscore a critical gap: intrusion detection research has advanced IDS architectures, but the underlying data used for training and evaluation has not kept pace. The limitations identified—inability to detect novel attacks, resource constraints, and the need for distributed intelligence—are not solely architectural problems. There are also data problems. Signature-based systems fail because no dataset captures the behavioral signatures of new attacks. Distributed systems struggle because no empirical baselines exist for normal cross-layer behavior against which anomalies can be collaboratively measured.

Therefore, the primary barrier to improved IDS for ESP32-based mesh networks is not the lack of detection algorithms, but the lack of hardware-derived datasets that capture realistic cross-layer behavior. Al-Anzi's analysis of IDS limitations directly motivates the present study: before effective intrusion detection can be designed, the community requires a foundational, empirical dataset documenting how routing manipulations manifest across PHY, MAC, and Network layers in real hardware.

By generating such a dataset under controlled normal and attack conditions, this study provides the essential empirical foundation for future IDS research—whether signature-based, anomaly-based, or distributed—to be built and validated.

Author and Title	Description	Approach	Contribution
Al-Anzi (2022). Design & analysis of IDS for WMNs	Baseline analysis of Wireless Mesh Network threats and Intrusion Detection System	Signature-based, centralized learning	Makes things clear on the accuracy of known attacks, also exposes zero-day blindness and the risks of centralization

Table 2.4 Summary of Intrusion Detection Systems (IDS) for WMNs

2.5. Machine Learning and Anomaly Detection in IDS

The integration of machine learning (ML) into Intrusion Detection Systems (IDS) has transformed how modern networks detect and respond to cyber threats. Traditional IDS often relies on static, signature-based methods that fail to recognize new or zero-day attacks. In contrast, ML models such as autoencoders and deep neural networks can learn patterns of normal network behavior and flag deviations as potential intrusions. This anomaly-based detection approach allows systems to identify unknown threats, adapt to evolving attack strategies, and minimize false positives. Rassam [41] demonstrated this using an autoencoder-based neural network for anomaly detection in Wireless Body Area Networks (WBANs), achieving high accuracy in identifying abnormal traffic at minimal computational cost. The study highlights how unsupervised learning enables IDS to detect subtle deviations without labeled datasets, making it ideal for resource-constrained IoT environments.

Expanding on this concept, Hernandez-Ramos et al. [40] conducted a systematic review of federated learning (FL) in IDS, emphasizing privacy-preserving collaboration across distributed devices. Instead of transmitting raw data, FL enables multiple IDS nodes to train models locally and share only learned parameters, thus maintaining data confidentiality while improving collective detection performance. The

review concluded that FL-enhanced IDSs can significantly improve detection accuracy and scalability but still face challenges in communication efficiency, data imbalance, and model drift. Similarly, Pedroso et al. [39] introduced a collaborative network intrusion detection system (C-NIDS) for IoT integration, demonstrating that cooperative ML-based nodes achieved up to 99% detection accuracy and faster response times than isolated systems.

Together, these studies affirm that machine learning–driven anomaly detection, particularly through autoencoders, neural networks, and federated learning, enables IDS to evolve from static monitoring tools into adaptive, intelligent, and distributed defense systems. By learning normal behavior and securely sharing knowledge across devices, these models provide a robust framework for detecting both known and unknown attacks in complex network environments, such as IoT and Wireless Mesh Networks.

Author and Title	Description	Approach	Contribution
Rassam et al. (2024). Autoencoder-based neural network model for anomaly detection in IoT networks	Unsupervised anomaly detection with low computational costs	Autoencoders	High accuracy with minimal labels that are ideal for constrained nodes
Hernandez-Ramos et al. (2022–2025). Intrusion detection based on federated learning: A comprehensive survey	Privacy-centric collaborative IDS	Federated Learning across devices	Better accuracy and scalability. Also notes communication, imbalance, and drift challenges
Pedroso et al. (2024). A direct collaborative network intrusion detection system for IoT network integration	Peer cooperative systems with compact features	Cooperative Machine Learning	Up to ~99% accuracy, faster response vs isolated nodes

Table 2.5 Summary of ML/Anomaly-Based IDS

2.6. Cross Layer

Cross-layer intrusion detection systems (IDSs) have emerged as a solution to the limitations of single-layer IDS models, which typically monitor only one layer of the network stack—such as the physical, MAC, or network layer—leaving other attack vectors undetected. Full cross-layer IDS frameworks aggregate parameters from multiple layers to improve detection coverage, enabling the system to observe correlations across events in different layers of the network stack. For example, Al-Anzi [5] developed a full cross-layer IDS for Wireless Mesh Networks (WMNs) that collected features including node forwarding rate, routing table changes, packet drop count at the network layer, and RSSI variance at the physical layer to detect attacks such as flooding and battery exhaustion. While this approach increased the ability to detect attacks across multiple stages, Al-Anzi [5] acknowledged several limitations: processing overhead and synchronization delays significantly impacted performance, and early-stage attacks were often missed. Specifically, during a data-link layer flooding attack, detection was delayed because inter-layer parameters needed to be fully exchanged before the system could flag an anomaly. Likewise, the first three battery-exhaustion attacks were undetected because physical-layer metrics could not be promptly transmitted to the network layer. This study demonstrates that although full

cross-layer IDSs enhance threat visibility compared to single-layer systems, the trade-off is increased latency and inefficiency, particularly on resource-constrained devices such as ESP32 nodes.

To mitigate these inefficiencies, recent research has emphasized optimized cross-layer feature selection, focusing only on the most relevant metrics rather than collecting all possible parameters from every layer. Amouri et al. [8] proposed a two-stage, anomaly-based cross-layer IDS for WSN and MANET environments. In their design, Stage 1 involved monitoring MAC-layer frames and neighboring node forwarding counts using promiscuous-mode sniffers, while Stage 2 applied a combination of decision-tree classifiers and linear regression on an accumulated-fluctuation metric (AMoF) derived from these features. This method enabled accurate detection even under high-mobility conditions and low-power constraints, highlighting the practical benefits of selective cross-layer monitoring. Similarly, Singh [9] presented an optimized feature-selection scheme that reduced a large set of cross-layer parameters—including retransmission count, route change frequency, and neighbor churn—to a minimal subset without compromising detection performance. This study emphasized that excessive parameter collection increases computational load and latency, and that careful selection of high-value features can achieve nearly the same detection accuracy with far lower resource consumption.

Li et al. [42] further explored the trade-offs between feature selection and feature extraction in IoT intrusion detection. Using the TON-IoT dataset, they found that feature extraction, which creates new derived features from existing data, could improve detection accuracy when the total number of features was extremely limited. However, feature selection, which identifies the most relevant raw features, produced faster inference times and lower computational overhead, making it more suitable for devices with limited memory and processing power. This work underscores the importance of balancing detection performance with system efficiency, particularly in resource-constrained environments. Across these studies, certain cross-layer metrics repeatedly emerge as high-value indicators for intrusion detection, including packet drop rate, retransmission count, RSSI variance, node forwarding behavior, route change frequency, and network table modifications. By focusing on these parameters, systems can retain most of the benefits of cross-layer monitoring while avoiding unnecessary computational and communication overhead.

Collectively, these studies reveal a clear pattern: while full cross-layer IDSs provide comprehensive detection coverage, the cost in terms of processing time, synchronization delays, and communication overhead often renders them impractical for real-time detection in low-power IoT and WMN devices, such as microcontroller-class ESP32 nodes. Optimized cross-layer feature selection offers a pragmatic alternative by identifying and monitoring only a targeted set of the most relevant parameters across layers. This insight directly informs the design of a lightweight IDS framework: instead of attempting to capture all parameters across the network stack, the proposed system will focus on a carefully chosen set of cross-layer features—such as packet drop rate, retransmission count, RSSI variance, route change frequency, and node forwarding rate—to enable rapid, resource-efficient anomaly detection. By leveraging these high-value features, this approach aims to achieve a balance between detection effectiveness and operational efficiency, ensuring that real-time intrusion detection remains feasible in multihop mesh networks composed of resource-constrained devices [5,8,9,42].

Author and Title	Description	Approach	Contribution

Al-Anzi, F. S. (2022). Design and analysis of intrusion detection systems for wireless mesh networks	Proposed a full cross-layer IDS for WMNs that integrates parameters from multiple layers of the network stack	Collected physical, MAC, and network-layer metrics such as RSSI variance, routing table changes, and packet drop rate	Enhanced visibility of multi-layer attacks, and identified synchronization delays and processing overhead as major constraints in resource-limited nodes
Amouri, A., et al. (2018). A Cross-Layer, Anomaly-Based IDS for WSN and MANET,	Introduced a two-stage anomaly-based cross-layer IDS for WSN and MANET	Monitors MAC-layer frames and forwarding behavior, and applies decision-tree and regression models	Achieved accurate detection under high mobility and low-power conditions
Singh, G. (2022). A Cross-Layer Based Optimized Feature Selection Scheme for IDS in WSNs	Presented an optimized feature-section model to reduce redundant cross-layer parameters	Used heuristic ranking to select minimal, high-value metrics	Maintained high detection accuracy with a significant reduction in computational load.
Li, J., et al. (2024). Optimizing IoT Intrusion Detection System: Feature Selection versus Feature Extraction	Compared the performance of feature selection and feature extraction strategies in IoT IDS	Evaluated both methods using the TON-IoT dataset to assess accuracy, inference time, and resource use	Provided an empirical basis for lightweight IDS feature optimization, and found that feature selection offers faster inference and lower overhead

Table 2.6 Summary of Cross-Layer

2.7. ESP32 Technical Literature

In recent research, the ESP32 microcontroller platform and its native mesh networking protocol (ESP-WIFI-MESH) have been increasingly adopted in wireless IoT systems due to their self-organizing, self-healing, and multi-hop communication capabilities. The ESP-WIFI-MESH protocol, provided as part of the official Espressif ESP-IDF framework, enables ESP32 devices to form a tree-based mesh topology in which nodes simultaneously act as both stations and access points, facilitating upstream and downstream links without requiring traditional Wi-Fi infrastructure (Espressif Systems, n.d.).

Empirical evaluations in open-access literature demonstrate the applicability of ESP-WIFI-MESH in real deployment scenarios. For example, Khan et al [25] deployed an ESP-MESH network using ESP32 nodes for environmental monitoring, showing that the protocol supports multi-hop packet forwarding with high delivery rates exceeding 97% and low packet error and loss rates below 1.8%, underscoring its effectiveness for medium-scale mesh networks in real environments. Khan et al. [25] also highlight the self-healing nature of the ESP-MESH protocol, in which nodes autonomously reorganize around failures and link variations without centralized routing control.

Beyond environmental sensing, recent open-access works also integrate ESP-WIFI-MESH in other IoT applications, illustrating its adaptability and ease of implementation. For instance, Qian et al. [26] developed a multi-bus instrument communication platform using ESP-WIFI-MESH to establish a robust multi-hop ad hoc wireless network, reporting rapid network establishment under 10 seconds and self-repair capabilities in the face of node failures, further validating the mesh protocol's utility for real-world IoT connectivity challenges.

The ESP32 platform offers distinct advantages for experimental network research and dataset generation, thanks to its programmability, built-in Wi-Fi, and support for detailed runtime logging. Khan et al. [25] demonstrated, through extensive performance evaluation, that ESP-MESH networks can reliably report packet delivery ratios, loss rates, and error rates, metrics essential for behavioral analysis. Their work confirms that ESP32-based testbeds can produce repeatable, quantifiable network data suitable for subsequent analysis. Similarly, Qian et al [26] validated the platform's empirical research value through their implementation of a multi-hop ESP-WIFI-MESH system, capturing network establishment times and self-repair capabilities under controlled conditions.

Collectively, these studies establish that ESP-WIFI-MESH on the ESP32 platform provides a low-cost, scalable, and well-characterized foundation for building experimental mesh network testbeds. The platform's combination of self-organizing protocol behavior, accessible hardware, and documented performance metrics makes it highly suitable for systematic dataset generation and behavioral analysis in wireless intrusion detection research. By enabling controlled observation of both normal and anomalous routing behaviors, ESP32-based testbeds address a critical gap identified in the literature: the absence of hardware-generated, cross-layer datasets from real mesh deployments [24][25][26]

Author and Title	**Description**	**Approach**	**Contribution**
Espressif. Systems (n.d.). ESP-WIFI-MESH Programming Guide	Official programming guide and API documentation for the ESP-WIFI-MESH protocol	Technical documentation of mesh architecture, node roles (root/intermediate/leaf), control and data plane operations, parent selection criteria, and RSSI thresholding mechanisms	Defines the protocol behavior, forwarding requirements, and configuration parameters that form the baseline for understanding normal mesh operation and potential exploitation vectors
Khan, A. et al. (2022). An Efficient Wireless Sensor Network Based on the ESP-MESH Protocol for Indoor and Outdoor Air Quality Monitoring.	Empirical evaluation of ESP-MESH performance using ESP32 nodes for environmental monitoring applications	Deployed a multi-hop ESP-MESH network; measured packet delivery ratio, packet loss rate, packet error rate, and self-healing capabilities under real indoor/outdoor conditions	Demonstrates 97%+ packet delivery ratio and <1.8% loss/error rates; confirms protocol supports reliable multi-hop forwarding and autonomous reorganization around failures; validates ESP32 testbeds as suitable for generating repeatable, quantifiable network data
Qian, L. et al. (2024). A Wireless Ad Hoc Network Communication Platform and Data Transmission Strategies for Multi-Bus Instruments	Implementation of ESP-WIFI-MESH for multi-bus instrument communication in industrial contexts	Developed robust multi-hop ad hoc wireless network; measured network establishment time, self-repair capabilities, and transmission reliability under controlled conditions	Reports rapid network formation (<10 seconds) and effective self-healing during node failures; validates ESP-MESH utility for real-world IoT connectivity challenges; confirms platform's value for empirical research through capture of

			establishment times and recovery behaviors

Table 2.7 Summary of ESP32 Technical Literature

2.8. Existing Wireless Network Datasets

Several public datasets have supported the development of intrusion detection systems for wireless networks, yet most remain unsuitable for studying routing-based misbehavior in microcontroller mesh deployments. Simulation-based datasets such as NSL-KDD (Tavallaee et al., 2009) and UNSW-NB15 (Moustafa & Slay, 2015) have been widely used as benchmarks for network intrusion detection. However, these datasets are derived from simulated traffic and lack the radio-frequency dynamics, hardware-specific constraints, and cross-layer interactions inherent to physical wireless mesh networks. They cannot capture phenomena such as received signal strength variation, retransmission behavior, or parent switching dynamics that arise from real environmental conditions and device-level resource limitations. Despite these limitations, feature extraction approaches from these datasets inform the present study. Specifically, Das et al [29] demonstrated the effectiveness of network port statistics for blackhole detection in their UNR-IDD dataset, providing a basis for selecting forwarding-related features such as packet counters and traffic asymmetry indicators. Similarly, Avina and Geetha [27] employed a ResNeXt and deep stacked autoencoder framework for wormhole attack detection, highlighting the importance of capturing topology-related anomalies, a principle that motivates the inclusion of hop count consistency and parent-child relationship features in the current cross-layer collection framework.

Hardware-derived datasets offer greater realism but remain limited to specific protocols and attack scenarios. Boiano et al [28] provide a Zigbee mesh dataset collected from 21 commercial devices, supporting IoT network forensic analysis. However, the authors note that models trained on one topology degrade significantly when tested on others, underscoring the need for topology-aware dataset design. Miekkala et al. [31] present cross-border QoS measurements in 5G and beyond networks, capturing physical and network-layer metrics, but in cellular infrastructure contexts rather than decentralized mesh architectures. Neither dataset addresses the ESP-WIFI-MESH protocol, routing-based attacks such as blackhole and wormhole, or the resource constraints of microcontroller-class devices.

Collectively, existing datasets fail to provide (1) ESP-WIFI-MESH protocol behavior, (2) cross-layer metrics spanning PHY, MAC, and network layers from resource-constrained nodes, (3) documented routing-based attack scenarios in real hardware, and (4) multiple topology configurations for comparative analysis. The present study directly addresses these gaps through a hardware-derived ESP32 mesh dataset, with feature selection informed by the detection-relevant indicators identified in Das et al. [29] and Avina and Geetha [27].

Author and Title	Description	Approach	Contribution
A, Avina & R, Geetha. (2025). ResNeXt and Deep Stacked Autoencoder-based framework for wormhole attack detection on network control system	Framework for wormhole attack detection in network control systems using deep learning architectures	ResNeXt combined with a Deep Stacked Autoencoder for feature learning and classification; evaluated on network control system data	Highlights the importance of capturing topology-related anomalies for wormhole detection; motivates the inclusion of hop count consistency and parent-child relationship features in the current

			cross-layer collection framework
Boiano, A., et al.(2026, February 3). Analyzing Zigbee Traffic: Datasets, classification, and storage trade-offs	Introduction of ZIOTP2025 Zigbee mesh dataset collected from 21 commercial devices for IoT network forensic analysis	Collection of Zigbee traffic in mesh topology configurations; analysis of classification performance and storage requirements	Reveals that models trained on one topology degrade significantly when tested on others; underscores the need for topology-aware dataset design in mesh network research
Das, T., et al. (2023, January). Unr-idd: Intrusion detection dataset using network port statistics	Development of an intrusion detection dataset specifically focused on network port statistics for attack detection	Collection of network port statistics under normal and attack conditions; emphasis on statistical features rather than packet payload inspection	Demonstrates the effectiveness of network port statistics for blackhole detection; provides a basis for selecting forwarding-related features such as packet counters and traffic asymmetry indicators in the current study
Moustafa, Nour & Slay, Jill. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)	Creation of a modern network intrusion dataset capturing contemporary attack patterns and normal traffic behaviors	Hybrid of realistic synthetic traffic generation and real network captures; includes nine families of attacks with associated features	Addresses limitations of older datasets (KDD'99, NSL-KDD) by including modern attack types and more representative network traffic; provides a comprehensive feature set for IDS evaluation
Miekkala, T., et al. (2024). NordicDat: A Cross-Border Predictive QoS Dataset	Cross-border QoS dataset for 5G and beyond networks, capturing physical and network-layer metrics	Collection of QoS measurements across Nordic country borders; includes physical layer metrics and network performance indicators	Provides valuable cross-layer metrics but in cellular infrastructure contexts; demonstrates the feasibility of hardware-derived QoS datasets while highlighting the absence of decentralized mesh architecture data
Tavallaee, M., et al. (2009). A detailed analysis of the KDD CUP 99 data set	Critical analysis and improvement of the original KDD'99 dataset for intrusion detection benchmarking	Statistical analysis of dataset redundancies and difficulties; proposed NSL-KDD as a refined version addressing the inherent problems of the original KDD'99	Provides a widely-used benchmark dataset with reduced redundancy and balanced record distribution; enables fairer evaluation of intrusion detection systems; widely cited as a baseline for network intrusion detection research

Table 2.8 Summary of Existing Wireless Network Datasets

2.9. Dataset Design and Feature Engineering

Constructing meaningful datasets for network behavior analysis requires careful attention to experimental design, data collection, and feature representation. Poorly designed datasets introduce biases, fail to capture real-world dynamics, and undermine subsequent analysis, regardless of the sophistication of the techniques (Sommer et al, 2010). Sommer et al. [30] critiqued the "closed-world assumption" in dataset construction, arguing that controlled laboratory conditions fail to capture the operational network complexity. They emphasized that representative data collection, rigorous ground-truth labeling, and thorough experimental documentation are essential prerequisites for meaningful analysis. This directly supports the present study's methodology of generating hardware-based ESP32 mesh data rather than relying solely on simulation.

Tavallaee et al. [32] analyzed the weaknesses of the KDD Cup 99 dataset, including redundancy, imbalance, and unrealistic traffic representation. They proposed improved preprocessing techniques that underscore the need for careful log aggregation, noise filtering, and exploratory analysis before pattern detection. Ring et al [34] surveyed network intrusion datasets, identifying common issues in traffic generation, attack diversity, and cross-layer observability, validating the need for modern, hardware-derived mesh datasets like the one developed in this study. Garcia-Teodoro et al. [35] emphasized that effective anomaly modeling depends on well-structured datasets and meaningful feature representations, which support the cross-layer metric selection adopted here.

Feature engineering is equally critical for capturing network behavior. Akyildiz et al. [1] established foundational characteristics of mesh networks, including hop count, link quality, and routing consistency, thereby grounding the selection of topology-based features such as hop count dynamics and parent switching frequency. Cilfone et al. [2] examined mesh networking in IoT environments, highlighting link-quality indicators and packet-level observables that enable the extraction of RSSI stability, packet delivery ratio, and retransmission counts without heavy payload inspection, critical given ESP32 resource constraints. Tsai et al. [36] reviewed the importance of feature representations, noting that poor feature engineering undermines modeling performance regardless of the algorithm, reinforcing the need for careful cross-layer feature examination. Chandola et al. [37] emphasized statistical profiling and feature transformation as critical preprocessing steps, strengthening the exploratory analysis approach adopted before any clustering. Building on this, the principles of feature selection, specifically the identification and removal of redundant or low-variance attributes to improve model performance and interpretability, are well-established in data mining literature (Aggarwal [36], Han et al. [37]). The present study applies these principles by incorporating a systematic feature relevance analysis, including variance thresholding and correlation matrices, to ensure the dataset is optimized for subsequent unsupervised clustering.

Collectively, these studies establish that dataset design must balance completeness with resource constraints, cross-layer feature selection enables behavioral characterization without excessive overhead, and ground-truth labeling is essential for validating dataset utility. The present study applies these principles to construct a hardware-derived ESP32 mesh dataset capturing routing behavior under normal and adversarial conditions, with feature selection informed by detection-relevant indicators from the literature.

Author and Title	Description	Approach	Contribution
Tavallaee, M., et al. (2009). A detailed analysis of the KDD CUP 99 data set	Evaluation of the KDD Cup 99 dataset weaknesses, including redundancy, imbalance, and unrealistic traffic	Statistical analysis of dataset characteristics; proposed preprocessing improvements	Reinforces the need for preprocessing steps (log aggregation, noise filtering) and EDA before clustering to avoid biased interpretations
Garcia-Teodoro et al. (2009) Anomaly-Based Network Intrusion Detection	Survey of anomaly-based IDS techniques in network security	Statistical profiling and anomaly modeling across network layers	Highlights the importance of cross-layer observability and behavioral consistency for intrusion detection
Tsai, C, et al.. (2009). Intrusion detection by machine learning: A review	Review of feature representation importance in network anomaly detection	Review of feature representation importance in network anomaly detection	Reinforces the need for detailed feature engineering and examination of cross-layer interactions before clustering
Ring, M., et al. (2019). A survey of network-based intrusion detection data sets	Comprehensive review of existing network intrusion detection datasets and common limitations	Survey methodology identifying issues in traffic generation, diversity, and cross-layer observability	Validates dataset-first contribution; situates ESP32 cross-layer dataset within broader research gap of hardware-generated mesh data
Chandola, V, et al. (2009). Anomaly Detection: A Survey.	Comprehensive survey of anomaly detection methodologies	Review of statistical profiling and feature transformation techniques	Strengthens the EDA-first approach; shows that statistical understanding is essential before modeling
Akyildiz, I. F., et al. (2005). Wireless Mesh Networks: A Survey	Foundational survey of mesh network architecture, routing behavior, and multi-hop dynamics	Literature review and conceptual analysis of mesh network characteristics	Supports selection of topology-based features (hop count dynamics, parent switching frequency) grounded in mesh network theory
Cilfone, A., et al. (2019). Wireless Mesh Networking: An IoT-Oriented Perspective	Examination of mesh networking in IoT environments with a focus on reliability and cross-layer challenges	Analysis of link quality indicators and packet-level observables in resource-constrained systems	Supports the extraction of RSSI stability, packet delivery ratio, and retransmission counts without heavy payload inspection
Sommer, et al. (2010) - Outside the Closed World: On Using Machine Learning for Network	Critical examination of machine learning limitations in network intrusion detection	Analysis of closed-world assumptions and realism gaps in existing datasets	Justifies the need for hardware-based dataset generation and careful experimental

Intrusion Detection. Proceedings	without carefully designed datasets		documentation; supports methodological decision to generate physical ESP32 mesh data

Table 2.9 Summary of Dataset Design and Feature Engineering

2.10. Exploratory Data Analysis and Clustering in Network Telemetry

Exploratory Data Analysis (EDA) provides a foundational approach to understanding the structure of a dataset before formal modeling or hypothesis testing. Tukey [38] formally introduced EDA as a philosophy focused on discovering underlying structure, detecting anomalies, and testing assumptions through visualization and summary statistics rather than confirmatory techniques. This framework is particularly relevant to network telemetry analysis, where distributed system behavior is often non-linear, cross-layer, and context-dependent. Tukey's work supports the present study's decision to conduct a dedicated EDA stage to examine cross-layer mesh metrics before clustering, ensuring that the behavioral characteristics of the ESP-WIFI-MESH network are understood before structural assumptions are imposed.

Building on EDA principles, Aggarwal [36] discusses essential preprocessing steps before clustering or classification, including data cleaning, feature transformation, normalization, and correlation analysis. These steps are critical in network telemetry contexts where distributed node logs may suffer from timestamp drift, inconsistent sampling intervals, or partial packet records. Aggarwal's emphasis on understanding feature relationships before algorithmic application directly supports this study's workflow of log aggregation, timestamp alignment, feature scaling, and cross-layer correlation analysis, all performed before applying unsupervised methods. Similarly, Han et al. [40] emphasize descriptive statistics, distribution analysis, and visualization to understand dataset structure, thereby justifying the inclusion of topology-dependent visualization and correlation analysis in the exploratory phase of the present research.

In networked and IoT environments, EDA is often complemented by unsupervised anomaly detection techniques because labeled attack datasets are scarce. Chatterjee et al. [48] survey anomaly detection methods in IoT systems and note that unsupervised approaches are particularly valuable when labeled ground truth is incomplete or unavailable. Their review categorizes detection methods across statistical, machine learning, and hybrid techniques, highlighting the practicality of baseline modeling in dynamic IoT environments. Similarly, broader anomaly detection literature emphasizes deviation-based modeling in high-dimensional systems where attack signatures may not be predefined [34]. These perspectives support the study's reliance on behavioral baselining rather than supervised intrusion classification.

Further strengthening this direction, Miguel-Diez et al. [49] provide a systematic literature review of unsupervised anomaly detection in network flow analysis. Their PRISMA-based review documents the use of density-based methods, isolation-based techniques, reconstruction models such as autoencoders, and one-class learning approaches for identifying deviations without labeled supervision. The findings of [49], together with surveys of network-based intrusion detection datasets and techniques [31][32][33],

reinforce the methodological decision to explore structural deviations in feature space rather than claim real-time detection performance.

Clustering methodologies naturally extend EDA by enabling the identification of natural groupings within unlabeled telemetry data. Jain [41] reviews 50 years of clustering research, stressing that clustering success depends critically on understanding data structure, feature relevance, and distance metrics before selecting an algorithm. Jain emphasizes that clustering is inherently exploratory rather than confirmatory and that it requires careful interpretation and validation. This aligns with the present study's objective of using unsupervised clustering to examine whether normal and manipulated network conditions exhibit separable patterns in feature space, without asserting operational intrusion detection capability.

Common clustering techniques applied in network telemetry include centroid-based approaches such as K-Means, density-based algorithms such as DBSCAN, and hierarchical clustering methods. These approaches are frequently used to identify operational modes, transitional behaviors, or structural regime changes in network flow and wireless telemetry datasets [32][49]. In mesh network contexts, clustering provides a means to evaluate whether baseline configurations and controlled manipulation phases occupy distinct regions in feature space.

High-dimensional telemetry datasets often require dimensionality reduction to improve interpretability and assess clusterability. Jolliffe [52] presents Principal Component Analysis (PCA) as a variance-maximizing linear projection technique useful for identifying dominant sources of variance and detecting feature redundancy. PCA supports preliminary feature validation by revealing which engineered metrics contribute most strongly to overall variability, consistent with preprocessing principles discussed in [39].

For nonlinear visualization, van der Maaten and Hinton [50] introduce t-Distributed Stochastic Neighbor Embedding (t-SNE), which preserves local neighborhood relationships in reduced-dimensional space. t-SNE is widely used for exploratory visualization of potential separability in high-dimensional datasets. Similarly, McInnes, Healy, and Melville [51] propose Uniform Manifold Approximation and Projection (UMAP), a scalable manifold-learning method that preserves both local and global structural properties. UMAP is particularly useful for visualizing topology-dependent grouping tendencies in network telemetry and for examining clusterability in complex embeddings. As emphasized in the visualization literature [50][51], these techniques primarily serve as exploratory tools and do not, on their own, validate clustering performance without complementary quantitative measures.

Collectively, these works establish EDA [38–40], unsupervised anomaly detection [48][49], clustering theory [41], and dimensionality reduction techniques [50–52] as complementary methodologies for understanding complex telemetry datasets. In the context of this study, EDA establishes statistical baselines and cross-layer feature relationships; dimensionality reduction evaluates variance structure and potential separability; and clustering explores whether baseline and manipulated mesh conditions form distinguishable behavioral groupings. Together, these methods form the analytical backbone of the research, supporting dataset validation and structural behavior discovery rather than real-time intrusion detection deployment.

Author and Title	Description	Approach	Contribution

Tukey, J. W. (1977). Exploratory Data Analysis	Foundational introduction to EDA for discovering structure and detecting anomalies	Philosophy emphasizing visualization over confirmatory techniques	Supports a dedicated EDA stage for examining cross-layer mesh metrics before clustering
Aggarwal, C. C. (2015). Data Mining: The Textbook	Discussion of preprocessing, transformation, and correlation analysis before clustering	Systematic coverage of data preparation methodologies	Supports log aggregation, timestamp alignment, and feature scaling before unsupervised methods
Han, J., Pei, J., & Kamber, M. (2011). Data Mining: Concepts and Techniques	Emphasis on descriptive statistics and visualization for understanding the dataset structure	Comprehensive treatment of data exploration methodologies	Justifies distribution analysis, topology visualization, and cross-layer correlation examination
Jain, A. K. (2010). Data Clustering: 50 Years Beyond K-Means	Review of clustering methodologies, stressing understanding data structure before algorithm selection	Historical and methodological analysis of clustering techniques	Supports exploratory clustering; reinforces careful feature examination before unsupervised methods
Chatterjee et al. (2022). IoT Anomaly Detection Survey	Summarizes IoT anomaly detection methods and application domains	Survey of anomaly detection categories, including supervised and unsupervised approaches	Motivates unsupervised baselining when labeled attack data are limited in IoT telemetry environments
Miguel-Diez et al. (2025). Unsupervised Anomalous Traffic Detection Based on Flows (SLR)	Reviews unsupervised algorithms for anomaly detection in network flow data	PRISMA-style systematic literature review of flow-based anomaly detection techniques	Supports the use of unsupervised methods (e.g., autoencoders, density-based, and outlier detection techniques) to detect deviations without supervised labels
van der Maaten, L., & Hinton, G. (2008). Visualizing Data Using t-SNE	Introduces t-SNE for nonlinear dimensionality reduction and visualization of high-dimensional datasets	Manifold-based embedding focused on preserving local neighborhood structure	Useful for inspecting whether telemetry windows form separable visual groupings; supports exploratory cluster visualization rather than confirmatory validation
McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection	Presents UMAP as a scalable nonlinear dimensionality reduction technique	dimensionality reduction technique Topological manifold learning for preserving both local and global structure	Useful for exploring global and local structure in mesh telemetry and examining topology-dependent grouping tendencies

Jolliffe, I. T. (2002). Principal Component Analysis (2nd ed.)	Foundational work on PCA and variance-based dimensionality reduction	Linear orthogonal projection maximizing variance explained	Supports feature redundancy checks and identification of dominant variance drivers in engineered cross-layer mesh features

Table 2.10 Summary of EDA and Clustering in Network Topology

3. Theoretical Framework

This chapter establishes the theoretical foundations that inform the design, generation, and subsequent analysis of a cross-layer dataset from ESP32-based wireless mesh networks. The theoretical framework integrates concepts from wireless mesh networking, routing protocol behavior, attack theory, and cross-layer observation to provide a comprehensive basis for understanding how routing-based misbehavior manifests in observable network telemetry.

The chapter is organized into five main sections. Section 3.1 presents wireless mesh networks, characterizing the ESP-WIFI-MESH architecture and its operational principles. Section 3.2 examines topology formation and physical influences, detailing how environmental factors mediate the relationship between physical node placement and logical network structure. Section 3.3 provides a theoretical analysis of routing-based misbehavior, adapting canonical blackhole and wormhole attack concepts to the ESP-WIFI-MESH context. Section 3.4 develops the cross-layer telemetry perspective, explaining why routing violations affect multiple protocol layers and identifying expected observable inconsistencies. Finally, Section 3.5 synthesizes these elements into a conceptual framework that positions normal behavior as consistent relationships and misbehavior as broken relationships, providing the theoretical motivation for dataset collection.

Throughout this chapter, the emphasis is on theoretical characterization rather than detection—the goal is to establish what can be observed under different network conditions, not to prescribe how to identify attacks in real time.

3.1. Wireless Mesh Network Fundamentals

3.1.1. Wireless Mesh Network Characteristics

Wireless Mesh Networks (WMNs) represent a decentralized communication paradigm characterized by self-organization, infrastructure-less operation, and multi-hop capabilities. Unlike traditional centralized networks, WMNs enable each node to function simultaneously as both a host and a router, forwarding packets for other nodes to maintain network connectivity (Akyildiz et al., 2005). This architectural flexibility makes WMNs particularly suitable for Internet of Things (IoT) deployments in smart cities, environmental monitoring, disaster recovery, and rural connectivity scenarios where fixed infrastructure is unavailable or impractical.

The ESP-WIFI-MESH protocol, implemented on ESP32 microcontrollers, organizes nodes into a hierarchical tree topology. This architecture comprises three distinct node roles: the Root Node (Layer 0), which coordinates the mesh and serves as a gateway to external networks; Intermediate Nodes (Layer 1), operating in dual Access Point/Station (AP/STA) mode to relay packets between upstream parents and downstream children; and Leaf Nodes (Layer 2 and above), functioning exclusively in STA mode as traffic generators without forwarding responsibilities (Khan et al., 2022).

The protocol's operation involves two logical traffic types: the control plane, which handles node discovery, parent selection, and layer assignment based on heuristics such as Received Signal Strength Indication (RSSI), link quality, and hop count; and the data plane, which carries application payloads

hop-by-hop from leaf nodes toward the root using best-effort delivery. This design enables simple, self-organizing connectivity but introduces security vulnerabilities, as forwarding behavior is not verifiable at the protocol level.

From a theoretical standpoint, routing decisions in ESP-WIFI-MESH assume that locally observed metrics accurately reflect physical proximity and forwarding reliability. When these assumptions hold, the network exhibits stable packet propagation and predictable performance. Under normal operating conditions, ESP-MESH networks achieve packet delivery ratios exceeding 97% while maintaining packet loss and packet fault rates below 1.8% (Khan et al., 2022). However, when routing metrics are manipulated or exploited, the absence of centralized verification introduces systemic vulnerabilities. This framework explains why routing-based attacks can fundamentally distort network behavior without violating protocol syntax.

3.1.2. ESP-WIFI-MESH Architecture and Node Roles

ESP-WIFI-MESH organizes nodes into a tree topology. At the top is the Root Node (Layer 0), which coordinates the mesh and may act as a gateway to an external network. Below it are Intermediate Nodes (Layer 1) that operate in dual AP + STA mode: they connect upward to a parent while simultaneously accepting connections from child nodes. At the bottom are Leaf Nodes (Layer 2), which operate in STA mode only and generate application data.

The node roles are defined as follows:

Root Node (Layer 0):

- Coordinates the mesh and may act as a gateway to an external network.

Intermediate Nodes (Layer 1):

- Operate in dual AP + STA mode; connect upward to a parent while simultaneously accepting connections from child nodes.

Leaf Nodes (Layer 2 and above):

- Function exclusively in STA mode as traffic generators without forwarding responsibilities. They transmit and receive their own data but do not forward packets for other nodes.

The right side of the diagram shows the ESP32 network stack, where ESP-WIFI-MESH operates below the application layer and above the IEEE 802.11 MAC/PHY. Routing and forwarding are implicit, with no routing tables exposed to the application layer.

This design separates functionality into two logical components:

Control Plane (dashed arrows):

- Handles node discovery, parent selection, and layer assignment based on heuristics such as RSSI, link quality, and hop count.

Data Plane (solid arrows):

- Carries application payloads hop-by-hop from leaf nodes toward the root using best-effort delivery.

This enables simple, self-organizing connectivity, but also means forwarding behavior is not verifiable at the protocol level.

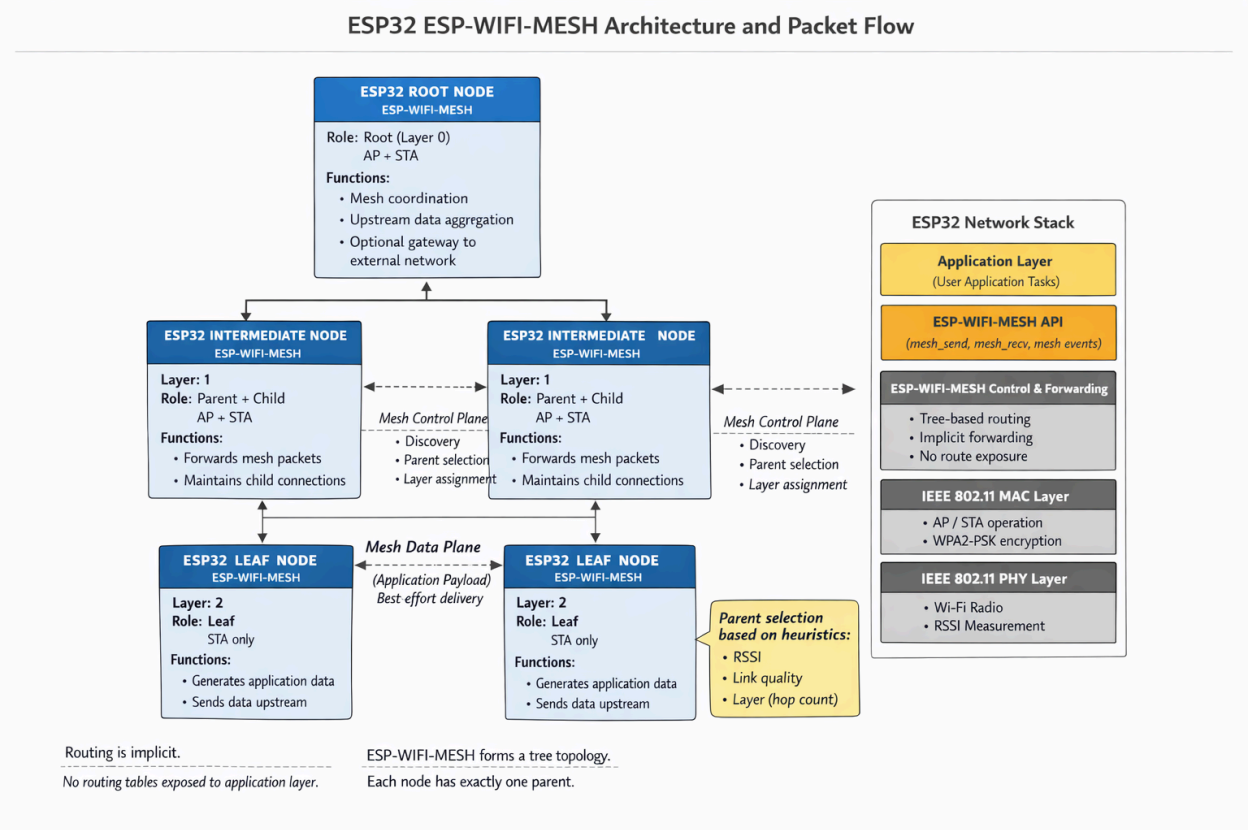

Figure 3.1 Architecture of ESP32-WFI-MESH

Figure 3.1 illustrates how ESP32 devices form a mesh network using ESP-WIFI-MESH and how data flows through it.

3.1.3. Control Plane vs. Data Plane Operations

3.1.3.1. Control Plane

The control plane handles network management and topology maintenance functions, including:

- **Node Discovery:** New nodes broadcast their presence and listen for responses from existing mesh members to identify potential parents.
- **Parent Selection:** Nodes evaluate candidate parents based on heuristics, including Received Signal Strength Indication (RSSI), link quality, hop count (layer), and the number of existing child connections.
- **Layer Assignment:** Based on parent selection, nodes determine their layer position within the hierarchical tree, with each hop from the root increasing the layer number.
- **Topology Maintenance**: Periodic exchanges maintain routing information and detect topology changes caused by node movement, failures, or link degradation.

3.1.3.2. Data Plane

- The data plane carries application payloads through the mesh using best-effort delivery. Key characteristics include:

- **Hop-by-Hop Forwarding:** Packets traverse from source to destination through intermediate nodes, with each node forwarding toward the root (upstream) or away from the root (downstream).
- **Forwarding Discipline**: Intermediate nodes must receive packets with *esp_mesh_recv()* and forward them with *esp_mesh_send()*, setting the MESH_DATA_P2P flag (Espressif Systems, 2024). This mandatory forwarding requirement establishes the behavioral baseline against which deviations can be observed.
- **Queue Managemen**t: Proper timing of packet forwarding is essential, as failure to forward packets promptly can lead to excessive memory consumption due to pending packets (Espressif Systems, 2024).

From a theoretical standpoint, routing decisions in ESP-WIFI-MESH assume that locally observed metrics accurately reflect physical proximity and forwarding reliability. When these assumptions hold, the network exhibits stable packet propagation and predictable performance. Under normal operating conditions, ESP-MESH networks achieve packet delivery ratios exceeding 97% while maintaining packet loss and packet fault rates below 1.8% (Khan et al., 2022).

Table 3.1 ESP-WIFI-MESH Node Roles and Responsibilities

Node Type	Layer	Mode	Forwarding Responsibility	Primary Function
Root	0	STA + AP (to router)	Yes	Gateway to external networks
Intermediate	1+	AP/STA dual	Yes	Relay packets + generate traffic
Leaf	2+	STA only	No	Generate traffic only

This design enables simple, self-organizing connectivity but introduces inherent trust assumptions: forwarding nodes are expected to comply with protocol requirements, and locally observed metrics are trusted for routing decisions. When these assumptions are violated, the absence of centralized verification creates systemic vulnerabilities that can distort network behavior without violating protocol syntax.

3.2. Topology Formation and Physical Influences

3.2.1. RSSI Thresholds and Parent Selection

Parent selection in ESP-WIFI-MESH is governed by a multi-metric algorithm that evaluates candidate parents based on parameters such as hop count, received signal strength indicator (RSSI), and the number

of associated child nodes. RSSI serves as a primary indicator of link quality and physical proximity, forming the basis for initial parent selection and subsequent topology maintenance.

3.2.1.1. RSSI Thresholding Mechanism

ESP-WIFI-MESH implements RSSI thresholding to prevent nodes from forming weak connections that could degrade network performance (Espressif Systems, 2023). The protocol defines three default RSSI thresholds:

Table 3.2 ESP-WIFI-MESH Default RSSI Thresholds

Threshold	Value	Significance
High	-78 dBm	Minimum acceptable RSSI for preferred parent relationships
Medium	-82 dBm	Intermediate threshold for parent evaluation
Low	-85 dBm	Threshold below which parent switching is triggered

These thresholds are defined in the ESP-MESH internal header file within the mesh_rssi_threshold_t structure (Espressif, 2023b). When a parent node's RSSI falls below the low threshold for a sustained period, the child node generates a MESH_WEAK_RSSI event and may initiate parent switching to find a more suitable connection (ESP32 Community Forum, 2023).

3.2.1.2. Parent Selection Criteria

According to official ESP-WIFI-MESH documentation, the preferred parent node is determined based on two primary criteria (Espressif Systems, 2023):

1. **Layer Position:** The layer of the candidate parent node—parents must reside at a lower layer than the connecting node.
2. **Child Count:** The number of downstream connections the candidate parent node currently has.

When multiple candidates exist on the same layer, "the candidate with the fewest child nodes will be preferred" (Espressif, 2023a). This load-balancing mechanism distributes connections across available parents but also creates exploitable patterns, as nodes with few children become algorithmically attractive regardless of other characteristics.

The mesh_switch_parent_t structure further defines the switch_rssi parameter, which determines when a node will "disassociate with current parent and switch to a new parent when the RSSI is greater than this set threshold" (Espressif, 2023b). This mechanism enables dynamic topology adaptation but also provides an exploitation vector for nodes capable of advertising superior RSSI values.

3.2.2. Environmental Effects on Topology Formation

Physical environment characteristics significantly influence observed RSSI values and, consequently, the logical topology formed by ES-WIFI-MESH. Understanding these effects is essential for interpreting network behavior and designing controlled experiments.

3.2.2.1. Impact of Physical Obstacles

RF shielding objects—including walls, metal structures, furniture, and building materials—can dramatically attenuate wireless signals, reducing RSSI even when nodes are physically proximate (Espressif Systems, 2023a). This attenuation creates several important phenomena:

- A physically close node may be rejected as a parent candidate due to low RSSI caused by obstacles
- A physically distant node with a clear line-of-sight may become the preferred parent despite greater physical separation
- The resulting logical topology may differ significantly from the physical node layout

As illustrated in the Espressif documentation, an RF shielding object can cause nodes to disregard physically adjacent parent candidates and instead connect to more distant nodes with stronger signals (Espressif Systems, 2023a). This decoupling of physical and logical topology has important implications for experimental design and behavior interpretation.

3.2.2.2. Multi-path and Interference Effects

Indoor environments typically exhibit complex RF characteristics, including:

- Multi-path propagation causes signal fading and reinforcement
- Co-channel interference from other Wi-Fi networks
- Transient signal variations due to movement or environmental changes

These factors introduce variability in RSSI measurements, which must be accounted for in the experimental design. Controlled indoor environments with minimal external interference improve repeatability and enable accurate ground-truth labeling (Keipour, 2022).

3.2.2.3. Chain Formation in Linear Deployments

When nodes are arranged linearly—for example, along a hallway or corridor—the network naturally forms a chain topology. This configuration has distinct characteristics compared to branched tree topologies, including:

- Longer end-to-end paths with higher hop counts
- Single points of failure where intermediate node loss partitions the network
- Cumulative latency and packet loss effects

Understanding these environmental influences guides experimental testbed design and informs interpretation of observed network behaviors.

3.2.3. Logical vs. Physical Topology Relationships

The RSSI-based parent selection mechanism mediates the relationship between physical node placement and logical network topology. This mediation creates several theoretically important relationships.

3.2.3.1. Monotonic RSSI Degradation Assumption

Under normal operating conditions, routing protocols implicitly assume that signal strength degrades monotonically with increasing hop distance. That is, as packets traverse multiple hops, RSSI should decrease predictably, with each hop contributing approximately 5–10 dB of attenuation under consistent environmental conditions.

This assumption enables nodes to infer physical proximity from observed signal strength: a node at Layer 4 should typically observe RSSI values near the switch_rssi threshold (approximately -78 dBm). In contrast, a node at Layer 1 should observe values in the -50 to -65 dBm range.

3.2.3.2. Correlation Between Hop Count and Expected RSSI

Table 3.3 Expected RSSI Ranges by Node Position

Node Position	Expected RSSI Range	Relationship to Thresholds
Proximal to the router	≈ -30 dBm	Far above high threshold (-78 dBm)
Normal parent node (Layer 1)	≈ -50 to -78 dBm	Above or near the high threshold
Leaf node (Layer 2-3)	≈ -78 to -85 dBm	Between high and low thresholds
Unstable connection	< -85 dBm	Below the low threshold—triggers parent switching

3.2.3.3. Topology Formation Principles

Based on these relationships, several principles govern topology formation:

1. **Layer Progression:** Each node must connect to a parent at a strictly lower layer, ensuring hierarchical structure.
2. **Signal-Based Selection:** Among candidates at acceptable layers, those with the strongest RSSI are preferred.
3. **Load Balancing:** Among candidates with comparable RSSI, those with fewer children are preferred.
4. **Stability Maintenance:** Nodes remain with their current parents unless a superior alternative appears or the current parent's RSSI degrades below thresholds.

These principles establish the theoretical baseline for normal network behavior against which attack-induced deviations can be observed. When malicious nodes manipulate RSSI advertisements or child count presentations, they can distort logical topology in ways that violate the expected relationships between physical placement and network structure.

3.3. Routing-Based Attack Theory

3.3.1. Blackhole Attack: Conceptual Foundation

Blackhole attacks are theoretically classified as routing-layer denial-of-service attacks in which a malicious node attracts network traffic by advertising favorable routing metrics and subsequently drops all received packets without forwarding them (Kurosawa et al., 2007). The defining characteristic of a blackhole attack is not the transmission of malformed packets or explicit protocol violations, but the selective non-forwarding of data after successful route establishment.

Unlike attacks that disrupt routing syntax or corrupt packet structure, blackhole behavior exploits cooperative routing assumptions. The attacker participates normally in control-plane exchanges, thereby appearing legitimate to neighboring nodes while silently discarding data traffic.

3.3.1.1. Theoretical Characterization

Baadache and Belmehdi [44] describe blackhole attacks as an extreme form of packet non-forwarding. In this scenario, the malicious node behaves correctly during route discovery and topology formation. Still, it creates a logical “hole” in the network by refusing to forward packets once it becomes part of an established route.

The attack exploits a core assumption of multi-hop wireless mesh routing: that intermediate nodes will forward packets received on behalf of others. When this assumption is violated, end-to-end communication reliability degrades without immediate structural collapse of the routing topology.

The operational stages of a canonical blackhole attack can be conceptualized as:

- **Attractor Phase:**
  The malicious node advertises favorable routing metrics—such as lower hop count, stronger signal quality, or superior positioning—causing legitimate nodes to select it as a next-hop forwarder.
- **Establishment Phase:**
  The attacker becomes integrated into active routing paths and begins receiving packets from nodes while maintaining protocol-compliant control-plane participation.
- **Execution Phase:**
  The attacker silently discards received data packets instead of forwarding them, while continuing to maintain normal neighboring control-plane behavior to avoid detection.

Importantly, the routing structure may appear stable even while data forwarding behavior is compromised.

3.3.1.2. Adaptation to ESP-WIFI-MESH Context

In ESP-WIFI-MESH, intermediate nodes must receive packets via the mesh API and forward them upstream or downstream, depending on the routing direction. Because forwarding is implicit and not cryptographically enforced, the protocol assumes that relay nodes comply cooperatively.

A blackhole node within ESP-MESH may therefore:

- Successfully associate as a parent node,
- Participate in mesh control exchanges,
- Receive packets from child nodes,
- Fail to forward those packets toward the root or destination.

Since MAC-layer acknowledgments may still occur at the link level, upstream nodes may observe successful transmission even though end-to-end delivery fails. This decoupling of link-layer success from network-layer forwarding creates observable behavioral inconsistencies.

3.3.1.3. Theoretical Expectations for Observable Behavior

From a cross-layer observational perspective, blackhole behavior produces measurable deviations from normal forwarding relationships.

Network Layer

- Forwarding ratio significantly below unity (packets received > packets forwarded).
- Positive ingress–egress delta at the malicious node.
- Decreased packet delivery ratio for nodes downstream of the attacker.
- End-to-end packet loss without a corresponding topology change.

MAC Layer

- Potential increase in retransmission attempts by upstream nodes due to missing higher-layer responses.
- Successful link-layer acknowledgments despite end-to-end delivery failure.

Physical Layer (PHY)

- RSSI patterns remain consistent with physical placement (unlike wormhole).
- No necessary distortion in signal strength or hop-layer relationship.

These observable effects form the theoretical basis for cross-layer telemetry collection in this study. The objective is not to define real-time detection rules, but to document how forwarding suppression manifests in measurable system behavior under controlled experimental conditions. This characterization supports subsequent exploratory data analysis and unsupervised clustering of behavioral patterns.

3.3.2. Wormhole Attack: Conceptual Foundation

Wormhole attacks are theoretically distinct from blackhole attacks in that they do not primarily rely on packet dropping, but on topology distortion. A wormhole attack involves two colluding malicious nodes that establish a low-latency out-of-band communication channel and relay packets between distant parts

of a wireless mesh network, thereby creating the illusion that physically remote nodes are direct wireless neighbors (Hu et al., 2003).

Unlike attacks that modify packet contents or violate protocol syntax, wormhole attacks exploit routing trust assumptions. By capturing packets at one location and replaying them at another with minimal delay, attackers artificially shorten perceived routing paths. As a result, nodes may believe that distant nodes are only one hop away, even though they remain separated by multiple legitimate intermediate nodes.

3.3.2.1. Theoretical Characterization

As described in the literature, during a wormhole attack, a malicious node captures packets from one location in the network. It tunnels them to another malicious node at a distant point, which replays them locally (Khalil et al., 2007). This tunnel may be implemented using an out-of-band channel, encapsulation, or high-speed relaying. The key requirement is that tunneled packets arrive faster or with fewer perceived hops than they would through legitimate multi-hop routing.

This process creates the illusion that the two endpoints of the tunnel are geographically close to each other. In routing protocols that rely on metrics such as hop count, signal strength (RSSI), or timing characteristics, this illusion significantly distorts the network's logical topology.

Key theoretical properties of wormhole attacks include:

- **Collusion Requirement**: The attack requires at least two cooperating malicious nodes to establish the tunnel.
- **Topology Distortion**: The perceived logical network structure is altered without necessarily modifying packet contents.
- **Stealth Operation**: Attackers may remain absent from routing tables while still influencing or controlling traffic flows.
- **Protocol Independence**: Wormhole attacks can be launched without compromising cryptographic keys or altering legitimate nodes (Khalil et al., 2007).

3.3.2.2. Adaptation to ESP-WIFI-MESH Context

In ESP-WIFI-MESH, parent selection and routing decisions rely on heuristics including hop count (layer), Received Signal Strength Indicator (RSSI), and load-related metrics. Because nodes select parents based on perceived proximity and signal quality, artificially favorable metrics introduced through a wormhole tunnel can attract routing decisions toward the malicious endpoints.

The tunnel's illusion disrupts the expected relationship between physical node placement and logical topology. Nodes may switch parents toward a malicious endpoint because it appears to offer:

- Lower hop count (closer to the root),
- Stronger signal quality,
- More attractive routing characteristics.

Importantly, this distortion occurs without overt protocol violations. Control-plane exchanges remain syntactically valid, and the routing state appears internally consistent from the perspective of individual nodes. The inconsistency only becomes evident when comparing physical deployment constraints with observed routing relationships.

In this study, the wormhole concept is used as the theoretical model of topology distortion. The implementation reproduces the expected cross-layer effects (e.g., latency anomalies and RSSI–hop inconsistencies) using application-layer relaying rather than raw IEEE 802.11 frame replay.

3.3.2.3. Wireless Topology Distortion

From a routing-theoretic perspective, wormhole behavior produces structural inconsistencies in the mesh:

- Nodes may form parent–child relationships that bypass legitimate intermediate layers.
- Hop counts may decrease abruptly without corresponding changes in physical node placement.
- Routing tables may reflect physically implausible neighbor relationships.
- Logical proximity may no longer correspond to physical proximity.

This results in a decoupling of the physical and logical topologies. Under normal conditions, physical signal propagation mediates the routing structure. Under wormhole conditions, the artificial tunnel overrides these physical constraints.

3.3.2.4. Cross-Layer Effects of Wormhole Distortion

Although wormhole attacks originate at the routing level, their effects propagate across protocol layers. Theoretical expectations include:

Network Layer

- Unexpected reductions in hop count relative to physical layout.
- Parents switch events toward physically distant nodes.
- Routing entries are inconsistent with deployment constraints.
- Apparent one-hop relationships between nodes separated by multiple legitimate hops.

Physical Layer (PHY)

- RSSI values are inconsistent with expected distance-based attenuation.
- Violations of the typical relationship between hop count and signal strength.
- Abrupt signal-strength changes without physical repositioning.

MAC Layer

- Abnormal frame arrival timing due to tunnel relay.
- Potential duplicate or out-of-order frame patterns.
- Local retransmission anomalies near tunnel endpoints.

These cross-layer inconsistencies form the theoretical basis for telemetry collection in this study. The objective is not to prescribe real-time detection mechanisms, but to document how topology distortion manifests in measurable system behavior under controlled experimental conditions. This observational characterization supports subsequent exploratory analysis and unsupervised clustering of behavioral patterns in later chapters.

3.4. Cross-Layer Telemetry Perspective

3.4.1. Why Routing Violations Affect Multiple Protocol Layers

Routing-based misbehavior in wireless mesh networks does not remain confined to the network layer where it originates. Instead, routing violations produce correlated effects across multiple protocol layers due to the interdependent nature of wireless communication. Understanding these cross-layer effects is essential for designing comprehensive telemetry that captures the full behavioral signature of network anomalies.

3.4.1.1. Physical Layer Effects

Routing manipulations influence physical layer observables through several mechanisms:

- **RSSI-Hop Correlation Disruption**: Under normal operation, RSSI degrades predictably with increasing hop count. When wormhole attacks create artificial shortcuts, this correlation breaks—nodes receive packets with RSSI values inconsistent with their layer position.
- **Signal Consistency Patterns**: Legitimate multi-hop paths produce characteristic RSSI variation patterns as packets traverse intermediate nodes. Attack-induced topology distortion introduces abrupt changes in RSSI that deviate from expected patterns.
- **Transmission Power Artifacts**: Malicious nodes may transmit at different power levels than legitimate nodes, creating observable inconsistencies in received signal strength relative to distance.

3.4.1.2. MAC Layer Effects

Link-layer behavior responds to routing anomalies through:

- **Retransmission Dynamics**: When packets are dropped at the network layer (blackhole) or tunneled with abnormal timing (wormhole), MAC-layer retransmission counters reflect the resulting delivery failures or timing anomalies.
- **Acknowledgment Patterns**: MAC-layer acknowledgments may succeed even when network-layer forwarding fails, creating observable dissociation between link-level and end-to-end delivery success.
- **Channel Access Contention**: Abnormal traffic patterns induced by routing manipulation can alter channel contention statistics observable through MAC-layer diagnostic counters.

3.4.1.3. Network Layer Effects

The network layer directly manifests routing anomalies through:

- **Forwarding Ratio Deviations**: The relationship between packets received and packets forwarded provides direct visibility into forwarding compliance.
- **Topology Consistency**: Parent-child relationships, hop counts, and routing table entries reflect the logical network structure, which may diverge from physical reality under attack.
- **End-to-End Delivery Metrics**: Packet delivery ratio, loss rate, and latency patterns capture the ultimate effect of routing manipulation on application-layer communication.

3.4.2. Collected Metrics and Data Sources

Received Signal Strength Indicator (RSSI)

- RSSI values are collected using the ESP32 Wi-Fi driver API (*esp_wifi_sta_get_rssi()*). Measurements are sampled at fixed intervals (e.g., once per second) during each experimental run and logged with timestamps.

RSSI Variation Over Time

- Temporal changes in RSSI are not computed on-device; instead, they are derived during dataset preprocessing by comparing successive samples within fixed observation windows.

Transmit Power Configuration (Metadata)

- Transmit power settings are recorded as configuration metadata to ensure consistent radio conditions across experimental trials. This information supports reproducibility and contextual interpretation of signal behavior. Physical-layer metrics provide context for interpreting routing and forwarding behavior by distinguishing between environmental wireless variability and structural network effects.

3.4.3. Temporal Aggregation of Cross-Layer Telemetry

Wireless network telemetry, particularly at the physical and MAC layers, exhibits rapid fluctuations due to environmental factors, transient interference, and normal protocol dynamics. Individual 1 Hz samples of RSSI, retransmission counts, or parent state may reflect momentary conditions rather than stable behavioral patterns. Therefore, for meaningful analysis, raw telemetry must be aggregated over appropriate time windows.

This study adopts a temporal aggregation approach in which raw 1 Hz samples are grouped into fixed-duration windows (e.g., 5 seconds). Within each window, raw counters are converted to deltas, and continuous metrics are summarized using statistical descriptors (mean, variance, minimum, maximum). This aggregation serves several purposes:

1. **Noise Reduction**: Aggregation smooths out transient fluctuations, revealing underlying behavioral trends.
2. **Pattern Capture**: Many attack-induced effects (e.g., increased parent switching, forwarding ratio degradation) manifest over periods of seconds rather than individual packet events.
3. **Statistical Robustness**: Derived features such as forwarding ratio require sufficient samples to produce stable estimates; windowing ensures this.
4. **Cross-Layer Alignment**: Windowing enables correlation of metrics from different layers within the same temporal context.

The choice of window duration (5 seconds) balances the need for sufficient samples with the desire to capture attack-induced behavioral changes that may persist for minutes.

3.4.4. Expected Observable Inconsistencies

Based on analyses of blackhole and wormhole behavior, specific cross-layer inconsistencies are expected to manifest for each attack type.

3.4.4.1. Expected Blackhole Observables

Table 3.4 Expected Observable Inconsistencies Under Blackhole Behavior

Layer	Metric	Expected Deviation
Network	Forwarding Ratio	< 1.0 at attacker node (packets received but not forwarded)
Network	Ingress/Egress Delta	> 0 at attacker node (positive difference indicating packets absorbed)
Network	Packet Delivery Ratio	Decrease for victims downstream of the attacker
MAC	Retransmission Count	Increase for victim nodes (repeated attempts with no acknowledgment)
PHY	RSSI	Stable at attacker (positioning unchanged throughout)

3.4.4.2. Expected Wormhole Observables

Table 3.5 Expected Observable Inconsistencies Under Wormhole Behavior

Layer	Metric	Expected Deviation
Network	Hop Count Consistency	Nodes report unexpectedly low hop counts given physical placement
Network	Parent-Child Validity	Relationships violate physical constraints (e.g., Layer 4 child of Layer 0)
Network	Routing Table Entries	Self-referential entries or impossible neighbor relationships
PHY	RSSI-Hop Correlation	High RSSI observed from physically distant nodes

PHY	RSSI Variance	Abnormally high RSSI values are inconsistent with the expected path loss
MAC	Frame Arrival Pattern	Duplicates or out-of-order frames due to tunneling
MAC	Retransmission Count	May increase near tunnel endpoints due to timing issues

3.4.4.3. Timing and Latency Inconsistencies

Both attack types may produce timing anomalies:

- Blackhole: End-to-end latency for successful packets may remain normal, but packet loss creates apparent "gaps" in traffic flow.
- Wormhole: Packets may arrive faster than physically possible, given the claimed hop count, or exhibit abnormal inter-arrival patterns due to tunnel latency variations.

3.4.5. Expected Observable Inconsistencies

The cross-layer telemetry perspective adopted in this study serves a specific purpose: characterizing how routing misbehavior manifests across protocol layers in a real ESP32 mesh network. This observational framework is explicitly distinguished from intrusion detection—the goal is not to identify attacks in real time, but to document behavioral patterns for subsequent exploratory analysis.

3.4.5.1. Observational Objectives

- **Baseline Characterization:** Establish empirical baselines for cross-layer metrics under normal operating conditions across multiple topology configurations.
- **Deviation Documentation:** Record how metrics change when controlled routing manipulations are introduced, capturing the full behavioral signature of each misbehavior type.
- **Pattern Identification:** Enable exploratory analysis to identify which metrics consistently differentiate normal from manipulated behavior.
- **Feature Selection Guidance:** Provide an empirical basis for selecting features most informative for behavioral characterization, supporting future research without pre-committing to specific detection approaches.

3.4.5.2. Methodological Principles

The cross-layer observation framework follows several methodological principles:

- **Non-intervention**: Observations are recorded without modifying network behavior or attempting to correct anomalies.
- **Comprehensive Coverage**: Metrics are collected from PHY, MAC, and network layers to capture correlated effects.

- **Ground Truth Alignment**: Observations are time-stamped and labeled according to controlled experimental conditions.
- **Reproducibility Focus**: Collection methods are documented to enable replication and extension.

3.4.5.3. Distinction from Detection System

This observational approach differs fundamentally from intrusion detection systems:

Table 3.6 Distinction Between Cross-Layer Observation and Intrusion Detection

Aspect	Cross-Layer Observation (This Study)	Intrusion Detection System
Purpose	Characterize behavior	Identify and alert on attacks
Timing	Offline analysis	Real-time or near-real-time
Decision Making	None (data collection only)	Automated response or alerting
Performance Metrics	Data quality, completeness	Detection accuracy, latency
Output	Labeled dataset	Alerts, classifications

By maintaining this distinction, the study focuses on its core contribution: a hardware-derived, cross-layer dataset documenting ESP-WIFI-MESH behavior under normal and manipulated conditions, suitable for subsequent exploratory analysis by the research community.

3.5. Conceptual Framework of the Study

3.5.1. Normal Behavior as Consistent Relationships

Based on the theoretical foundations established in preceding sections, normal ESP-WIFI-MESH operation is characterized by consistent, predictable relationships among observable network metrics. These relationships arise from the protocol's design principles and the physical constraints of wireless communication.

Table 3.7 Consistent Relationships in Normal ESP-WIFI-MESH Operation

Relationship	Theoretical Basis	Expected Consistency
Hop Count ↔ RSSI	Monotonic signal degradation with distance	RSSI decreases predictably as hop count increases

Reception ↔ Forwarding	Protocol forwarding requirement	Packets received ≈ packets forwarded (ratio ≈ 1.0)
Parent ↔ Child Layer	Hierarchical structure	Parent layer < child layer
Physical ↔ Logical Topology	RSSI-based parent selection	Logical topology reflects physical proximity (mediated by obstacles)
Transmission ↔ Acknowledgment	MAC-layer reliability	Successful transmissions receive timely acknowledgments

These relationships serve as the baseline against which deviations can be observed. When the network operates normally, these relationships hold consistently across time and topology configurations.

3.5.2. Misbehavior as Broken Relationships

Routing-based misbehavior fundamentally disrupts the relationships that characterize normal operation. Both blackhole and wormhole attacks break specific relationships in theoretically predictable ways.

3.5.2.1. Blackhole: Broken Reception-Forwarding Relationship

The defining characteristic of blackhole behavior is the decoupling of packet reception from packet forwarding:

- **Normal:** reception → forwarding (causal link preserved)
- **Blackhole**: reception → NO forwarding (causal link broken)

This broken relationship manifests in multiple observable indicators:

- Forwarding ratio deviates from unity
- Ingress/egress delta becomes positive
- Victims experience packet loss despite successful reception at the attacker

3.5.2.2. Wormhole: Broken Physical-Logical Topology Relationship

Wormhole behavior disrupts the expected correspondence between physical node placement and logical network topology:

- **Normal**: physical proximity → strong RSSI → parent selection
- **Wormhole**: forged advertisements → strong RSSI perception → parent selection WITHOUT physical proximity

This broken relationship manifests through:

- Violation of monotonic RSSI-hop correlation
- Physically impossible parent-child relationships

- Routing tables are inconsistent with node placement

3.5.3. Experimental Emulation of Routing Manipulations

In controlled experimental research on wireless mesh networks, it is not always feasible or necessary to implement attacks exactly as they would occur in the wild. Full protocol-level exploitation—such as raw 802.11 frame capture and replay for wormhole attacks—often requires low-level driver access and firmware modifications that may be unavailable or impractical on commercial microcontroller platforms like the ESP32. Furthermore, such low-level implementations introduce significant complexity and may violate platform constraints or ethical research boundaries.

Therefore, this study adopts an experimental emulation approach to routing-based misbehavior. Rather than replicating the exact low-level mechanics of blackhole and wormhole attacks, the methodology focuses on reproducing their observable behavioral effects within a controlled laboratory environment. This approach is justified on several grounds:

1. **Platform Constraints:** The ESP32's closed-source Wi-Fi firmware and limited access to raw frame injection make true protocol-layer attacks difficult to implement reliably across firmware versions. Application-layer emulation ensures reproducibility and platform compatibility.
2. **Behavioral Focus:** The primary research objective is to generate a dataset of cross-layer telemetry under both normal and manipulated conditions. The specific mechanism by which manipulation is achieved is secondary to the resulting behavioral patterns. As long as the emulation produces the same observable inconsistencies (e.g., forwarding ratio degradation, hop-count anomalies, RSSI-hop mismatches), the dataset remains valid for exploratory analysis.
3. **Controlled Activation:** Emulation allows precise temporal control over manipulation windows, enabling clean ground-truth labeling that would be difficult to achieve with true protocol exploits that may have variable activation timing.
4. **Ethical Considerations:** Application-layer emulation operates entirely within the normal bounds of mesh communication, avoiding any risk of unintended network disruption or security compromise beyond the controlled testbed.

The emulation model for blackhole attacks reproduces forwarding suppression through intentional omission of packet forwarding at the application layer. The emulation model for wormhole attacks reproduces topology distortion through an out-of-band tunnel that relays probe metadata between distant network regions, creating artificial path compression without raw frame capture. Both approaches generate the same cross-layer inconsistencies predicted by the theoretical framework while maintaining full protocol compliance and experimental reproducibility.

3.5.4. Secondary Broken Relationships

Both attack types may induce secondary relationship disruptions:

- **MAC-Network Layer Consistency:** MAC-layer success may persist despite network-layer failure (blackhole) or abnormal timing (wormhole)
- **End-to-End Reliability Expectations:** Packet delivery falls below protocol-specified baselines
- **Topology Stability:** Parent switching frequency may increase as nodes react to abnormal conditions

3.5.5. These Inconsistencies Motivate Dataset Collection

The theoretical framework developed in this chapter establishes a clear rationale for dataset collection: documenting how routing-based misbehavior disrupts the consistent relationships that characterize normal ESP-WIFI-MESH operation.

3.5.5.1. Theoretical Justification for Dataset Requirements

Table 3.8 Theoretical Basis for Dataset Design Requirements

Requirement	Theoretical Basis
Cross-layer feature collection	Attacks affect multiple protocol layers simultaneously.
Hardware-based generation	Real RF dynamics and protocol timing cannot be simulated accurately.
Multiple topology configurations	Relationship consistency varies with network structure.
Controlled attack scenarios	Enables observation of specific relationship disruptions
Ground truth labeling	Essential for identifying which relationships break under which conditions

3.5.6. Exploratory Behavioral Analysis and Pattern Discovery

Given that the precise manifestations of routing misbehavior in real ESP32 hardware are not fully characterized a priori, this study adopts an exploratory analytical approach. Rather than assuming known attack signatures, the methodology employs unsupervised techniques to discover whether normal and manipulated network states form separable patterns in the feature space.

This approach is justified on several grounds:

1. **Unknown Behavioral Signatures:** While theoretical expectations exist (e.g., forwarding ratio degradation for blackhole, RSSI-hop mismatch for wormhole), how these manifest in real hardware under controlled conditions is an empirical question. Unsupervised discovery allows patterns to emerge without imposing pre-conceived classifications.
2. **Natural Separability Hypothesis:** If the theoretical framework is correct—if forwarding suppression and topology distortion indeed produce distinct, measurable cross-layer inconsistencies—then normal and manipulated behavioral states should form separable clusters in

the multidimensional feature space. Clustering serves as a validation mechanism for this theoretical prediction.

3. **Anomaly Discovery Orientation:** The DBSCAN algorithm's ability to identify noise points aligns with the possibility that some experimental windows may exhibit transitional or mixed behaviors not cleanly belonging to any single category. These points can be examined separately to understand boundary conditions between behavioral states.
4. **Avoidance of Overfitting:** Supervised classification would require partitioning the limited dataset into training and test sets, potentially overfitting to specific experimental conditions. Unsupervised clustering uses all available data to discover structure, making no assumptions about which windows correspond to which labels until after clustering is complete.
5. **Alignment with Research Objectives:** The study's primary objective is to generate a dataset and explore its structure, not to build a deployable classifier. Clustering directly supports this objective by revealing whether the collected features capture meaningful distinctions between behavioral states.
6. **Multivariate Nature of Anomalie**s: As established in Section 3.4, routing attacks create correlated inconsistencies across multiple protocol layers. Relying on a single metric (e.g., a low forwarding ratio) could lead to false positives or miss subtle, sophisticated manipulations. Therefore, the analysis must consider the combined, multivariate state of features to capture the full behavioral signature of an attack, a principle that motivates the subsequent feature selection and clustering methodology.

Thus, the conceptual framework positions normal behavior as consistent relationships and misbehavior as broken relationships, with the expectation that these broken relationships will manifest as separable behavioral states in the feature space.

3.5.7. Expected Contribution

The dataset generated under this theoretical framework will enable:

1. **Empirical Validation:** Testing whether theoretically expected relationship disruptions actually manifest in real hardware.
2. **Pattern Discovery:** Identifying which metrics most consistently indicate specific types of misbehavior.
3. **Baseline Establishment:** Documenting normal relationship ranges across topologies for future comparison.
4. **Feature Prioritization:** Determining which cross-layer features are most informative for behavioral characterization.

3.5.8. Framework Summary

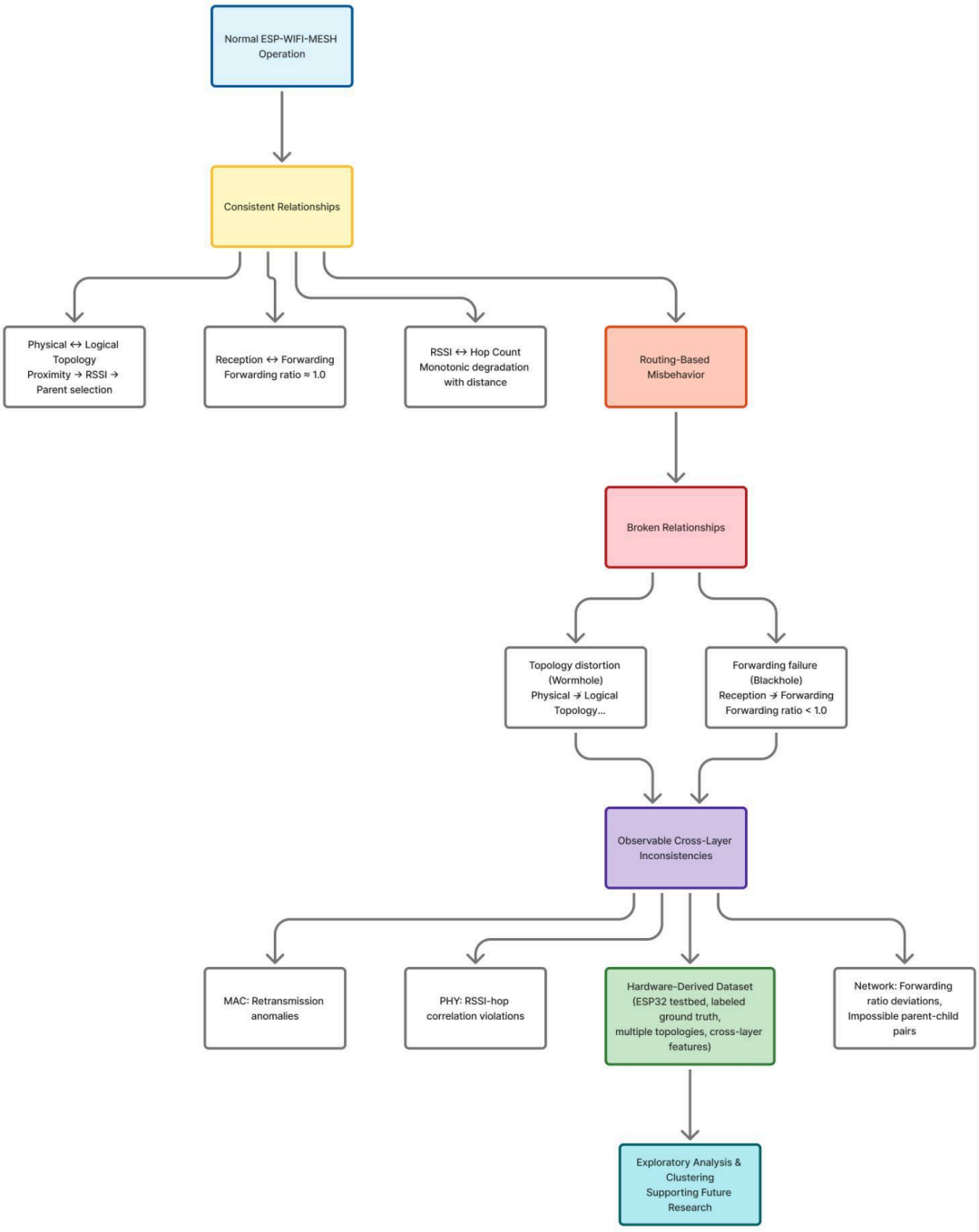

Figure 3.2 Conceptual Framework of the Study

Figure 3.2 presents the conceptual framework of this study, illustrating the progression from wireless mesh networking theory to the generation of an empirical dataset and exploratory analysis. Wireless mesh theory establishes the operational properties of ESP-WIFI-MESH and the baseline relationships expected during normal operation (e.g., hop count–RSSI consistency, hierarchical parent–child layering, and

receive–forward consistency at intermediate nodes). Routing-based misbehavior is then described as the disruption of these expected relationships, where forwarding suppression alters packet propagation patterns and topology distortion alters the correspondence between physical signal conditions and logical routing structure.

Building on these foundations, the study adopts a cross-layer telemetry perspective to justify why disruptions at the routing level may produce observable inconsistencies across the PHY, MAC, and Network layers. Importantly, this framework is observational rather than detection-oriented: it does not define real-time decision rules or claim operational attack identification. Instead, it motivates collecting cross-layer indicators and ground-truth labels so that subsequent EDA and unsupervised clustering can examine whether normal and manipulated runs exhibit separable behavioral structure in the feature space. In this way, the framework directly informs feature selection and experimental design in Chapter 4 and provides the interpretive lens for the exploratory analyses in later chapters.

The theoretical relationships and observable deviations described in this chapter guide the experimental design and dataset generation procedures presented in Chapter 4. By grounding the dataset in established wireless mesh theory and attack theory, the study ensures that the collected telemetry captures behaviorally meaningful patterns that can support future research on intrusion detection and network analysis.

4. Research Design

This study follows an experimental system-development approach focused on the construction and observation of an ESP32-based ESP-WIFI-MESH network. At the current stage of the research, the methodology outlines the planned processes for building the testbed, introducing controlled network behaviors, and collecting cross-layer data that may later support anomaly analysis. Rather than presenting finalized experimental outcomes, this chapter describes the intended workflow and ongoing implementation procedures. The study aims to observe how mesh networking behavior evolves under both baseline and adversarial conditions while the system is still under active development.

The methodology is currently organized into three working phases:

1. Mesh network configuration using ESP32 devices.
2. Progressive implementation of controlled network behaviors that resemble known routing anomalies.
3. Continuous logging and refinement of cross-layer network metrics.

These phases are being refined iteratively as the mesh infrastructure and data collection tools are gradually completed.

4.1. Architectural Design

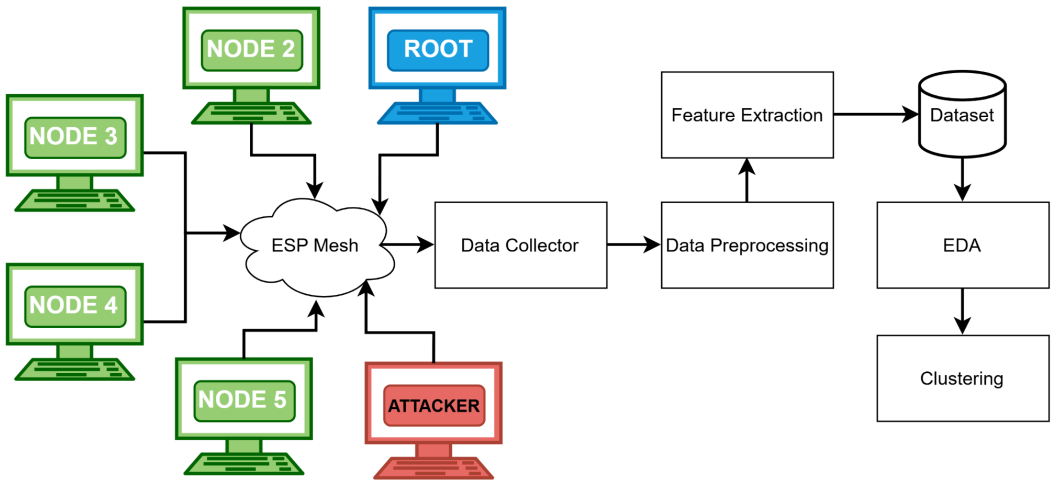

Figure 4.1 Architecture Design

The architectural design in *Figure 4.1* illustrates the complete experimental workflow of the study, showing how data flows from the physical ESP32 mesh network through a structured processing pipeline to produce analytical results. The ESP-WIFI-MESH testbed consists of multiple ESP32 nodes, including a root node, several regular nodes, and attacker nodes, which together form a wireless mesh network that generates telemetry data during experimental runs.

Each node periodically logs telemetry measurements such as RSSI, parent node information, hop count, retransmission statistics, and application-layer probe behavior. These logs are collected by the Data Collector, which aggregates telemetry from all nodes during the experiment.

The collected data is then processed by the Data Preprocessing stage, which performs timestamp alignment and aggregates continuous measurements into five-second time windows to create consistent observation intervals across all nodes.

Next, Feature Extraction transforms the preprocessed windows into meaningful behavioral metrics such as forwarding ratios, retry rates, hop stability indicators, and RSSI–hop inconsistency values. These computed metrics form the structured Dataset, which contains engineered features together with the ground-truth phase labels corresponding to baseline operation, blackhole manipulation, and wormhole manipulation.

Finally, Exploratory Data Analysis (EDA) and Clustering are applied to this dataset. EDA is used to examine the statistical properties of the data and visualize behavioral patterns, while clustering is performed to determine whether normal operation, blackhole behavior, and wormhole-induced anomalies form separable groups within the feature space.

4.2. Experimental Topology and Controlled Behavior Framework

This section describes the experimental implementation of two routing manipulations—forwarding suppression (blackhole emulation) and topology distortion (wormhole-inspired emulation)—designed to generate labeled telemetry data in the ESP-WIFI-MESH testbed. Both attacks are implemented at the application layer without modifying the underlying ESP-WIFI-MESH firmware. Manipulations are activated only during controlled experiment windows, with the root node broadcasting phase transitions to all mesh participants. All telemetry is logged locally on each node and retrieved post-experiment for offline analysis.

4.2.1. Attack Implementation: Wormhole and Blackhole Manipulations

This section describes the step-by-step implementation procedures for generating two types of controlled routing manipulation in the ESP-WIFI-MESH testbed: forwarding suppression (blackhole emulation) and topology distortion (wormhole-inspired emulation). Both manipulations are implemented at the application layer without modifying the ESP-WIFI-MESH firmware or low-level protocol stacks. All behaviors are activated during precisely controlled time windows managed by the root node via phase broadcasts. Each node logs telemetry locally at 1 Hz, with phase labels embedded in every record. The procedures documented here ensure reproducible dataset generation across multiple experimental runs and topology configurations.

4.2.1.1. Common Experiment Control Infrastructure

All experiments share a common control infrastructure managed by the root node. This infrastructure ensures consistent phase timing, synchronized manipulation activation, and reliable ground-truth labeling across all network participants.

A. Phase Definition and Labeling

Each experimental run follows a fixed temporal structure with distinct phases. Two parallel numbering schemes are maintained:

- Phase ID: Used for experiment orchestration and broadcast messaging (0–4)
- Ground-Truth Label: Embedded in telemetry records for offline analysis (0, 1, 2)

Table 4.1 Phase Definition and Labeling

Phase ID	Ground-Truth Label	Duration	Phase Name	Description
0	0	300 s	Baseline	Normal mesh operation; no manipulation active
1	1	180 s	Blackhole Manipulation	Forwarding suppression enabled
2	2	180 s	Wormhole Manipulation	Tunnel-based shortcut enabled
3	0	120 s	Cooldown	Manipulation disabled; network stabilization
4	—	—	Terminate	Experiment end; logs ready for extraction

Note: The root broadcasts only the Phase ID. Each node maintains a local mapping from Phase ID to Ground-Truth Label, which is appended to every telemetry record for offline analysis.

B. Root Phase Controller Implementation

As shown in *Figure 4.2*, the root node executes a controller that manages the sequence of experiment phases, including baseline, manipulation (blackhole or wormhole), cooldown, and termination. Phase updates are broadcast to all mesh nodes using group addressing, with each transition repeated multiple times and tagged with a sequence number to improve reliability and ensure nodes adopt the most recent valid phase state.

Figure 4.2 Root Phase Controller

```
procedure EXPERIMENT_CONTROLLER(experiment_type):
    /**
     * Controls the timeline of a single experimental run.
     *
     * Input:
     *   experiment_type: "blackhole" or "wormhole"
     *
     * Behavior:
     *   - Broadcasts phase transitions (repeatedly) to all mesh nodes
     *   - Logs phase start/end events with timestamps
     *   - Activates manipulation flags via broadcast
     */

    // Phase 0: Baseline (5 minutes)
    current_phase ← 0
    broadcast_phase_repeated(current_phase, 5)   // 5 repetitions
    log_event("PHASE_START", current_phase, get_time())
    sleep(300)
    log_event("PHASE_END", current_phase, get_time())

    // Phase 1 or 2: Manipulation (3 minutes)
    if experiment_type == "blackhole":
        current_phase ← 1
    else:
        current_phase ← 2

    broadcast_phase_repeated(current_phase, 5)
    log_event("PHASE_START", current_phase, get_time())
    sleep(180)
    log_event("PHASE_END", current_phase, get_time())

    // Phase 3: Cooldown (2 minutes)
    current_phase ← 3
    broadcast_phase_repeated(current_phase, 5)
    log_event("PHASE_START", current_phase, get_time())
    sleep(120)
    log_event("PHASE_END", current_phase, get_time())

    // Phase 4: Terminate
    current_phase ← 4
    broadcast_phase_repeated(current_phase, 5)
    log_event("EXPERIMENT_END", current_phase, get_time())
end procedure

procedure broadcast_phase_repeated(phase, count):
    /**
     * Broadcasts the current phase repeatedly.
     * Message format: { phase_id, seq_num, timestamp }.
     */
    message ← pack(phase, get_global_seq(), get_time())
    for i = 1 to count:
        esp_mesh_send(ESP_MESH_ADDR_ALL, message, MESH_DATA_P2P, NULL, 0)
```

```
    sleep(100)   // short delay between repetitions
  end for
end procedure
```

C. Node-Side Phase Monitoring

Every mesh node runs a background monitoring task that continuously listens for broadcast phase messages from the root node, as implemented in *Figure 4.3*. When a phase update with a newer sequence number is received, the node updates its local phase state and corresponding ground-truth label. This mechanism ensures that all nodes remain synchronized with the experiment timeline while maintaining consistent labeling of the collected dataset.

Figure 4.3: Node Phase Monitor

```
// Global state maintained by each node
int current_phase_id = 0
int ground_truth_label = 0
uint32_t last_seq = 0
mutex phase_mutex

procedure PHASE_MONITOR_TASK():
  while experiment_active:
    message ← esp_mesh_recv(portMAX_DELAY)

    if is_phase_broadcast(message):
      (received_phase, seq, _) ← unpack(message)

      if seq > last_seq:
        lock(phase_mutex)
        current_phase_id ← received_phase
        last_seq ← seq

        // Map phase ID to ground-truth label
        if received_phase == 0:
          ground_truth_label ← 0
        else if received_phase == 1:
          ground_truth_label ← 1
        else if received_phase == 2:
          ground_truth_label ← 2
        else:
          ground_truth_label ← 0   // Cooldown also labeled 0
        unlock(phase_mutex)

        log_event("PHASE_UPDATE", received_phase, ground_truth_label, get_time())
      end if
    end if
  end while
end procedure

function get_current_label():
  lock(phase_mutex)
  label ← ground_truth_label
  unlock(phase_mutex)
```

```
    return label
end function
```

4.2.1.2. Forwarding Suppression (Blackhole) Implementation

The blackhole attacker operates as an application-layer relay: victim nodes intentionally send their application-layer probe packets to the attacker (i.e., destination = attacker's MAC address). The attacker either forwards these packets to the root (during baseline) or drops them (during the manipulation window). The attacker remains a legitimate mesh participant, maintaining parent relationships and responding to control messages.

A. Design Scope and Implementation Boundaries

The implementation described in this section is conducted exclusively within a controlled laboratory testbed composed of researcher-owned ESP32 development boards. The purpose of detailing the manipulation model is to ensure reproducibility of dataset generation for academic evaluation, not to enable deployment against external or production networks. All experiments are performed in an isolated laboratory network with no connection to external or production infrastructure and under controlled conditions with explicit activation windows and ground-truth labeling.

Canonical blackhole attacks described in wireless networking literature are typically implemented at the routing protocol level, often involving the advertisement of favorable routing metrics to attract traffic before dropping packets. Such implementations generally require manipulation of routing control messages and protocol state. However, these capabilities are not reliably exposed within the ESP-WIFI-MESH framework on ESP32 devices, where parent selection and forwarding decisions are handled internally by the protocol implementation.

ESP32 microcontrollers operate using a closed-source Wi-Fi firmware stack beneath the ESP-IDF application layer. Core networking operations—including parent selection, forwarding decisions, and mesh control-plane management—are handled internally by the ESP-WIFI-MESH protocol implementation. As a result, direct manipulation of routing advertisements or forwarding tables at the firmware level is not accessible through standard ESP-IDF APIs.

Accordingly, this study does not attempt to implement a protocol-level routing advertisement attack. Instead, the experiment adopts a blackhole-inspired forwarding suppression emulation model, designed to reproduce the observable behavioral effects of a blackhole attack rather than replicating its exact protocol-level mechanics.

B. Design Principles

The objective of this research is not to exploit or compromise the ESP-WIFI-MESH protocol, but rather to generate a reproducible hardware-derived dataset capturing cross-layer behavioral anomalies associated with forwarding suppression. To achieve this, the manipulation model is designed according to the following principles:

- Operate entirely within supported ESP-IDF and ESP-WIFI-MESH APIs
- Maintain full protocol compliance and mesh membership at all times
- Introduce controlled forwarding suppression that produces packet loss effects
- Induce measurable cross-layer inconsistencies consistent with blackhole theory
- Enable precise activation windows and reliable ground-truth labeling for dataset generation

This design ensures that the resulting dataset remains:

- Reproducible across multiple experimental runs
- Stable across ESP-IDF software versions
- Ethically contained within a controlled laboratory environment
- Suitable for exploratory analysis and offline intrusion-detection research

By focusing on observable cross-layer deviations—such as forwarding ratio degradation, increased retransmissions, and end-to-end packet loss—the study preserves the theoretical characteristics of blackhole-induced forwarding suppression while maintaining practical feasibility within the constraints of the ESP32 platform.

The resulting manipulation is therefore best described as behavioral blackhole emulation, rather than low-level protocol manipulation. This distinction is important for interpreting the collected dataset and for ensuring that future researchers clearly understand the experimental conditions under which the forwarding suppression traces were generated.

C. Attacker Role and Traffic Model

To reproduce the forwarding suppression effects associated with blackhole attacks, the experiment deploys a single attacker node that serves as an intentional relay for victim probe traffic. The traffic model follows a simple design:

Attacker Node. This node is positioned such that victim nodes select it as their parent and send their probe packets directly to it (destination = attacker's MAC address). During baseline operation, the attacker forwards these probes to the root. During the suppression window, it drops them instead.

Victim Nodes. These nodes are configured to send their application-layer probe packets to the attacker as the designated relay. This ensures that all victim traffic must pass through the attacker, enabling controlled forwarding decisions.

Root Node. The root receives probe packets forwarded by the attacker during baseline operation and records their arrival times as part of the measurement framework. During suppression phases, the root observes packet loss due to the attacker dropping relay traffic.

The attacker joins the mesh network as a legitimate participant and maintains normal protocol behavior throughout the experiment. No mesh control-plane messages are altered, and all communication with the network continues to use standard ESP-WIFI-MESH APIs.

Through this intentional relay mechanism, the attacker introduces controlled packet loss that distorts the expected forwarding behavior—thereby reproducing the cross-layer behavioral signatures associated with blackhole-induced forwarding suppression.

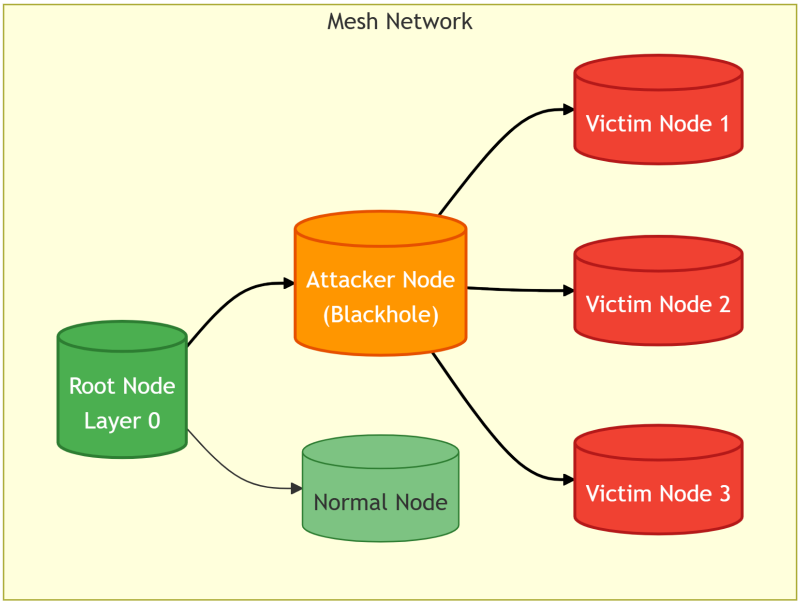

Figure 4.4 Blackhole Topology with Attacker as Intentional Relay

Figure 4.4 shows the blackhole topology with the attacker as an intentional relay. Victims (red) send their probe packets to the attacker (orange), which forwards them to the root (green) during baseline. During suppression, the attacker drops packets instead of forwarding them.

D. Attacker State Management

The blackhole attacker maintains counters and a state flag, as shown in Table 4.1. All counters are atomic to ensure thread safety, as they are accessed by both the packet processing and logging tasks.

Table 4.2 Blackhole Attacker State Variables and Counters

Variable	Type	Initial Value	Description	Update Trigger
suppression_active	bool	false	Manipulation state flag	Set true when phase ID = 1 received; cleared when phase ID = 3 received
recv_counter	uint32	0	Total application packets received	Incremented on every successful *esp_mesh_recv()*
forward_counter	uint32	0	Total packets forwarded to root	Incremented after successful *esp_mesh_send()* to root

drop_counter	uint32	0	Total packets intentionally dropped	Incremented when a packet is dropped during suppression
self_packet_counter	uint32	0	(Not used in this design)	—

E. Baseline Stabilization

Before Phase 0 (Baseline) begins, the mesh is allowed to converge to a stable topology for a short stabilization interval (e.g., 60 seconds). During this interval, the following baseline properties are recorded:

- Each node's mesh layer (hop distance), parent identity, and parent switching events (if any)
- Per-node RSSI statistics (mean/variance) under stable topology
- Baseline end-to-end latency and packet delivery ratio (PDR) from victim nodes to the root
- Verification that victims are correctly sending probes to the attacker (destination = attacker MAC)

This baseline serves as the reference distribution against which suppression effects are evaluated.

F. Packet Processing Logic

The attacker's main processing loop continuously receives packets from victims addressed to it. Based on the current phase and the suppression state, it either forwards the packets to the root or drops them during the attack phase. This behavior is illustrated in *Figure 4.5*.

Figure 4.5 Blackhole Attacker Relay Loop

```
/**
 * Main packet processing task for blackhole attacker.
 * Victims send application-layer probe packets to this node.
 *
 * Behavior:
 *   - During baseline (suppression_active = false): forward all received packets to root.
 *   - During suppression (suppression_active = true): drop all received packets.
 */

bool suppression_active = false
atomic_uint32_t recv_counter = 0
atomic_uint32_t forward_counter = 0
atomic_uint32_t drop_counter = 0

procedure ATTACKER_RELAY_TASK():
   while experiment_active:
      // Receive packet (blocks until packet arrives)
      packet ← esp_mesh_recv(portMAX_DELAY)
      phase_id ← get_current_phase()

      atomic_increment(recv_counter)

      if suppression_active and phase_id == 1:
         // Suppression active: drop the packet
```

```
            atomic_increment(drop_counter)
            // No forwarding call – packet is silently discarded
        else:
            // Normal relay: forward to root
            result ← esp_mesh_send(root_address,
                                   packet.data,
                                   packet.len,
                                   MESH_DATA_P2P,
                                   NULL, 0)
            if result == ESP_OK:
                atomic_increment(forward_counter)
            else:
                log_error("FORWARD_FAIL", result)
        end if

        taskYIELD()
    end while
end procedure
```

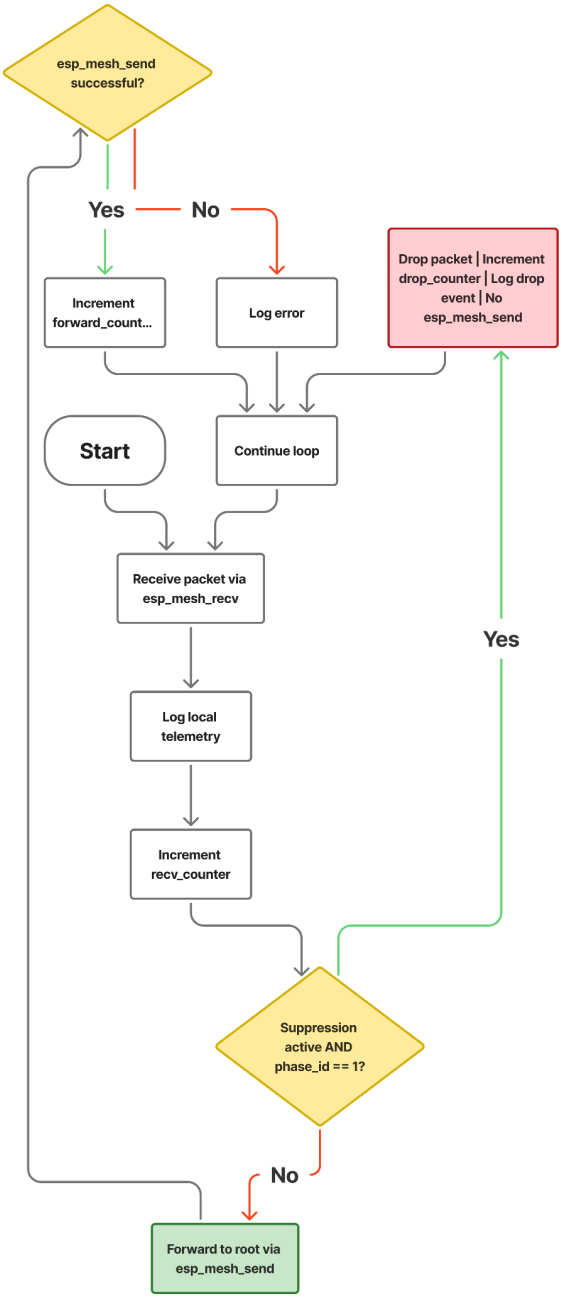

Figure 4.6 Blackhole Attacker Decision Flowchart

Figure 4.6 shows the blackhole attacker packet processing flowchart. The attacker receives packets, logs telemetry, and increments the receive counter. If suppression is active during phase 1, the packet is dropped, and the drop counter is incremented. Otherwise, the packet is forwarded to the root, and success/failure is tracked.

G. Explanation of Key Behaviors

The blackhole-inspired implementation creates a forwarding suppression effect through several coordinated behaviors:

1. **Intentional Relay:** Victims send their probe packets directly to the attacker (destination = attacker MAC). The attacker thus legitimately receives them via *esp_mesh_recv()*. This ensures that no promiscuous capture or transit traffic monitoring is required.
2. **Normal Forwarding:** During baseline (suppression_active = false), the attacker forwards each received packet to the root using *esp_mesh_send()*. The path is Victim → Attacker → Root, and the root receives the probe as usual. This establishes the baseline forwarding behavior and latency.
3. **Suppression Mechanism:** During the manipulation window (suppression_active = true and phase ID = 1), the attacker simply omits the *esp_mesh_send()* call for received packets. The packet buffer is freed, and no forwarding occurs. This creates controlled packet loss while the attacker continues normal mesh participation.
4. **Protocol Compliance:** The attacker never modifies packet contents, never injects malformed frames, and never disrupts control-plane exchanges (beacons, parent maintenance, etc.). Only application-layer relay behavior is altered. The attacker remains a legitimate mesh node in all other respects.
5. **Counter Updates:** Atomic counters track the exact number of received, forwarded, and dropped packets. The recv_counter increments for every packet, while forward_counter and drop_counter track the outcomes of forwarding decisions. These provide raw data for subsequent feature extraction.
6. **Normal Traffic Preservation:** The attacker continues to participate in all control-plane exchanges and maintains parent relationships throughout all phases. This ensures that the mesh protocol's behavior is minimally perturbed, allowing clean observation of the suppression effect.

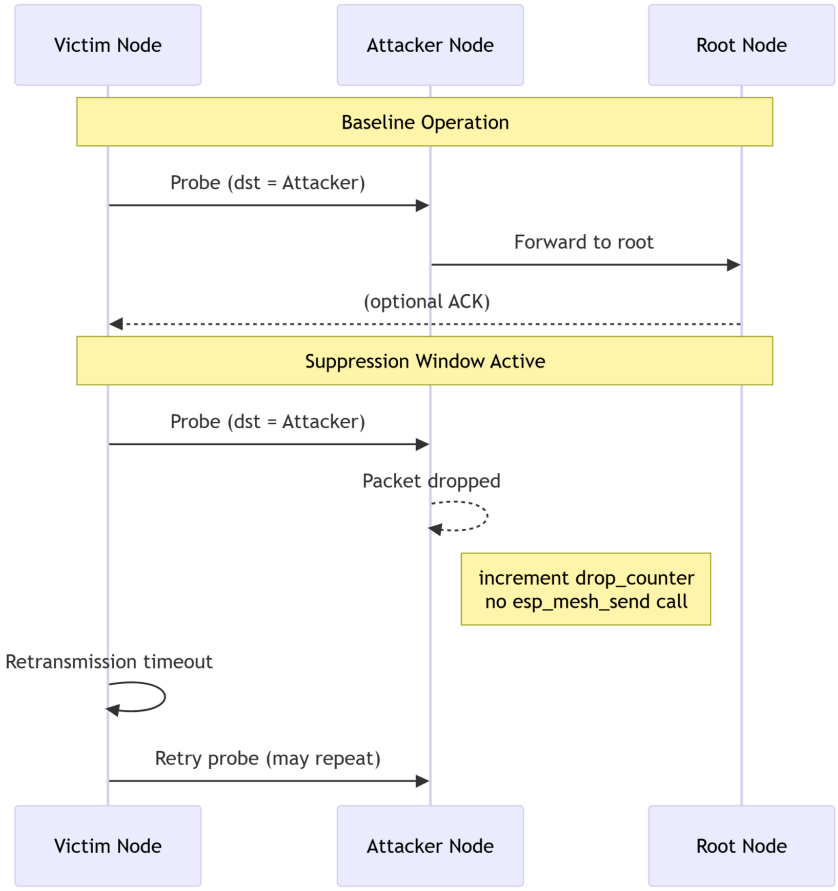

Figure 4.7 Baseline vs Suppression Sequence Diagram

Figure 4.7 is the comparison of baseline and suppressed operation. During baseline (top), probes flow, Victim → Attacker → Root. During suppression (bottom), the attacker drops packets, causing victims to experience timeouts and retransmissions.

H. Observable Effects and Dataset Contributions

The forwarding suppression mechanism produces several measurable deviations that are captured in the unified dataset:

- **Forwarding Ratio Degradation**: The attacker's forwarding ratio (packets forwarded / packets received) drops significantly during the suppression window, approaching zero if all transit packets are dropped. This is the primary indicator of blackhole behavior.
- **Increased Retransmissions**: Victim nodes experience timeouts and increase their retransmission attempts when probes are dropped. MAC-layer retry counters reflect this link-level stress.
- **Reduced Packet Delivery Ratio**: End-to-end packet delivery from victims to the root decreases during suppression, as probes are dropped at the attacker.
- **Stable RSSI at Attacker**: Unlike wormhole distortion, the attacker's RSSI remains consistent with its physical placement, providing a baseline for comparison.

- **Parent Switching (Secondary Effect)**: Persistent packet loss may trigger victims to attempt parent switching, though this is monitored as a secondary effect.

I. Implementation Summary for Blackhole

The blackhole-inspired implementation is designed around several key principles:

- **Intentional Relay:** Victims address their probes to the attacker, making it an intentional relay and eliminating any need for passive packet capture or transit traffic visibility.
- **Suppression by Omission:** Forwarding suppression is achieved by omitting the *esp_mesh_send()* call, not by modifying control traffic or injecting malformed packets.
- **Protocol Compliance:** The attacker remains a legitimate mesh participant throughout all phases, maintaining parent relationships and control-plane exchanges.
- **Counter-Based Telemetry:** Dedicated atomic counters track received, forwarded, and dropped packets, providing the raw data for subsequent feature extraction.
- **Phase-Controlled Activation:** The suppression_active flag is tied directly to experiment phase broadcasts, ensuring precise temporal alignment with ground-truth labels.
- **Normal Traffic Preservation:** The attacker continues forwarding all original packets during baseline, maintaining realistic mesh behavior during non-suppression windows.

4.2.1.3. Topology Distortion (Wormhole-Inspired) Implementation

The wormhole-inspired manipulation uses two colluding attacker endpoints connected by an out-of-band tunnel. No raw 802.11 frames are captured or injected; only application-layer probe metadata is relayed through the tunnel. This approach maintains full protocol compliance while generating measurable cross-layer inconsistencies. Both attackers remain legitimate mesh participants throughout the experiment.

A. Design Scope and Implementation Boundaries

The implementation described in this section is conducted exclusively within a controlled laboratory testbed composed of researcher-owned ESP32 development boards. The purpose of detailing the manipulation model is to ensure reproducibility of dataset generation for academic evaluation, not to enable deployment against external or production networks. All experiments are performed in an isolated laboratory network with no connection to external or production infrastructure and under controlled conditions with explicit activation windows and ground-truth labeling.

Canonical wormhole attacks described in wireless networking literature are typically implemented at the IEEE 802.11 control-plane level, often involving the capture, replay, or manipulation of management frames and topology-related information elements. Such implementations generally require low-level driver access and raw frame injection capabilities. However, these capabilities are not reliably exposed within the ESP-WIFI-MESH framework on ESP32 devices.

ESP32 microcontrollers operate using a closed-source Wi-Fi firmware stack beneath the ESP-IDF application layer. Core networking operations—including parent selection, beacon generation, association handling, and mesh control-plane management—are handled internally by the ESP-WIFI-MESH protocol implementation. As a result, direct manipulation of IEEE 802.11 management frames, vendor-specific information elements, or mesh discovery advertisements at the firmware level is not accessible through standard ESP-IDF APIs.

Accordingly, this study does not attempt to implement a firmware-level management-frame replay wormhole. Instead, the experiment adopts a wormhole-inspired topology distortion emulation model, designed to reproduce the observable behavioral effects of a wormhole rather than replicating its exact physical-layer mechanics.

B. Design Principles

The objective of this research is not to exploit or compromise the ESP-WIFI-MESH protocol, but rather to generate a reproducible hardware-derived dataset capturing cross-layer behavioral anomalies associated with routing-based topology distortion. To achieve this, the manipulation model is designed according to the following principles:

- Operate entirely within supported ESP-IDF and ESP-WIFI-MESH APIs
- Maintain full protocol compliance and mesh membership at all times
- Introduce a controlled auxiliary tunnel that produces artificial shortcut effects
- Induce measurable cross-layer inconsistencies consistent with wormhole theory
- Enable precise activation windows and reliable ground-truth labeling for dataset generation

This design ensures that the resulting dataset remains:

- Reproducible across multiple experimental runs
- Stable across ESP-IDF software versions
- Ethically contained within a controlled laboratory environment
- Suitable for exploratory analysis and offline intrusion-detection research

By focusing on observable cross-layer deviations—such as hop–latency inconsistencies, weakened RSSI–hop relationships, topology instability, and traffic concentration shifts—the study preserves the theoretical characteristics of wormhole-induced topology distortion while maintaining practical feasibility within the constraints of the ESP32 platform.

The resulting manipulation is therefore best described as behavioral wormhole emulation, rather than low-level protocol manipulation. This distinction is important for interpreting the collected dataset and for ensuring that future researchers clearly understand the experimental conditions under which the topology distortion traces were generated.

C. Attacker Roles and Tunnel Architecture

To reproduce the shortcut effects associated with wormhole attacks, the experiment deploys two cooperating attacker nodes connected by an auxiliary out-of-band tunnel. Each attacker performs a distinct role within the topology distortion model.

Two attacker endpoints are deployed with the following responsibilities:

- **Attacker A (Root-Side Endpoint).**
  This node is positioned near the root region of the mesh network. It receives tunneled probe metadata from the remote attacker and reconstructs shortcut probe responses. These responses are then injected toward the root node using standard ESP-WIFI-MESH messaging, creating the appearance of accelerated communication paths.
- **Attacker B (Leaf-Side Endpoint).**

This node is positioned near the victim leaf nodes at the network edge. Victim nodes send their probe packets to Attacker B, which acts as the designated relay. During baseline operation, Attacker B forwards these probes normally toward the root node. During distortion phases, it extracts probe metadata and transmits it through the out-of-band tunnel to Attacker A.

Both attackers join the mesh network as legitimate participants and maintain normal protocol behavior throughout the experiment. No mesh control-plane messages are altered, and all communication with the network continues to use standard ESP-WIFI-MESH APIs.

Through this coordinated relay mechanism, the tunnel introduces artificial shortcut effects that distort the expected relationship between physical distance, hop count, and latency—thereby reproducing the cross-layer behavioral signatures associated with wormhole-induced topology distortion.

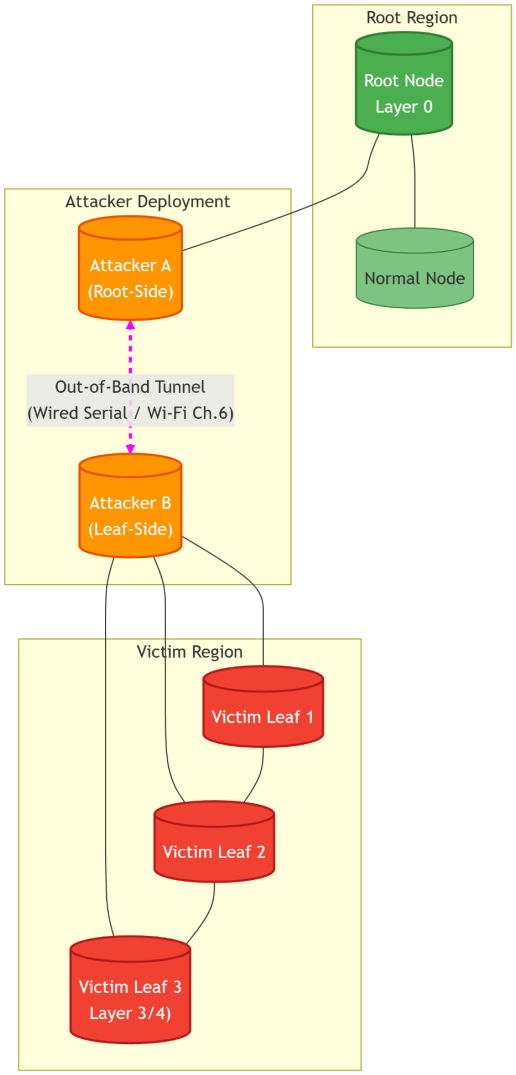

Figure 4.8 Wormhole Endpoint Setup with Auxiliary Tunnel

Figure 4.8 is the Wormhole endpoint setup with an auxiliary tunnel. Attacker A (orange) is positioned near the root region (green), while Attacker B (orange) is positioned near the victim leaf nodes (red). The

out-of-band tunnel (magenta dashed line) connects the two attackers independently of the mesh network, enabling the shortcut effect without interfering with normal mesh routing.

D. Tunnel Implementation

The out-of-band tunnel is implemented as a point-to-point communication channel independent of the mesh network shown in *Figure 4.9*. Several implementation options exist, all providing similar functionality:

- Wired serial: UART connection between the two ESP32 boards
- Host relay: Both boards connected to a laptop via USB, with software forwarding
- Dedicated Wi-Fi: Secondary Wi-Fi interfaces on a different channel

For consistency, all implementations use a common packet format for tunneled metadata.

Figure 4.9 Wormhole Tunnel Packet Structure

```
struct tunnel_packet {
   uint32_t magic;      // 0x544E4C44 ("TUNL")
   uint32_t seq;        // Sequence number for ordering
   uint32_t len;        // Payload length
   uint8_t  data[256];  // Metadata payload
   uint32_t crc;        // CRC32 for integrity
};
```

The tunnel is configured to exhibit:

- Stable connectivity throughout baseline and manipulation windows
- Lower and more consistent latency than the equivalent multi-hop mesh route under baseline conditions
- No dependence on ESP-MESH routing decisions

E. Attacker State Management

Both attackers maintain counters to track tunnel activity and reinjection operations. Table 4.2 lists all state variables.

Table 4.3 Wormhole Attacker State Variables and Counters

Variable	Type	Initial Value	Description	Update Trigger
distortion_active	bool	false	Manipulation flag	Set true when phase ID = 2 received; cleared when phase ID = 3 received
tunnel_tx_counter	uint32	0	Messages sent through the tunnel (Attacker B)	Incremented on each tunnel send

tunnel_rx_counter	uint32	0	Messages received from tunnel (Attacker A)	Incremented on each tunnel received
tunnel_bytes_tx	uint32	0	Bytes sent through the tunnel	Accumulated per tunnel send
tunnel_bytes_rx	uint32	0	Bytes received from tunnel	Accumulated per tunnel received
inject_counter	uint32	0	Probes reinjected into the root (Attacker A)	Incremented after successful *esp_mesh_send()* of reconstructed probe
capture_queue_size	uint16	0	Current queue depth (Attacker B)	Updated on queue push/pop

All counters are implemented as atomic variables to ensure thread safety, as they are accessed by both the packet processing task and the logging task.

F. Baseline Stabilization

Before Phase 0 (Baseline) begins, the mesh is allowed to converge to a stable topology for a short stabilization interval (e.g., 60 seconds). During this interval, the following baseline properties are recorded:

- Each node's mesh layer (hop distance), parent identity, and parent switching events (if any)
- Per-node RSSI statistics (mean/variance) under stable topology
- Baseline end-to-end latency and packet delivery ratio (PDR) from victim region nodes to the root

This baseline serves as the reference distribution against which distortion effects are evaluated.

G. Attacker B (Leaf-side) Implementation

Attacker B, as shown in *Figure 4.10*, receives probe packets from victims addressed to B. During baseline, it forwards them to the root. During distortion, it extracts metadata, sends it through the tunnel, and continues forwarding the original packets to maintain normal traffic patterns.

Figure 4.10 Attacker B – Relay and Tunnel

```
/**
 * Attacker B: Leaf-side endpoint of wormhole manipulation.
 *
```

```
 * Responsibilities:
 *   - Joins mesh normally and maintains local state
 *   - During baseline: forwards probes to root normally
 *   - During distortion: extracts metadata, relays through tunnel,
 *     and continues forwarding original packets
 */

// Global state
bool distortion_active = false
atomic_uint32_t tunnel_tx_counter = 0
atomic_uint32_t tunnel_bytes_tx = 0
queue_t relay_queue  // FIFO queue for pending metadata
address_t root_address  // Pre-configured root MAC address

procedure ATTACKER_B_TASK():
    // Initialize mesh participation
    esp_mesh_join()

    // Establish tunnel connection to Attacker A
    tunnel_connect(peer_address)

    while experiment_active:
        // Receive mesh packet with 100ms timeout
        packet ← esp_mesh_recv(100)  // timeout in milliseconds
        phase_id ← get_current_phase()

        if packet != NULL:
            // Always log local mesh state (layer, parent, RSSI)
            log_local_telemetry()

            // Step 1: Check if this is a probe packet during distortion
            if distortion_active and phase_id == 2 and is_probe_packet(packet):
                // Extract metadata from probe
                metadata ← extract_probe_metadata(packet)

                // Add capture timestamp
                metadata.t_capture = get_time()

                // Queue for tunnel transmission
                queue_push(relay_queue, metadata)

                // Log capture event for debugging
                log_event("PROBE_CAPTURE", metadata.seq, metadata.t_capture)
            end if

            // Step 2: Always forward original packet to root (normal relay)
            // This maintains normal mesh behavior
            esp_mesh_send(
                root_address,
                packet.data,
                packet.len,
                MESH_DATA_P2P,
                NULL,
                0
```

```
            )
        end if

        // Step 3: Process relay queue (send pending metadata through tunnel)
        while queue_size(relay_queue) > 0 and distortion_active:
            metadata ← queue_pop(relay_queue)

            // Prepare tunnel packet
            tunnel_pkt ← pack_tunnel_packet(metadata)

            // Send through tunnel (non-mesh path)
            bytes_sent ← tunnel_send(&tunnel_pkt, sizeof(tunnel_pkt))

            // Update counters
            atomic_add(tunnel_tx_counter, 1)
            atomic_add(tunnel_bytes_tx, bytes_sent)

            // Log tunnel transmission
            log_event("TUNNEL_TX", metadata.seq, bytes_sent)
        end while

        // Yield to allow other tasks
        taskYIELD()
    end while
end procedure

function is_probe_packet(packet):
    /**
     * Determines if a packet is a probe that should be relayed.
     * Probes are identified by application data type and subtype.
     * All probes are addressed to Attacker B as per experiment design.
     */
    return (packet.type == APP_DATA and
            packet.subtype == PROBE_TYPE)
end function

function extract_probe_metadata(packet):
    /**
     * Extracts relevant metadata from probe packet.
     * Returns a structure containing:
     *   - Source node ID
     *   - Probe sequence number
     *   - Original transmission timestamp
     */
    metadata.seq ← packet.sequence
    metadata.src ← packet.source
    metadata.t_send ← packet.t_send
    return metadata
end function
```

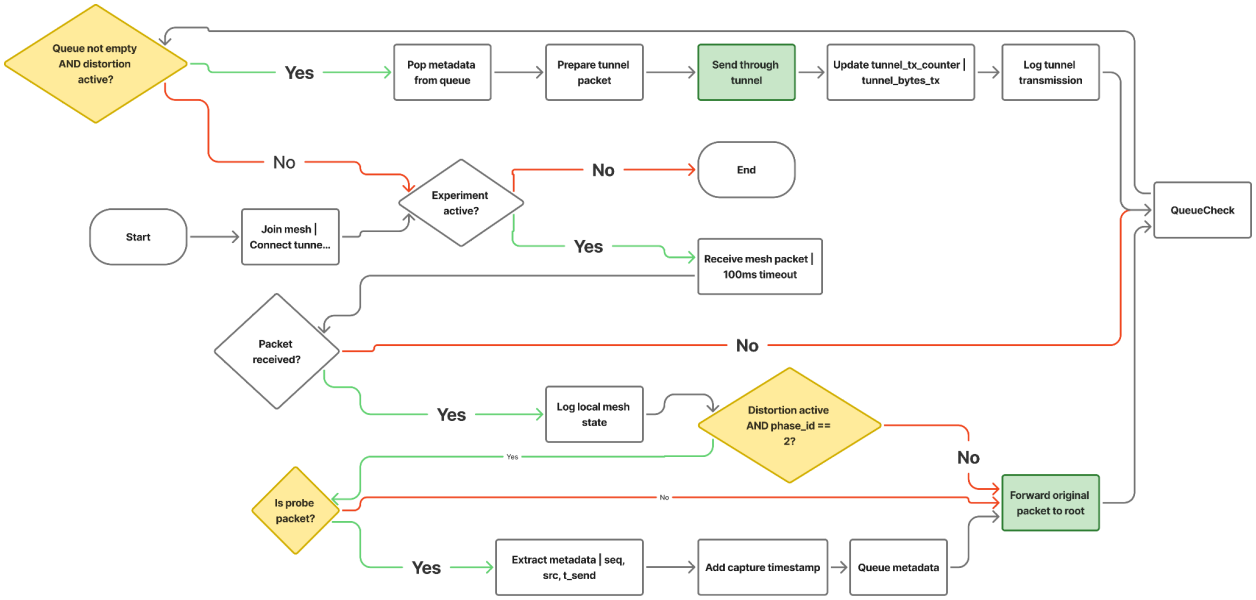

Figure 4.11 Attacker B (Leaf-side) Process Flowchart

Figure 4.11 shows Attacker B (leaf-side) process flowchart. The attacker receives packets, logs telemetry, and forwards all original packets to the root. During distortion, probe packets are identified, metadata is extracted, and queued. Queued metadata is sent through the out-of-band tunnel, with counters updated accordingly.

H. Attacker A (Root-side) Implementation

Attacker A receives tunneled metadata and reconstructs a probe replica, as shown in *Figure 4.12*, that mimics the original probe format. This reconstructed probe is then sent toward the root via normal mesh messaging, creating the appearance that a probe from the victim arrived through an unusually fast path.

Figure 4.12 Attacker A – Tunnel Receive and Inject

```
/**
 * Attacker A: Root-side endpoint of wormhole manipulation.
 *
 * Responsibilities:
 *   - Joins mesh normally and maintains local state
 *   - During distortion window, receives metadata from tunnel
 *   - Reconstructs probe replicas from metadata
 *   - Injects reconstructed probes toward root via mesh
 */

// Global state
bool distortion_active = false
atomic_uint32_t tunnel_rx_counter = 0
atomic_uint32_t tunnel_bytes_rx = 0
atomic_uint32_t inject_counter = 0
address_t root_address  // Pre-configured root MAC address

procedure ATTACKER_A_TASK():
   // Initialize mesh participation
   esp_mesh_join()
```

```
    // Start tunnel server to receive from Attacker B
    tunnel_server_start()

    while experiment_active:
        phase_id ← get_current_phase()

        // Always log local mesh state
        log_local_telemetry()

        // Step 1: Check for tunnel data during distortion window
        if distortion_active and phase_id == 2 and tunnel_available():
            // Receive metadata from tunnel
            metadata ← tunnel_receive()

            // Update counters
            atomic_add(tunnel_rx_counter, 1)
            atomic_add(tunnel_bytes_rx, sizeof(metadata))

            // Log tunnel reception
            log_event("TUNNEL_RX", metadata.seq, get_time())

            // Step 2: Reconstruct probe replica from metadata
            probe_replica ← create_probe_replica(metadata)
            probe_replica.t_tunnel_in = get_time()

            // Step 3: Send reconstructed probe to root via mesh
            result ← esp_mesh_send(
                root_address,
                &probe_replica,
                sizeof(probe_replica),
                MESH_DATA_P2P,
                NULL,
                0
            )

            if result == ESP_OK:
                atomic_add(inject_counter, 1)
                log_event("INJECT_SUCCESS", metadata.seq, get_time())
            else:
                log_event("INJECT_FAIL", metadata.seq, result)
            end if
        end if

        // Step 4: Also handle normal mesh traffic
        packet ← esp_mesh_recv(10)  // 10ms timeout
        if packet != NULL:
            handle_normal_traffic(packet)
        end if

        taskYIELD()
    end while
end procedure
```

```
function create_probe_replica(metadata):
    /**
     * Reconstructs a probe replica from tunneled metadata.
     * The replica duplicates the original probe format.
     */
    replica.type ← APP_DATA
    replica.subtype ← PROBE_TYPE      // Same type as original probe
    replica.seq ← metadata.seq
    replica.src ← metadata.src
    replica.dst ← root_address        // Destination is root
    replica.t_send_orig ← metadata.t_send
    replica.t_capture ← metadata.t_capture
    replica.t_tunnel_in ← 0           // Set by caller
    replica.t_reconstruct ← get_time()
    return replica
end function
```

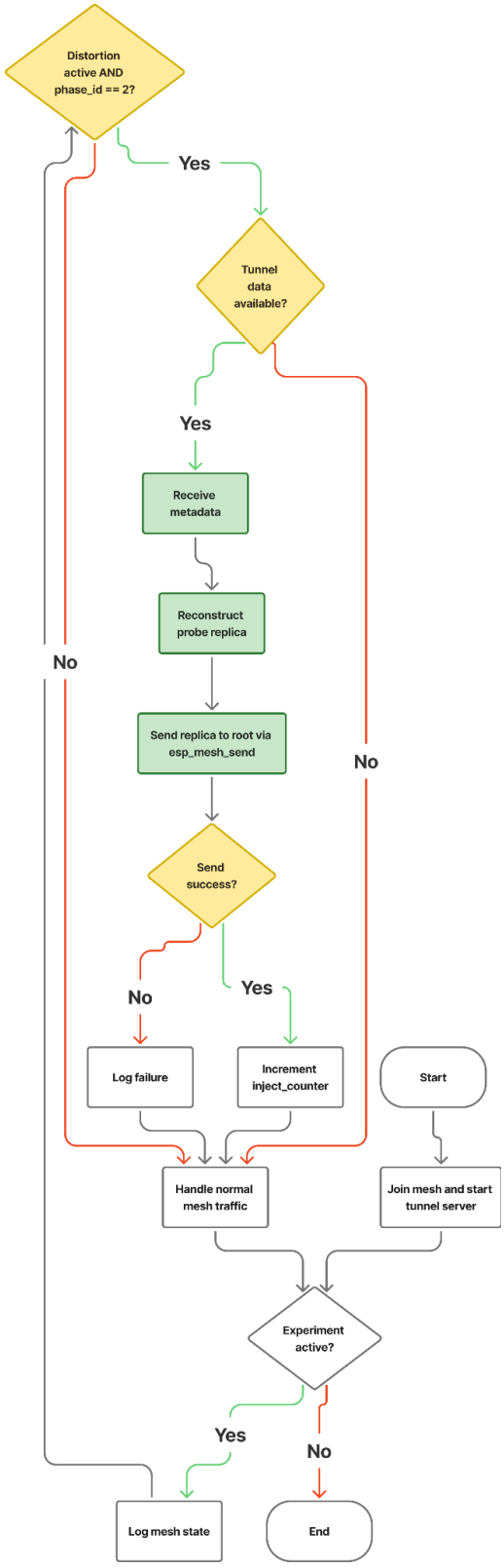
Distortion active AND phase_id == 2?
Yes
Tunnel data available?
Yes
Receive metadata
Reconstruct probe replica
Send replica to root via esp_mesh_send
Send success?
No
Yes
Log failure
Increment inject_counter
No
No
Start
Handle normal mesh traffic
Join mesh and start tunnel server
Experiment active?
Yes
No
Log mesh state
End

Figure 4.13 Attacker A (Root-side) Process Flowchart

In *Figure 4.13,* Attacker A (root-side) process flowchart. The attacker logs telemetry and checks for tunnel data during distortion. When metadata is received, a probe replica is reconstructed and sent to the root via the mesh. Normal mesh traffic is also handled concurrently.

F. Explanation of Key Behaviors

The wormhole-inspired implementation creates an artificial shortcut effect through several coordinated behaviors:

1. **Probe Reception**: Victims send their probe packets to Attacker B (the leaf-side endpoint). B thus legitimately receives them via *esp_mesh_recv()*. This ensures that no raw frame capture or promiscuous mode is required.
2. **Normal Relay**: During baseline, B simply forwards each probe to the root using *esp_mesh_send()*. The path is Victim → B → Root, and the root receives the probe as usual. This establishes the baseline latency for the path.
3. **Distortion Relay**: During the manipulation window, B extracts probe metadata (sequence number, source node ID, original transmission timestamp) and sends it through the out-of-band tunnel to A. The tunneled message contains only the fields needed to reproduce timing and identity (sequence number, source, timestamps), reducing tunnel overhead compared to relaying full traffic.
4. **Probe Reconstruction**: A receives the metadata and reconstructs a probe replica that mimics the original probe format. The replica contains the same sequence number and source information as the original, with the destination set to the root address.
5. **Mesh Reinjection**: A injects the reconstructed probe toward the root via normal mesh messaging. The root therefore receives two copies of the probe: one via the normal multi-hop path through B (slower) and one via the tunnel shortcut (faster). This creates an apparent reduction in path length, as the tunneled copy arrives earlier than the original. The root treats the replica as a valid probe because it matches the application probe schema and sequence number, enabling duplicate-arrival and latency anomalies to be measured without modifying the mesh control plane.
6. **Normal Traffic Preservation**: Both attackers continue to forward all original packets normally. This ensures that the mesh protocol's behavior is minimally perturbed, allowing clean observation of the shortcut effect.
7. **Tunnel Independence**: The tunnel operates on a separate channel (or physical medium) and does not interfere with mesh communication. Its lower latency relative to the mesh path is what creates the observable shortcut effect.

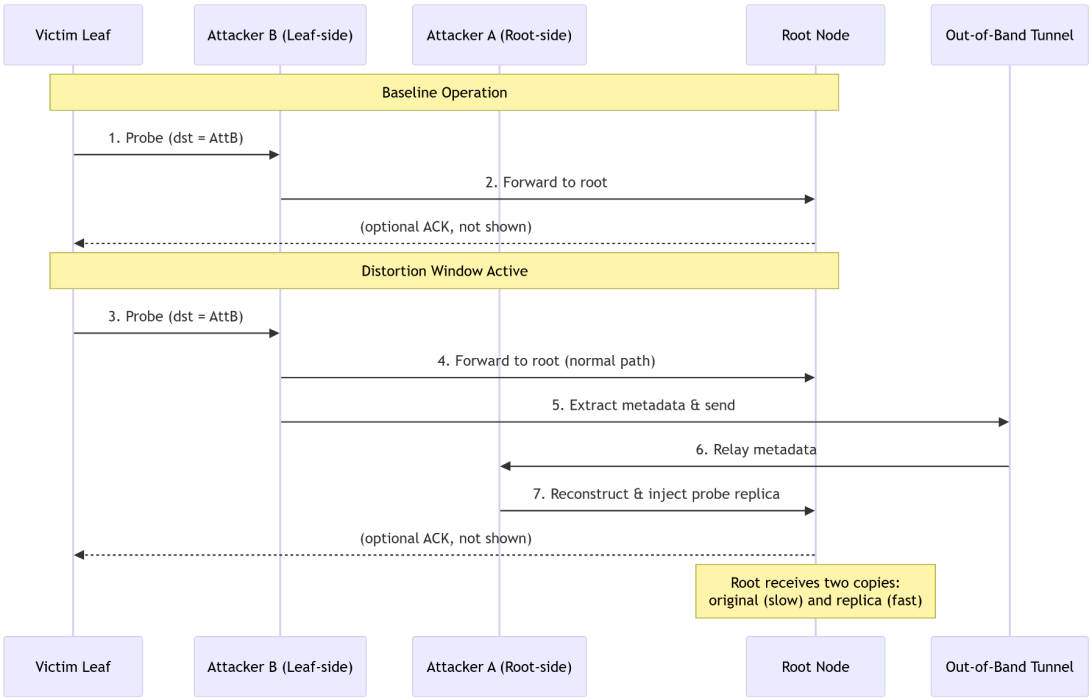

Figure 4.14 Wormhole Shortcut Relay Mechanism

Figure 4.14 is the Wormhole shortcut relay mechanism. During baseline (top), victims send probes to Attacker B, which forwards them to the root. During distortion (bottom), Attacker B also extracts metadata and sends it through the out-of-band tunnel to Attacker A, which reconstructs a probe replica and injects it toward the root. The root thus receives two copies, with the tunneled copy arriving faster, creating an apparent shortcut effect.

J. Observable Effects and Dataset Contributions

The topology distortion mechanism produces several measurable deviations that are captured in the unified dataset:

- **Reduced End-to-End Latency:** Probes arriving via the tunnel shortcut exhibit lower latency than those traversing the normal multi-hop path, creating an inconsistency between reported hop count and observed delay.
- **Duplicate Probe Reception:** The root receives two copies of the same probe (identical sequence numbers) within a short time window, which does not occur during normal operation.
- **Traffic Concentration Shifts:** The injection of shortcut probes may alter traffic distribution patterns, potentially concentrating forwarding activity on attacker-adjacent paths.
- **RSSI-Hop Inconsistency:** The relationship between physical signal strength and logical hop count may weaken as traffic patterns shift, though this is monitored as a secondary effect.
- **Topology Instability:** The presence of shortcut paths may coincide with topology instability (e.g., parent switching) when combined with natural RSSI variation and traffic load changes, and is therefore monitored as a secondary effect.

All these effects are captured through the cross-layer metrics defined in Section 4.2.3 and contribute to the feature set used for exploratory analysis and clustering.

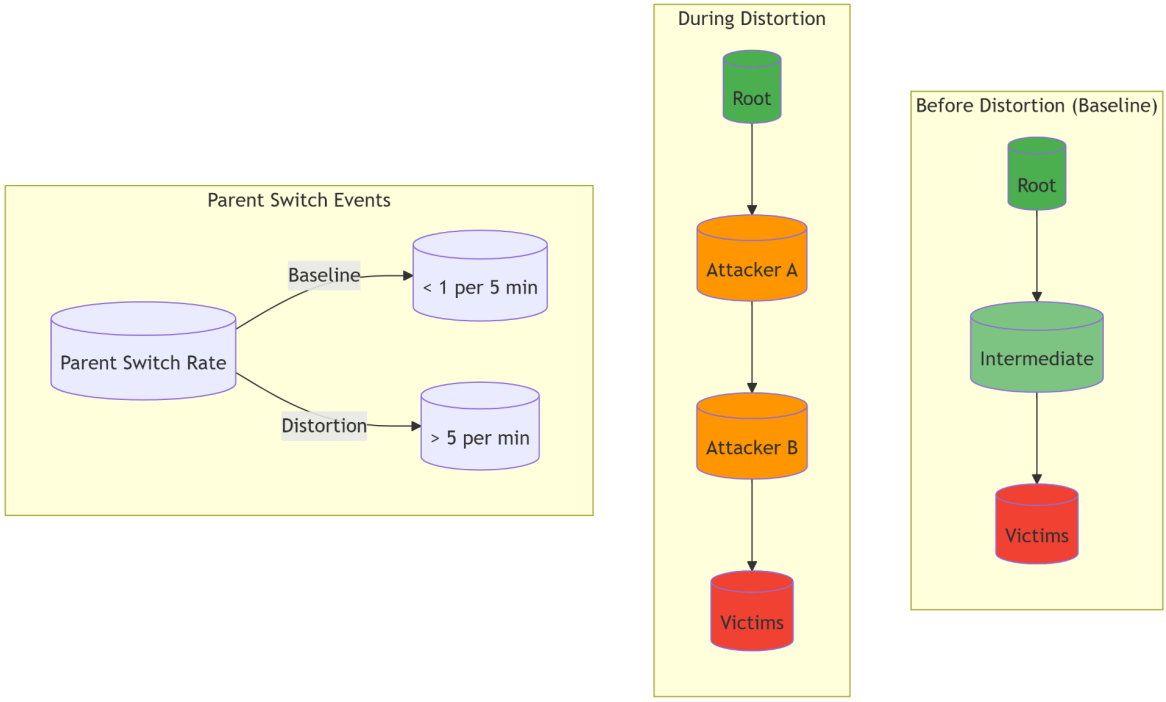

Figure 4.15 Traffic Redistribution During Distortion Window

Figure 4.15 shows the traffic redistribution and topology instability during the distortion window. Left: baseline traffic flows through legitimate intermediate nodes. Center: during distortion, traffic concentrates on attacker-adjacent paths (orange). Right: parent switching frequency increases significantly during the distortion window compared to baseline.

K. Implementation Summary for Wormhole

The wormhole-inspired implementation is designed around several key principles:

- **Intentional Relay**: Victims address their probes to Attacker B, making B an intentional relay and eliminating any need for passive packet capture.
- **Metadata-Only Tunneling**: Only essential probe metadata (sequence number, source, timestamps) is tunneled, not raw frames, ensuring application-layer compliance and minimal overhead.
- **Probe Duplication, Not Modification**: Attackers never modify original packets; they only create additional replicas, preserving all original traffic for baseline comparison.
- **Independent Tunnel**: The out-of-band channel operates completely separately from the mesh network, preventing interference with normal protocol operation.
- **Normal Traffic Preservation**: Both attackers continue forwarding original packets, maintaining realistic mesh behavior during the distortion window.
- **Counter-Based Telemetry**: Dedicated atomic counters track tunnel activity and reinjection success, providing the raw data for subsequent feature extraction.

- **Phase-Controlled Activation**: The distortion_active flag is tied directly to experiment phase broadcasts, ensuring precise temporal alignment with ground-truth labels.

4.2.1.4. Implementation Summary

The implementation described in this section provides a complete, reproducible framework for generating labeled datasets containing both normal and manipulated mesh behavior. Key characteristics include:

1. **Application-layer only:** All manipulations are implemented using standard mesh APIs; no firmware modifications or raw frame injection are performed.
2. **Phase-controlled activation:** Manipulations are activated only during designated time windows via root broadcasts, ensuring precise temporal alignment with ground-truth labels.
3. **Protocol compliance:** Attackers remain legitimate mesh participants throughout all phases, maintaining parent relationships and control-plane exchanges. Only the specific relay behavior (blackhole) or tunnel relay (wormhole) is modified.
4. **Comprehensive state tracking:** Each attacker maintains dedicated atomic counters that capture the intensity and outcome of manipulations. These counters, together with event-driven logs (parent changes, phase transitions, send/recv outcomes), provide rich raw data for subsequent feature extraction.
5. **Reproducible procedures:** The documented algorithms and state machines can be implemented identically across multiple experimental runs and topology configurations.

Telemetry logging, data storage, and post-experiment extraction are described separately in Section 4.2.3. This separation keeps the implementation logic focused on manipulation behavior, while data collection is handled by a unified framework common to all node types.

4.2.2. Physical Deployment Topologies

In ESP-WIFI-MESH, the network topology is not manually configured as in traditional networks, such as bus, ring, or star. Instead, nodes automatically organize themselves into a hierarchical tree structure via the protocol's parent-selection mechanism, in which each node selects an upstream parent based on metrics such as Received Signal Strength Indicator (RSSI), link quality, and hop count. This self-organizing behavior allows the mesh network to construct routing paths without predefined topology assignments dynamically.

However, in controlled experimental environments such as this study, different logical layouts can be created by physically positioning ESP32 nodes in specific spatial arrangements. While these layouts resemble traditional network topologies (e.g., star or linear chain), the underlying routing behavior still follows the ESP-WIFI-MESH hierarchical tree architecture. The resulting logical topology therefore emerges from the interaction between node placement, RSSI relationships, and the mesh protocol's parent-selection algorithm.

The physical placement of nodes plays a critical role in shaping the resulting logical network structure. Because parent selection is influenced by RSSI and link stability, node spacing and relative positioning must be carefully arranged to encourage the desired routing relationships. Proper spacing helps ensure that nodes preferentially connect to intended neighbors rather than forming unintended long-range links. This stabilization of RSSI relationships is essential for maintaining consistent topology behavior across

experimental runs and for ensuring that attack effects can be clearly observed in the collected telemetry data.

Node placement is particularly important when deploying routing-based attacks such as wormhole and blackhole attacks. The positioning requirements differ fundamentally between the two attack types:

For blackhole attacks, the attacker must be positioned where it can become the preferred parent node for multiple victim nodes. This is achieved by placing the attacker so that it exhibits stronger RSSI values or more favorable routing metrics than neighboring devices. Once selected as the parent, the attacker receives traffic from downstream nodes but intentionally drops packets instead of forwarding them during the manipulation window.

For wormhole attacks, two attacker nodes are deployed in physically distant regions of the network, with one attacker positioned near the root node and the other positioned near leaf or victim nodes at the network edge. Under normal operation, communication between these regions would require packets to traverse multiple intermediate nodes through the mesh hierarchy. The wormhole attack introduces an out-of-band tunnel between the two attackers, allowing captured probe metadata to be relayed directly across distant network segments. This creates an artificial shortcut in the perceived network topology, potentially reducing observed latency and distorting the expected relationship between hop count and RSSI.

To investigate how these attack behaviors manifest under different routing conditions, the experiments employ several physical deployment layouts, each designed to emphasize different aspects of mesh routing behavior. It is important to note that these layouts represent physical node arrangements used for experimentation rather than manually enforced routing topologies. The mesh protocol still constructs its routing tree dynamically based on link metrics. The following deployment layouts are implemented in the experimental environment:

- Star topology
- Tree topology
- Linear chain topology
- Partial mesh topology

Each layout introduces different communication patterns and routing constraints that influence how attacks propagate through the network.

4.2.2.1. Star Topology

In the **star topology**, a central ESP32 node acts as the communication hub, while surrounding nodes transmit traffic directly to it. This setup provides a baseline environment where routing complexity is minimized, and direct RSSI relationships can be observed.

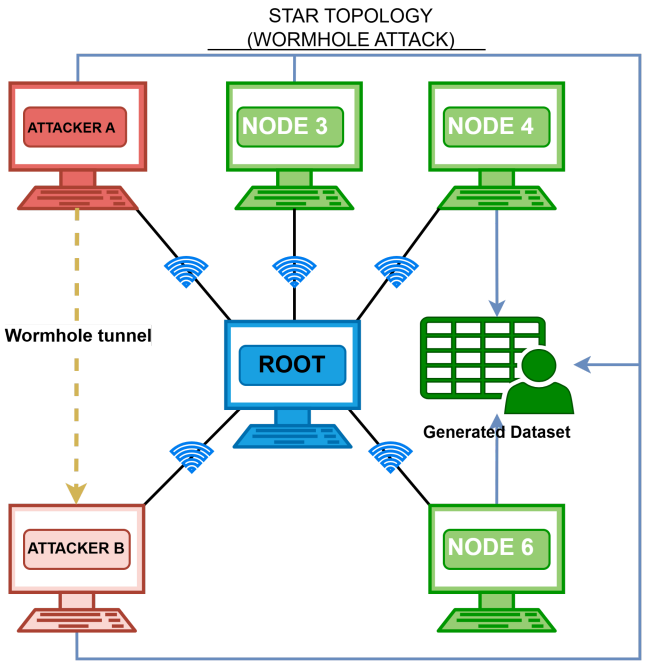

Figure 4.16 Wormhole Attack on Star Topology

For wormhole experiments in *Figure 4.16*, two attacker endpoints are deployed: Attacker A is placed adjacent to the root node, and Attacker B is placed near the farthest victim group on the perimeter. These endpoints are connected via an out-of-band tunnel operating independently of the mesh network. During the baseline phase, communication between the root-side and perimeter regions follows the natural star topology through the central hub. When the distortion window is activated, Attacker B captures probe metadata from victim-side traffic and relays it through the tunnel to Attacker A, which reconstructs and forwards shortcut probe responses near the root node, creating an artificial shortcut effect. The physical star layout remains unchanged, but the logical routing behavior becomes distorted as responses appear to arrive faster than the normal communication path through the star topology would allow. All nodes log continuous telemetry, with tunnel endpoints recording additional counters for messages relayed and bytes transferred.

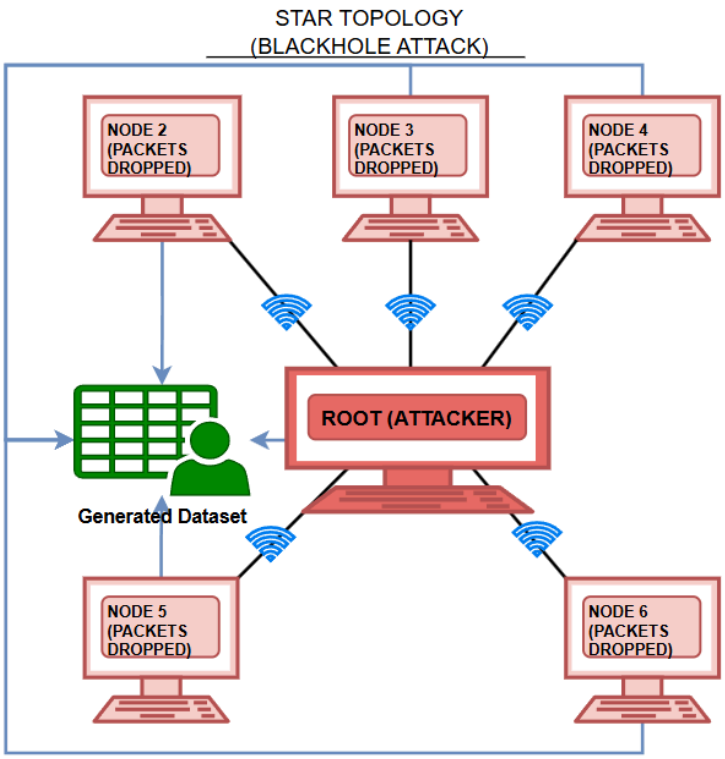

Figure 4.17 Blackhole Attack on Star Topology

For blackhole experiments in *Figure 4.17*, the root node is configured as the attacker, with victim nodes connecting directly to it as their parent, maintaining the star topology in which all traffic from perimeter nodes is directed toward the central node. During the baseline phase, all perimeter nodes establish stable parent relationships, with victims forwarding traffic to the root node. When the blackhole manipulation is activated, the attacker continues to receive packets from victims but intentionally drops them rather than forwarding them, causing packet loss and triggering retransmissions. Throughout the experiment, all nodes log telemetry, including timestamps, node IDs, phase labels, layer information, parent MAC addresses, RSSI values, and role-specific counters. The dataset captures the contrast between normal forwarding behavior during baseline and the packet-dropping behavior during the manipulation window.

4.2.2.2. Tree Topology

The **tree topology** reflects the native ESP-WIFI-MESH structure, where nodes are layered beneath a root node. Intermediate nodes connect upward to a parent while accepting downstream child connections, enabling multi-hop packet forwarding within the hierarchical mesh architecture.

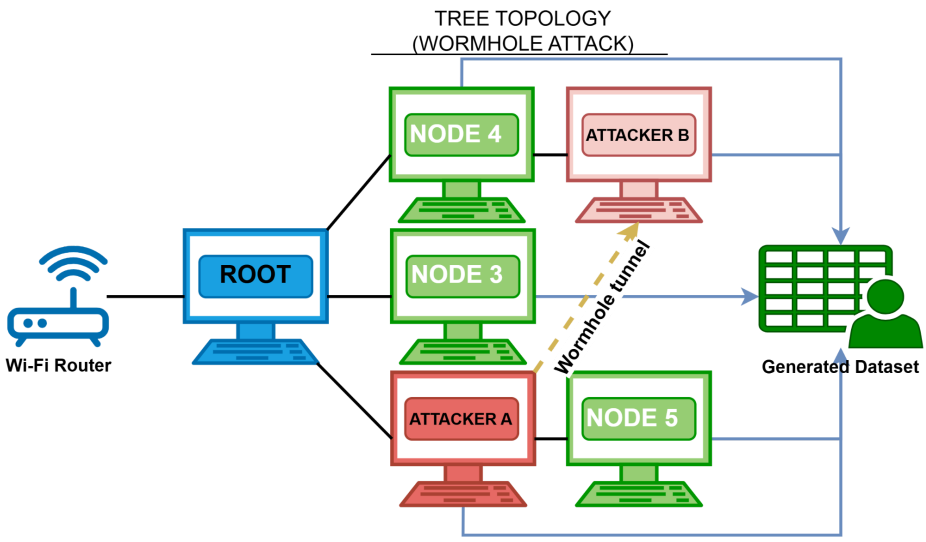

Figure 4.18 Wormhole Attack on Tree Topology

For wormhole experiments in *Figure 4.18*, Attacker A is positioned at Layer 1 near the root, while Attacker B is positioned at Layer 3 near the victim leaf nodes. The physical separation ensures that baseline communication between these regions requires traversing multiple layers through legitimate intermediate nodes. During the baseline phase, probe packets from victims follow the natural multi-hop path up the tree. When the distortion window is activated, Attacker B captures probe metadata from victim-side traffic and relays it through the out-of-band tunnel to Attacker A, which reconstructs and forwards shortcut probe responses toward the root. This creates an apparent reduction in path length, potentially observable as decreased latency and RSSI-hop inconsistencies. Tunnel endpoints log relay counters, while all nodes record standard mesh telemetry for comparison between baseline and distortion windows.

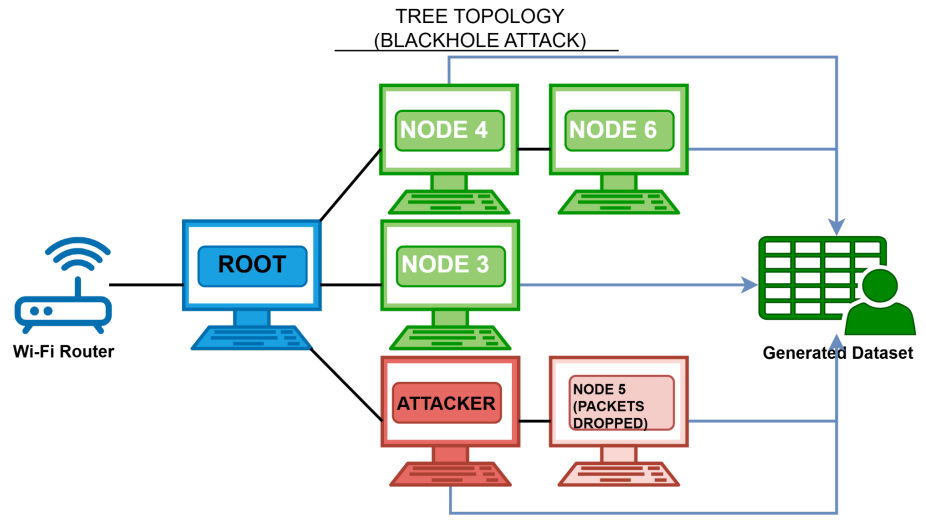

Figure 4.19 Blackhole Attack on Tree Topology

For blackhole experiments in *Figure 4.19*, the attacker is positioned at Layer 2 as an intermediate node that serves as the parent for multiple downstream victim leaf nodes. The root occupies Layer 0, with legitimate intermediate nodes at Layer 1. During baseline operation, victims at Layer 3 forward traffic upward through the attacker, which relays packets directly to the root node. When the blackhole manipulation activates, the attacker continues to receive transit packets from its victims but intentionally drops them instead of forwarding them to the root. This creates packet loss and triggers retransmissions from the victim nodes, which may also attempt to switch parents as they detect path failures. The dataset captures the attacker's forwarding ratio degradation, increases in victim-side retransmissions, and any parent-switching events that occur during the manipulation window. All nodes log layer information, parent MAC addresses, RSSI values, and packet counters throughout all phases.

4.2.2.3. Linear Chain Topology

A linear chain arrangement of ESP32 nodes is used to observe sequential packet propagation and localized forwarding dependencies within the ESP-WIFI-MESH network. Nodes are positioned in a straight line so that each device primarily communicates with its immediate neighbor, forming a chain-like multi-hop routing path from the farthest node toward the root. This configuration constrains routing paths and enables controlled observation of how packets propagate hop-by-hop through intermediate nodes.

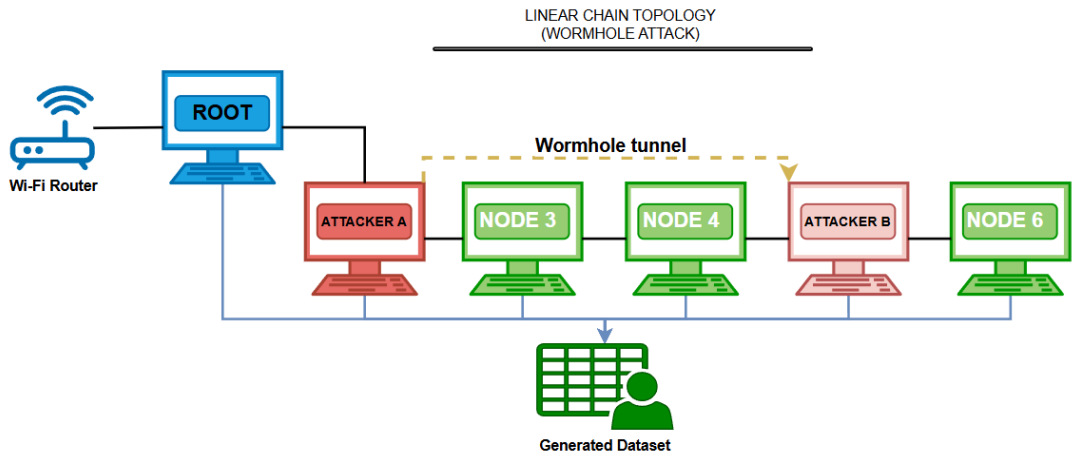

Figure 4.20 Wormhole Attack on Linear Chain Topology

For wormhole experiments in *Figure 4.20*, Attacker A is positioned near the root end of the chain, while Attacker B is placed near the far end, close to downstream victim nodes. The physical separation ensures that baseline communication between these regions must traverse all intermediate nodes sequentially, resulting in latency proportional to the number of hops. An out-of-band tunnel connects the two attackers independently of the mesh network. During the distortion window, Attacker B captures forwarded probe packets from downstream nodes and relays them through the tunnel to Attacker A, which reinjects shortcut probe responses near the root node. This creates an apparent path compression effect, as responses appear to arrive faster than the normal multi-hop path along the chain would allow. The dataset captures latency anomalies, hop-count stability, and tunnel activity metrics alongside standard mesh telemetry.

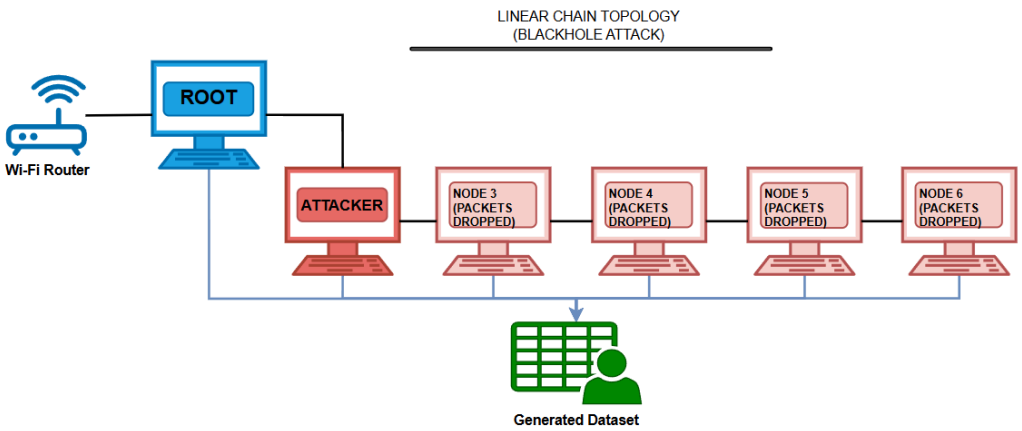

Figure 4.21 Blackhole Attack on Linear Chain Topology

For blackhole experiments in *Figure 4.21*, the attacker is positioned as an intermediate forwarding node within the chain such that all traffic between the root at one end and the downstream nodes at the other must pass through it. Nodes are spaced several meters apart (typically 3–5 m in the laboratory setup) so that the strongest RSSI links occur between adjacent devices, encouraging sequential parent selection along the linear arrangement. During baseline operation, packets from the root side to downstream nodes traverse each intermediate node in sequence, with the attacker acting as a normal relay. When the blackhole manipulation activates, the attacker continues to receive packets from upstream nodes but intentionally drops them instead of forwarding them further along the chain, effectively partitioning the network. Nodes beyond the attacker experience complete packet loss, while nodes on the root side may continue to operate normally. The dataset captures the cumulative impact of forwarding suppression, including increased retransmissions from nodes immediately upstream of the attacker and complete communication failure for downstream nodes.

4.2.2.4. Partial Mesh Topology

The **partial mesh topology** represents the primary experimental environment, in which nodes are connected to a subset of other nodes rather than all of them. Nodes select parents based on RSSI, link quality, and hop count within the available connections. This semi-structured topology still allows observation of network self-organization and adaptive parent switching, but connectivity is limited compared to a full mesh.

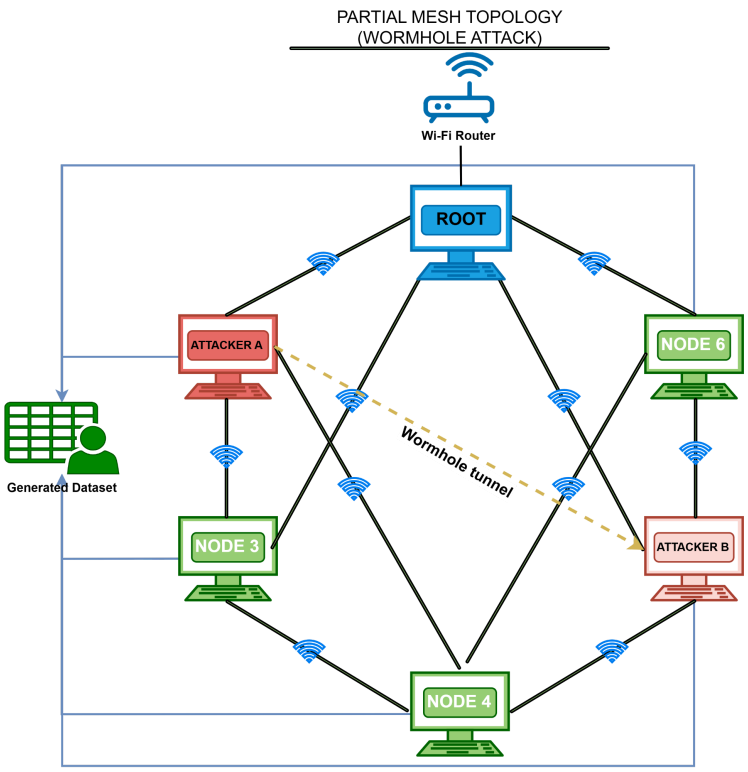

Figure 4.22 Wormhole Attack on Partial Mesh Topology

For wormhole experiments in *Figure 4.22*, Attacker A is positioned near the root region, and Attacker B is positioned near a cluster of victim leaf nodes. The connectivity matrix ensures multiple potential paths between regions, with baseline routes determined by the protocol's parent selection algorithm. An out-of-band tunnel connects the two attackers independently of the mesh. During the distortion window, Attacker B captures forwarded probe metadata from victim-side traffic and relays it through the tunnel to Attacker A, which reconstructs and forwards shortcut probe responses toward the root. This creates apparent path compression, which may influence routing decisions as nodes perceive faster response times along certain paths. The dataset captures changes in end-to-end latency, hop-count stability, parent switching frequency, and RSSI-hop correlation across baseline and distortion windows, providing insight into how the artificial shortcut affects the self-organizing behavior of the mesh network.

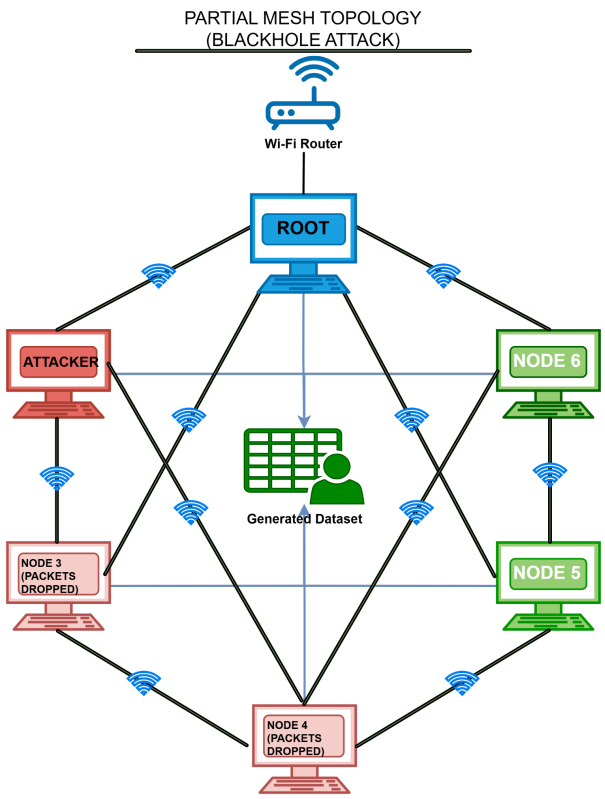

Figure 4.23 Blackhole Attack on Partial Mesh Topology

For blackhole experiments in *Figure 4.23*, the attacker is positioned as a high-betweenness relay node that becomes the preferred parent for multiple victim nodes via RSSI dominance and favorable routing metrics. During baseline operation, victims select the attacker as their parent, and traffic flows normally through it toward the root. When the blackhole manipulation activates, the attacker continues to receive transit packets but omits forwarding them, causing packet loss and triggering retransmissions. As victims experience persistent failures, they may attempt to switch to alternative parents, leading to topology instability and parent-switching events observable in the logs. The dataset captures not only the direct effects of forwarding suppression but also the adaptive responses of the mesh protocol as nodes attempt to re-route around the failed path.

4.2.3. Data Collection

This section describes the systematic approach employed to capture cross-layer telemetry from all ESP32 nodes participating in the experimental ESP-WIFI-MESH testbed. The data collection architecture is designed to operate independently of routing availability, ensuring that controlled manipulations—particularly forwarding suppression (blackhole)—do not compromise dataset integrity. All telemetry is recorded locally on each node and retrieved post-experiment using a single laptop workstation that serves exclusively as a consolidation interface rather than a live packet-capture device.

The data collection framework is organized into four functional components: (1) per-node metric definitions for victim and attack nodes, (2) data acquisition and logging architecture, (3) phase labeling and synchronization, and (4) local storage and post-experiment extraction. The following subsections

detail each component, with emphasis on the raw telemetry fields captured during experimentation. Derived features (e.g., forwarding ratio, RSSI-hop consistency) are computed offline during the feature extraction stage described in Section 4.2.4.

4.2.3.1. Victim Node Data Collection

Victim nodes are standard mesh participants that generate periodic application-layer probe traffic and record raw cross-layer telemetry. These nodes are typically positioned as leaf nodes (Layer 2 or higher) and serve as the primary sources of observable network behavior during both baseline and manipulation phases.

A. Physical Layer (PHY) Metrics

Table 4.4 Physical Layer (PHY) Metrics

Metric	Description	Collection Method	Sampling
rssi	Received Signal Strength Indicator	esp_wifi_sta_get_rssi()	1 Hz
timestamp	Microsecond-resolution timestamp	esp_timer_get_time()	Per sample

B. Medium Access Control (MAC) Layer Metrics

Table 4.5 Medium Access Control (MAC) Layer Metrics

Metric	Description	Collection Method	Sampling
retry_count	Cumulative MAC-layer retransmissions	ESP-IDF Wi-Fi statistics API (if available)	1 Hz
tx_count	Cumulative frames transmitted	ESP-IDF Wi-Fi statistics API	1 Hz

Note: When the specific ESP-IDF version does not expose MAC counters, fields are recorded as zero and supplemented by application-level transmission logs during preprocessing.

C. Network Layer Metrics

Table 4.5 Network Layer Metrics

Metric	Description	Collection Method	Sampling

layer	Current mesh layer (hop count)	esp_mesh_get_layer()	On change
parent_mac	Upstream parent MAC address	*esp_mesh_get_parent_mac()*	On change
parent_switch_event	Parent change notification	MESH_EVENT_PARENT_CONNECTED/DISCONNECTED	Event-driven
probes_sent	Application-layer probe counter	Incremented on each probe transmission	Per probe

D. Probe Generation

Victim nodes generate periodic UDP-like probe packets addressed to the root node. Each probe contains:

- Sequence number
- Transmission timestamp
- Node identifier

These probes enable end-to-end latency measurement and delivery ratio calculation during offline analysis.

4.2.3.2. Attack Node Data Collection

Attack nodes are responsible for executing controlled behavioral manipulations while simultaneously logging telemetry. These nodes record all metrics collected by victim nodes, plus additional counters specific to their manipulation roles.

A. Attack Node Types

Two distinct attack node configurations are implemented:

- **Forwarding Suppression (Blackhole) Node:** A single attacker that selectively drops transit packets. All manipulation occurs at the application layer; no lower-layer protocol violations are introduced.
- **Topology Distortion (Wormhole-Inspired) Nodes:** Two colluding attackers that relay information through an out-of-band tunnel to create a shortcut effect. No raw IEEE 802.11 frame capture, modification, or replay is performed. Wormhole behavior is emulated at the application/flow and telemetry level via an auxiliary tunnel, preserving full protocol compliance.

B. Forwarding Suppression (Blackhole) Node Metrics

In addition to the victim node metrics, the blackhole attacker records:

Table 4.6 Forwarding Suppression (Blackhole) Node Metrics

Metric	Description	Collection Method
packets_received	Transit packets received	Incremented on *esp_mesh_recv()*
packets_forwarded	Transit packets forwarded	Incremented on *esp_mesh_send()*
packets_dropped	Packets intentionally dropped	Computed as received - forwarded
suppression_active	Manipulation state flag	Boolean logged per sample

C. Topology Distortion (Wormhole-Inspired) Node Metrics

Both tunnel endpoints record the following metrics to document tunnel activity and its effects:

Table 4.7 Topology Distortion (Wormhole-Inspired) Node Metrics

Metric	Description	Collection Method
tunnel_tx_count	Messages sent through the tunnel	Incremented on each tunnel send
tunnel_rx_count	Messages received from the tunnel	Incremented on each tunnel received
tunnel_bytes_tx	Total bytes transmitted over the tunnel	Accumulated per tunnel send
tunnel_bytes_rx	Total bytes received from tunnel	Accumulated per tunnel received
tunnel_latency_ms	Round-trip tunnel latency (optional)	Timestamped echo messages
shortcut_inject_count	Measurement summaries reinjected toward the root	Incremented after each reinjection
distortion_active	Manipulation state flag	Boolean logged per sample

These metrics document the intensity and effect of the manipulation without requiring low-level frame capture. All wormhole-inspired effects are achieved via the application-layer tunnel and do not involve modifying mesh control frames.

4.2.3.3. Data Acquisition and Logging Architecture

The data acquisition system implements a decentralized logging framework embedded within each node's firmware. Logging operates as a background task that does not interfere with mesh responsibilities.

A. Sampling Loop

All nodes execute a continuous telemetry sampling loop with a 1-second interval. *Figure 4.24* illustrates the victim node implementation; attack nodes extend this with manipulation-specific counters.

Figure 4.24 Victim Node Telemetry Sampling Loop

```
Constants:
  SAMPLING_INTERVAL ← 1000 milliseconds

Variables:
  node_id : unique identifier
  current_phase : updated via root broadcasts

procedure VICTIM_TELEMETRY_LOOP():
  initialize_storage()

  while experiment_active == TRUE do
    // Capture timestamp
    timestamp ← get_high_resolution_time()

    // Physical layer
    rssi ← esp_wifi_sta_get_rssi()

    // MAC layer (cumulative counters)
    retry_count ← read_retry_counter()
    tx_count ← read_tx_counter()

    // Network layer
    layer ← esp_mesh_get_layer()
    parent_mac ← esp_mesh_get_parent_mac()

    // Application layer
    probes_sent ← read_probe_counter()

    // Format CSV record
    log_entry ← CSV_FORMAT(
      timestamp,
      node_id,
      "victim",
      layer,
      parent_mac,
      rssi,
      retry_count,
      tx_count,
```

```
        probes_sent,
        current_phase
    )

    // Write to local flash storage
    append_to_log(log_entry)

    // Wait for next sampling interval
    sleep(SAMPLING_INTERVAL)
  end while
end procedure
```

B. Forwarding Counter Implementation

Accurate tracking of receive and forward events is essential for subsequent computation of the forwarding ratio. Attack nodes maintain cumulative counters updated on each mesh API call.

Figure 4.25 Forwarding Counter Update (Attack Node)

```
// Global counters
atomic uint32_t recv_counter = 0
atomic uint32_t forward_counter = 0
atomic uint32_t drop_counter = 0

// Called by mesh receive callback
on_event esp_mesh_recv(packet):
  atomic_increment(recv_counter)
  process_packet_normally(packet)
end

// Called when forwarding a packet
procedure forward_packet(packet, destination):
  if should_forward(packet):
    result = esp_mesh_send(destination, packet, flags)
    if result == ESP_OK:
      atomic_increment(forward_counter)
    end if
  else:
    atomic_increment(drop_counter)
  end if
end procedure

// Forwarding suppression decision
procedure should_forward(packet):
  if packet.is_transit and suppression_active:
    return FALSE
  end if
  return TRUE
end procedure
```

The algorithm in *Figure 4.25* uses atomic counters to track packets across concurrent tasks safely. When a packet arrives, *recv_counter* is incremented. For outgoing packets, *should_forward()* checks whether suppression is active. If active, *drop_counter* is incremented, and the packet is discarded; otherwise, forward_counter is incremented after a successful *esp_mesh_send()*. This provides the raw counts needed to compute the forwarding ratio offline.

C. Tunnel Metric Sampling (Wormhole-Inspired Nodes)

For topology distortion experiments, tunnel endpoints record activity metrics using the sampling loop in *Figure 4.26*.

Figure 4.26 Tunnel Metric Sampling

```
procedure TUNNEL_METRIC_SAMPLING():
   while experiment_active == TRUE do
      timestamp ← get_high_resolution_time()

      // Snapshot current tunnel counters
      tx ← read_tunnel_tx_counter()
      rx ← read_tunnel_rx_counter()
      bytes_tx ← read_tunnel_bytes_tx()
      bytes_rx ← read_tunnel_bytes_rx()
      latency ← measure_tunnel_latency()  // optional

      log_entry ← CSV_FORMAT(
         timestamp,
         node_id,
         "attacker",
         "distortion",
         tx,
         rx,
         bytes_tx,
         bytes_rx,
         latency,
         distortion_active,
         current_phase
      )

      append_to_log(log_entry)
      sleep(SAMPLING_INTERVAL)
   end while
end procedure
```

This loop snapshots the current tunnel counters at 1-second intervals. The counters track the number of messages and bytes relayed, providing a direct measure of manipulation intensity. The distortion_active flag indicates whether the wormhole effect is currently enabled, allowing correlation between tunnel activity and observed network behavior.

4.2.3.4. Phase Labeling and Synchronization

Precise ground-truth labeling is achieved through coordinated phase transitions broadcast by the root node. Each node embeds the current phase label in every telemetry record.

A. Phase Control Algorithm

Figure 4.27 Experimental Phase Control (Root Node)

```
Input: baseline_duration, manipulation_duration, post_duration

procedure CONTROL_EXPERIMENT_PHASES():
   // Baseline phase: normal operation, no manipulation
   phase ← BASELINE
   broadcast_phase(phase)
   log_event("PHASE_START", phase, current_time())
   sleep(baseline_duration)
```

```
    // Manipulation phase: activate attack behavior
    phase ← MANIPULATION
    activate_manipulation()    // Enable forwarding suppression or tunnel
    broadcast_phase(phase)
    sleep(manipulation_duration)

    // Post-condition phase: disable manipulation, observe recovery
    phase ← POST_CONDITION
    deactivate_manipulation()  // Disable all attack behaviors
    broadcast_phase(phase)
    sleep(post_duration)

    // Termination: end of experiment
    phase ← TERMINATE
    broadcast_phase(phase)
end procedure
```

In *Figure 4.27,* the root node controls the experiment timeline by broadcasting phase change messages to all mesh participants. Each phase has a fixed duration, and the root logs the exact start and end times for later validation. The *broadcast_phase()* function uses ESP‑WIFI‑MESH's reliable broadcast mechanism to ensure all nodes receive the transition.

B. Phase Broadcast Mechanism

Phase transitions are disseminated using ESP‑WIFI‑MESH broadcast messaging. All nodes update their local current_phase variable upon receipt, ensuring temporal alignment across the network. Nodes also log each phase transition event locally, providing a secondary check during dataset consolidation.

4.2.3.5. Experimental Label Encoding and Assignment

Ground-truth labels are assigned to every telemetry record based on the phase broadcasts from the root node, as described in Section 4.2.3.4. Each node maintains a local current_phase variable updated upon receipt of phase transition messages; this value is appended to every logged telemetry sample. The mapping between the experimental phase and numerical label is defined in Table 4.8.

Table 4.8 Experimental Phase Label Encoding

Label	Experimental State
0	Baseline (Normal Operation)
1	Controlled Forwarding Suppression (Blackhole)
2	Controlled Topology Distortion (Wormhole-Inspired)

This labeling approach ensures ground-truth integrity for the following reasons:

- **Independence from feature values:** Labels are derived from experiment control logs and broadcast messages, not from any observed metric threshold.

- **No implicit detection logic:** The labeling process does not embed assumptions about what constitutes an attack; it only records which manipulation was active at a given time.
- **Consistency across all nodes and windows:** Every node records the same phase label during each interval, enabling reliable temporal alignment during dataset consolidation.

4.2.3.6. Local Storage and Post-Experiment Extraction

A. Local Storage Architecture

Each node maintains its own telemetry log using the on-chip flash file system (SPIFFS/LittleFS). Logs are written in CSV format with the following characteristics:

Table 4.9 Local Storage Architecture

Parameter	Value
File System	SPIFFS or LittleFS
Log Format	CSV
File Naming	nodeID_runID_YYYYMMDD_HHMMSS.csv
Buffer Size	256 bytes (flushed every 10 records)
Maximum Log Size	500 KB per run
Log Rotation	Automatic if size threshold exceeded (new file created)

B. Post-Experiment Log Extraction

After all experimental phases are complete, a single laptop workstation retrieves logs sequentially via USB serial connection. Figure 4.28 details this process.

Figure 4.28 Sequential Log Retrieval

```
Input: node_list, run_id

procedure EXTRACT_ALL_LOGS(node_list, run_id):
    for each node in node_list do
        // Establish physical USB connection
        connect_serial(node.port, baudrate=115200)

        // Send command to initiate log transfer
        serial_send("EXPORT_LOGS")

        // Wait for node to signal readiness
        response ← serial_read_line(timeout=5000)
        if response == "READY_TO_SEND":
            // Receive log data stream
```

```
        log_data ← receive_log_via_serial()

        // Save to laptop with descriptive filename
        filename ← format("%s_%s.csv", run_id, node.id)
        save_file(filename, log_data)

        // Verify file integrity using checksum
        if verify_checksum(filename):
            // Confirmation successful; node may delete logs
            serial_send("DELETE_LOGS")
            log_success(node.id, filename)
        else:
            // Retry or flag for manual inspection
            log_failure(node.id, "checksum mismatch")
      end if

      // Close connection and proceed to next node
      disconnect_serial()
   end for
end procedure
```

The laptop connects to each node in turn via USB serial. The EXPORT_LOGS command triggers the node to stream its stored log file. After receiving the data, the laptop computes a checksum and requests deletion only if verification passes. This sequential approach ensures all logs are safely transferred before they are cleared from the node memo.

C. Data Integrity During Manipulation

The local storage architecture ensures dataset completeness even during active attacks:

- Logging continues regardless of routing success – The logging task runs independently of mesh operations.
- All transmission attempts and failures are recorded – Counters capture every packet, regardless of outcome.
- Logs are retained until verified extraction – No data is deleted until the laptop confirms successful transfer.
- Telemetry logs are not streamed over the mesh during experimentation – Only experiment traffic (e.g., probes, control messages) is exchanged over the mesh; logging data stays local.

This design guarantees that the final dataset contains a complete record of network behavior across all phases, with ground‑truth labels embedded directly in the data stream, and is immune to data loss caused by the very attacks being studied.

The raw telemetry collected through this framework serves as input to the feature extraction stage described in Section 4.2.4, where derived behavioral indicators (e.g., forwarding ratio, RSSI‑hop consistency, parent switching frequency) are computed for exploratory analysis and clustering.

4.2.3.7. Dataset Completeness Verification

To ensure that the final dataset accurately reflects the intended experimental conditions and to identify potential data loss due to high traffic or transient logging failures, a completeness verification procedure is performed during preprocessing.

- **Expected Sample Count:** During each experimental phase, nodes are configured to log telemetry at 1 Hz. For a 5-minute baseline phase (300 seconds), the expected number of samples per node is 300. For manipulation phases (3 minutes), the expected count is 180.

- **Completeness Check**: After log extraction, the actual number of logged entries per node per second is counted. Windows (5-second aggregation intervals) where the actual sample count falls below 90% of the expected (i.e., fewer than 5 samples per window) are flagged as incomplete. Such windows may be excluded from downstream analysis, and the proportion of incomplete windows per run is reported as a dataset quality metric.

- **Handling of Gaps:** If a node misses a sample, linear interpolation is used for continuous metrics (e.g., RSSI), provided the gap is ≤ 2 seconds. Longer gaps cause the affected window to be discarded. All decisions regarding interpolation and exclusion are logged to ensure transparency and reproducibility.

4.2.4. Feature Extraction

This set of tunnel activity includes the computational procedures for transforming raw telemetry logs into structured behavioral features suitable for exploratory analysis and clustering. All derivations are performed offline during dataset preprocessing, after log extraction and timestamp alignment. The resulting feature vectors capture cross-layer behavioral patterns across PHY, MAC, and Network layers, with optional auxiliary tunnel activity features recorded only during topology-distortion runs.

4.2.4.1 Preprocessing

Raw telemetry logs collected at 1 Hz are first processed to ensure temporal consistency and to convert cumulative counters into interpretable metrics.

Timestamp Alignment. Although nodes are synchronized prior to each experiment using phase-broadcast markers, residual clock drift may occur during extended runs. Timestamps are aligned offline using the root node's phase transition log as a reference. Per-node offsets are computed and applied to all records, ensuring that telemetry from different nodes can be correlated within the same observation window. Each node records a high-resolution local timestamp using *esp_timer_get_time()*; cross-node alignment is performed during this offline step.

Counter Delta Conversion. Cumulative counters (e.g., retry_count, tx_count, recv_count, forward_count) are recorded as monotonically increasing values. For analysis, these are converted into per-window deltas:

$$\Delta counter = counter(t_{end}) - counter(t_{start}) \quad \textbf{Equation 4.1}$$

This conversion captures the activity intensity within each window independent of prior history.

Handling Missing MAC Counters. When the ESP-IDF version in use does not expose MAC-layer counters, the corresponding fields are recorded as zero during logging. In such cases, application-level transmission logs and response-timing proxies are used as substitutes during feature computation, with appropriate flags indicating the substituted values.

Handling Missing Values

Raw telemetry logs may occasionally contain missing samples due to transient logging failures, buffer overflows, or temporary node unavailability. Several strategies exist for dealing with such missing data:

- **Linear interpolation** – Estimate missing values by averaging the nearest valid samples before and after the gap. Suitable for continuous metrics that change smoothly (e.g., RSSI).
- **Forward/backward fill** – Propagate the last observed value forward (or the next observed value backward) to fill short gaps. Appropriate for cumulative counters that should not decrease.
- **Deletion of incomplete windows** – Discard any aggregation window that contains missing samples beyond a tolerable threshold. This preserves data integrity but may reduce dataset size.
- **Flagging and separate handling** – Retain incomplete windows but mark them with a special flag, allowing downstream analysis to decide whether to include or exclude them.

The choice among these strategies depends on the nature of the metric, the extent of missing data, and the tolerance for estimation error. Also, adopt a hybrid approach that balances data completeness with reliability, applying different rules depending on the type of metric and the duration of the gap.

1. **For continuous metrics (e.g., RSSI):**
    - If a gap of 1–2 consecutive missing samples occurs within a window, linear interpolation is applied using the nearest valid samples.
    - If the gap exceeds 2 seconds (i.e., more than two consecutive missing samples), the entire 5-second window is discarded from further analysis. This threshold ensures that estimated values do not introduce excessive uncertainty.
2. **For cumulative counters (e.g., forward_count, retry_count):**
    - Counters are monotonic; therefore, if a sample is missing for ≤ 2 seconds, the counter value is assumed unchanged from the previous logged sample (i.e., forward fill).
    - Gaps longer than 2 seconds cause the window to be discarded because we cannot reliably determine the counter increment over the interval.
3. **MAC counters not exposed by the ESP-IDF version:**
    - When certain MAC-layer counters (e.g., retry_count) are unavailable, those fields are recorded as zero and flagged in the metadata.
    - For such nodes, features that depend on these counters (e.g., Retry Rate) are omitted from the Core Behavioral Set. An alternative proxy may be used where possible (e.g., application-layer retransmissions).
4. **Window completeness threshold:**
    - Regardless of the imputation method, any window that contains fewer than 4 valid 1-Hz samples (out of the expected 5) is discarded. This guarantees that summary statistics are based on a representative subset of the window's duration.

All interpolation and flagging decisions are logged per node and per window, ensuring full transparency. The fraction of discarded windows is reported as a dataset quality metric, allowing future users to assess the potential impact of missing data.

Window Aggregation. To reduce sensitivity to transient fluctuations and to capture behavioral trends, all telemetry is aggregated into non-overlapping 5-second windows:

Table 4.10 Window Aggregation

Parameter	Value
Window Duration	5 seconds
Window Type	Non-overlapping
Continuous Metrics	Mean, variance, min, max per window
Cumulative Counters	Delta per window
Event Counts	Total events per window (e.g., parent switches)

Windowed aggregation balances granularity with statistical robustness, producing approximately 12 windows per minute of experimentation.

4.2.4.2 Feature Computation

The following features are computed for each node per observation window. Features are organized by the behavioral dimension they capture, with formulas and interpretations provided. Where relevant, we distinguish between transit packets (those received for forwarding) and locally generated traffic.

A. Forwarding Behavior Features

Forwarding Ratio (Blackhole Indicator)

Measures the proportion of received transit packets that are forwarded by a node. This feature is the primary indicator of forwarding suppression. The counters recv_count and forward_count refer only to transit packets; locally generated packets are excluded.

$$\text{Forwarding Ratio} = \frac{\Delta forward\_count}{\Delta recv\_count + \epsilon}$$

Equation 4.2

Where:

- $\Delta forward\_count$ = forward_count at window end − forward_count at window start
- $\Delta recv\_count$ = recv_count at window end − recv_count at window start
- ϵ is a small constant (10^{-6}) to prevent division by zero

Interpretation:

- ≈ 1.0 → Normal forwarding behavior (all received transit packets forwarded)

- ≈ 0.0 → Complete forwarding suppression (blackhole active)
- 0.0 < ratio < 1.0 → Partial suppression or forwarding instability
- Undefined (no packets received) → Node not handling transit traffic

Ingress-Egress Delta

Absolute difference between transit packets received and forwarded, indicating packet absorption.

$$\text{Ingress-Egress Delta} = \Delta recv\_count - \Delta forward\_count \quad \textbf{Equation 4.3}$$

Interpretation:

- ≈ 0 → Balanced forwarding (normal operation)
- > 0 → Packets absorbed (potential blackhole)
- < 0 (rare) → May indicate local packet generation or forwarding of previously queued packets

B. Link Reliability Features

Retry Rate

Proportion of transmission attempts that required MAC-layer retransmission, indicating link-level stress.

$$\text{Retry Rate} = \frac{\Delta retry\_count}{\Delta tx\_count + \epsilon} \quad \textbf{Equation 4.4}$$

Interpretation:

- Low (< 0.05) → Stable link conditions
- Moderate (0.05–0.15) → Some link contention or minor instability
- High (> 0.15) → Significant link stress, often induced by forwarding failures or topology changes

Packet Delivery Ratio (PDR)

Proportion of probes successfully received at the root node, computed per victim node per window. The root counts received probes by tracking unique sequence numbers embedded in each probe.

$$\text{PDR} = \frac{\#\text{ unique probe sequence numbers received at root}}{\#\text{ probes sent by victim}} \quad \textbf{Equation 4.5}$$

Interpretation:

- 0.97 → Normal operation (aligned with Khan et al. [25] baseline)
- Decreased values → End-to-end delivery degradation, potentially due to blackhole or routing disruption

C. Topology Stability Features

Parent Switch Rate

Frequency of parent changes within the observation window, indicating topology instability.

$$\text{Parent Switch Rate} = \frac{\text{Number of parent change events}}{\text{Window Duration (seconds)}}$$ **Equation 4.6**

Interpretation:

- Very low (≈ 0) → Stable topology
- Moderate (> 0 but small) → Normal adaptive adjustments
- High values → Persistent instability or distortion pressure

Layer Change Count

Number of times the node's mesh layer (hop count) changed during the window.

$$\text{Layer Change Count} = \text{Count of layer change events}$$ **Equation 4.7**

Interpretation:

- 0 → Stable layer assignment
- > 0 → Node moving up/down in hierarchy, indicating topology adaptation or instability

Hop Stability Duration

Longest continuous interval within the window during which the node maintained the same layer and parent. Computed by scanning event logs and measuring the maximum duration where (layer, parent_mac) remained constant.

$$\text{Hop Stability Duration} = \max(\text{stable intervals})$$ **Equation 4.8**

Interpretation:

- Long durations → Stable topology
- Short durations → Frequent reconfiguration, potential distortion effects

D. Physical Layer Features

RSSI Mean

Average received signal strength within the window.

$$\text{RSSI Mean} = \frac{1}{n}\sum_{i=1}^{n} RSSI_i$$ **Equation 4.9**

Interpretation:

- Values reflect link quality and physical proximity to the parent
- Under normal operation, RSSI decreases predictably with hop count

RSSI Variance

Statistical variance of RSSI samples within the window, indicating signal stability.

$$\text{RSSI Variance} = \frac{1}{n}\sum_{i=1}^{n}(RSSI_i - \overline{RSSI})^2$$ **Equation 4.10**

Interpretation:

- Low variance → Stable link conditions
- High variance → Environmental interference, mobility, or topology instability

RSSI Stability Duration

Longest continuous interval within the window where RSSI remains within ±3 dBm of the window mean.

$$\text{RSSI Stability Duration} = \max(\text{intervals with } |RSSI_i - \overline{RSSI}| \leq 3)$$ **Equation 4.11**

Interpretation:

- Long duration → Stable signal conditions
- Short duration → Fluctuating signal, potentially due to interference or parent switching

E. Cross-Layer Inconsistency Features

RSSI–Hop Inconsistency

Deviation between observed RSSI and the value expected based on the node's reported layer. This feature captures physical-logical topology mismatches characteristic of wormhole distortion.

The expected RSSI for a given layer is not a fixed global table; instead, it is derived empirically from the baseline windows of the same experimental run. For each stable baseline segment, the median RSSI per layer is computed. During distortion windows, the observed RSSI is compared against the baseline median for the reported layer:

$$\text{RSSI-Hop Diff} = \left|\text{Observed RSSI} - \text{Expected\_RSSI(layer)}\right|$$ **Equation 4.12**

Where

$$\text{Expected\_RSSI(layer)} = \text{Median}\{RSSI \mid \text{layer, baseline, stable}\}$$ **Equation 4.13**

This approach makes the metric environment-specific and robust to testbed variations.

Interpretation:

- Small diff (< 5 dB) → RSSI consistent with reported layer
- Large diff (> 10 dB) → Potential physical-logical mismatch; node may be receiving packets from distant sources with unusually strong signal (wormhole effect)

Latency–Hop Inconsistency

Ratio of end-to-end latency to hop count, indicating whether path length matches expected delay.

$$\text{Latency-Hop Ratio} = \frac{\text{Mean RTT (ms)}}{\text{Layer}}$$ **Equation 4.14**

where Mean RTT is computed from probe-response exchanges between victim nodes and the root (using the timestamps embedded in probe/response messages).

Interpretation:

- Consistent ratio (10–30 ms per hop) → Normal multi-hop forwarding
- Abnormally low ratio → Path appears shorter than reported layer (potential shortcut effect)
- Abnormally high ratio → Congestion or indirect routing

Forwarding Consistency Score

Deviation of the forwarding ratio from the ideal value of 1.0 provides a unified measure of forwarding compliance.

$$\text{Consistency Score} = |1.0 - \text{Forwarding Ratio}| \quad \textbf{Equation 4.15}$$

Interpretation:

- ≈ 0 → Perfect forwarding compliance (Forwarding Ratio ≈ 1.0)
- > 0 → Increasing deviation from expected behavior
- Closer to 1 → Severe forwarding suppression

F. Auxiliary Tunnel Activity Features (Experiment-Level)

These metrics are recorded only on nodes participating in topology distortion experiments. They are not part of the standard PHY/MAC/Network layer set, but are included to document manipulation intensity.

Tunnel Intensity

Combined rate of tunnel activity, indicating the intensity of the wormhole-inspired manipulation.

$$\text{Tunnel Intensity} = \frac{\Delta tunnel\_tx\_count + \Delta tunnel\_rx\_count}{\text{Window Duration}} \quad \textbf{Equation 4.16}$$

where:

- Δtunnel_tx_count = tunnel transmissions within the window
- Δtunnel_rx_count = tunnel receptions within the window
- Window Duration = 5 seconds (in this study)

Interpretation:

- 0 → No tunnel activity (baseline or tunnel inactive)
- > 0 → Tunnel active; higher values indicate more frequent relaying

Tunnel Bytes Transferred

Total bytes transmitted through the tunnel per window, indicating data volume.

$$\text{Tunnel Bytes} = \Delta tunnel\_bytes\_tx + \Delta tunnel\_bytes\_rx \quad \textbf{Equation 4.17}$$

Interpretation:

- Correlates with manipulation intensity; higher values may indicate a more significant shortcut

Tunnel Latency

Round-trip latency of the tunnel itself, measured using periodic echo messages.

Interpretation:

- Consistent low latency → Tunnel operating as designed

- Variable latency → May affect shortcut effectiveness

4.2.4.3 Feature Summary

The complete feature set per node per window comprises the following 16 attributes, categorized by the layer they primarily represent:

Table 4.11 Feature Summary

Feature	Layer	Behavioral Dimension
Forwarding Ratio	Network	Forwarding compliance
Ingress-Egress Delta	Network	Packet absorption
Retry Rate	MAC	Link stress
Packet Delivery Ratio	Network	End-to-end reliability
Parent Switch Rate	Network	Topology stability
Layer Change Count	Network	Topology stability
Hop Stability Duration	Network	Topology stability
RSSI Mean	PHY	Signal strength
RSSI Variance	PHY	Signal stability
RSSI Stability Duration	PHY	Signal stability
RSSI-Hop Inconsistency	Cross-layer	Physical-logical mismatch
Latency-Hop Inconsistency	Cross-layer	Path length mismatch
Forwarding Consistency Score	Cross-layer	Forwarding deviation
Tunnel Intensity	Auxiliary	Manipulation intensity

Tunnel Bytes	Auxiliary	Manipulation volume
Tunnel Latency	Auxiliary	Tunnel performance

These features form the input space for clustering analysis, capturing the cross-layer behavioral signatures of normal operation, forwarding suppression, and topology distortion.

4.2.4.4 Dataset Record Structure (Windowed Per-Node Schema)

Dataset Scope and Observation Granularity

To clarify the structure of the dataset used in this study, telemetry observations are collected from individual ESP32 nodes and subsequently merged into a unified dataset representing the overall experimental mesh network environment. Rather than representing the network as a single aggregated measurement, the dataset is constructed at the node-level observation granularity, where each record corresponds to the behavior of a specific node during a defined observation window.

During each experimental phase, all nodes participating in the ESP-WiFi-Mesh testbed generate telemetry logs containing cross-layer indicators related to routing behavior, link conditions, packet forwarding activity, and topology participation. These logs are retrieved after each experiment and synchronized using timestamp alignment to compensate for asynchronous device clocks. Once synchronized, logs from all nodes are merged into a single dataset while preserving the identity of each node and the corresponding observation window.

Each dataset entry, therefore, represents a node–window observation, consisting of:

- Node identifier
- Observation window or timestamp interval
- Experimental phase label (baseline, wormhole attack, blackhole attack, or combined attack)
- Extracted behavioral features derived from telemetry metrics

After preprocessing and feature computation, each 5-second observation window produces one record per node. The complete dataset is structured as a flat table, where rows correspond to node–window observations and columns contain the features defined in Section 4.2.4.2, together with metadata and experimental labels. Table 4.12 summarizes the dataset schema.

This design preserves node-level behavioral characteristics, which are essential for identifying routing anomalies. For example, a blackhole attack typically manifests as a decrease in forwarding activity at the attacker node, while neighboring nodes may exhibit increased retransmissions or routing adjustments. If the dataset were aggregated at the network level, these localized behavioral changes would be obscured.

The final dataset combines observations from all experimental runs and all topology configurations (star, tree, linear chain, and partial mesh) into a single unified table. Additional metadata fields, such as *topology* and *run_id,* are included to allow researchers to filter or stratify the dataset during analysis. For instance, analyses may be restricted to a specific topology to examine localized attack behavior, or the entire dataset may be used to evaluate clustering performance across multiple network configurations.

Table 4.12 Windowed Per-Node Dataset Schema

Column	Description
window_start	Start timestamp of the 5-second aggregation window
node_id	Unique node identifier
node_role	Node role (root, attacker, victim, other)
RSSI_mean	Average RSSI within window (dBm)
RSSI_var	RSSI variance
RSSI_stability	Longest stable RSSI interval (seconds)
RetryRate	MAC retransmission ratio
PDR	Packet delivery ratio (victim nodes only)
ForwardingRatio	Transit packet forwarding ratio
IngressEgressDelta	Packet absorption count
ConsistencyScore	Forwarding deviation metric
ParentSwitchRate	Parent changes per second
LayerChangeCount	Number of layer transitions

HopStabilityDuration	Longest stable (layer, parent) interval
RSSI_Hop_Diff	Physical-logical mismatch (dB)
LatencyHopRatio	Latency per hop (ms)
TunnelIntensity*	Tunnel messages per second
TunnelBytes*	Total tunnel bytes transferred
TunnelLatency*	Tunnel round-trip time (ms)
Label	Ground-truth phase (0,1,2)

*Tunnel features are present only for attacker nodes during topology-distortion runs; for all other nodes and phases, these fields are null or zero.

Clustering analysis is performed on these windowed records; each window is treated as an independent behavioral observation. No time-series or sequential models are applied, ensuring that clustering results reflect feature-space separation rather than temporal dependencies.

4.2.4.5 Feature Engineering Design Principles

The feature set defined in this section follows four methodological principles that guide selection, computation, and documentation:

- **Observability:** All features are derived from metrics accessible through standard ESP-IDF APIs (*esp_wifi_sta_get_rssi()*, mesh events, cumulative counters). No protocol modifications or custom instrumentation are required.
- **Low Overhead:** Feature computation is performed offline during dataset preprocessing; on-node logging captures only raw counters and timestamps. This ensures that the logging process does not interfere with mesh operations or manipulation effects.
- **Relationship Preservation:** Features are designed to preserve cross-layer relationships (e.g., receive–forward consistency, RSSI-hop correlation) that theoretical analysis identifies as attack-sensitive. They encode structural expectations rather than attack-specific thresholds.
- **Reproducibility:** All derived metrics are explicitly defined with fixed window durations (5 seconds), delta conversion rules, and expected value baselines computed empirically from stable windows. This ensures that the feature extraction process can be replicated exactly by other researchers.

4.2.4.6 Feature Selection and Dimensionality Analysis

While the feature set defined in **Section 4.2.4.3** captures a comprehensive range of cross-layer behaviors, not all features may contribute equally to distinguishing between normal and malicious network states. To

address the scenario where a single feature might dominate or where features are redundant, a feature selection and dimensionality analysis phase is integrated into the preprocessing workflow. The primary goals are to identify and potentially remove features that are highly correlated or have low variance, ensuring the subsequent clustering analysis is robust and not biased by irrelevant data.

First, a low-variance filter will be applied. Features that exhibit near-constant values across the vast majority of observation windows (e.g., >90% of samples have the same value) provide little discriminative power and can distort distance-based clustering. Such features will be flagged for removal. For example, if RSSI_Hop_Diff is zero for nearly all baseline windows, its presence might add noise rather than signal.

Second, a correlation analysis will be performed to identify pairs of features that are highly collinear (e.g., Pearson correlation coefficient $|r| > 0.85$). Highly correlated features can over-emphasize a single underlying behavioral characteristic. In such cases, one of the features from the pair may be considered for removal to reduce redundancy. For instance, Forwarding Ratio and Forwarding Consistency Score are mathematically related; if they are found to be perfectly correlated, one might be dropped.

Third, to complement the clustering algorithms, Principal Component Analysis (PCA), as introduced by Jolliffe [52], will be used as an exploratory tool. By projecting the high-dimensional feature vectors onto the principal components that capture the most variance, we can visualize the data's structure and assess its inherent clusterability. More importantly, analyzing the component loadings will reveal which original features contribute most heavily to the overall variance in the dataset. A feature that consistently shows high loadings across the first two or three principal components is empirically more relevant to the overall behavioral structure of the network.

This feature selection process is not intended to pre-define a rigid subset of features before clustering. Instead, it serves as a diagnostic step to ensure the dataset's integrity. Clustering will be performed on both the full feature set and the reduced, non-redundant feature set to compare the stability and interpretability of the results. This approach directly addresses the potential pitfall of relying on a single, dominant indicator and ensures that the discovered behavioral patterns are driven by a confluence of cross-layer evidence.

4.2.5. Data Clustering

This section describes the unsupervised clustering techniques applied to the extracted feature vectors to discover natural groupings corresponding to different network behavioral states. The goal is to determine whether normal operation, forwarding suppression, and topology distortion form separable clusters in feature space without relying on pre-defined attack signatures.

4.2.5.1. Feature Normalization

Before clustering, all continuous features are standardized to ensure that attributes with different numerical scales contribute proportionally to distance-based similarity calculations. Since both DBSCAN and K-Means rely on Euclidean distance, unscaled features (e.g., byte counts or event counts) could otherwise dominate ratio-based metrics.

Standardization (z-score normalization) is applied:

$$z = \frac{x - \mu}{\sigma}$$ **Equation 4.18**

where:

- μ is the mean of the feature computed across all nodes and all observation windows in the dataset
- σ is the corresponding standard deviation

Normalization is performed **after feature extraction and window aggregation**, and before clustering and parameter selection. The transformation is applied feature-wise across the entire dataset, not per node, ensuring that clustering reflects global behavioral structure rather than node-specific scaling.

For experiments evaluating purely cross-layer behavior, auxiliary tunnel features may be excluded before normalization to assess whether behavioral separation emerges without explicit manipulation indicators.

After transformation, each feature has zero mean and unit variance, allowing distance-based clustering algorithms to treat all dimensions comparably.

4.2.5.2. Clustering Algorithms

Two complementary clustering algorithms are employed to explore the structure of the feature space.

A. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is selected as the primary clustering method due to its ability to identify density-connected regions in feature space without requiring a predefined number of clusters. This property is particularly appropriate for exploratory behavioral analysis, where the true number of underlying behavioral states is not assumed a priori.

DBSCAN offers the following advantages:

- **No predefined number of clusters:** The algorithm determines the number of clusters based on the density structure.
- **Noise detection capability:** Points that do not belong to any dense region are labeled as noise, which may correspond to transitional or unstable behavioral windows.
- **Arbitrary cluster shapes:** Unlike centroid-based methods, DBSCAN does not assume spherical clusters.
- **Suitability for anomaly discovery:** Normal operation is expected to form dense regions, while attack behaviors may either form separate dense clusters or appear as sparse anomalies.

Table 4.13 Parameter Selection:

Parameter	Description	Selection Method
ε (eps)	Maximum distance between two samples to be considered neighbors	Determined using the k-distance graph (k = min_samples)

min_samples	Minimum number of points to form a dense region	Evaluated within the range {D+1, 2D}, where D is feature dimensionality

To determine ε, the distances to each point's k-th nearest neighbor (where k = min_samples) are computed and sorted in ascending order. The resulting k-distance graph typically exhibits a knee (elbow) point separating dense regions from sparse noise. The value of ε corresponding to this knee is selected as the optimal threshold.

Final parameter choices are validated using internal cluster evaluation metrics (silhouette score and Davies–Bouldin index) to ensure structural stability.

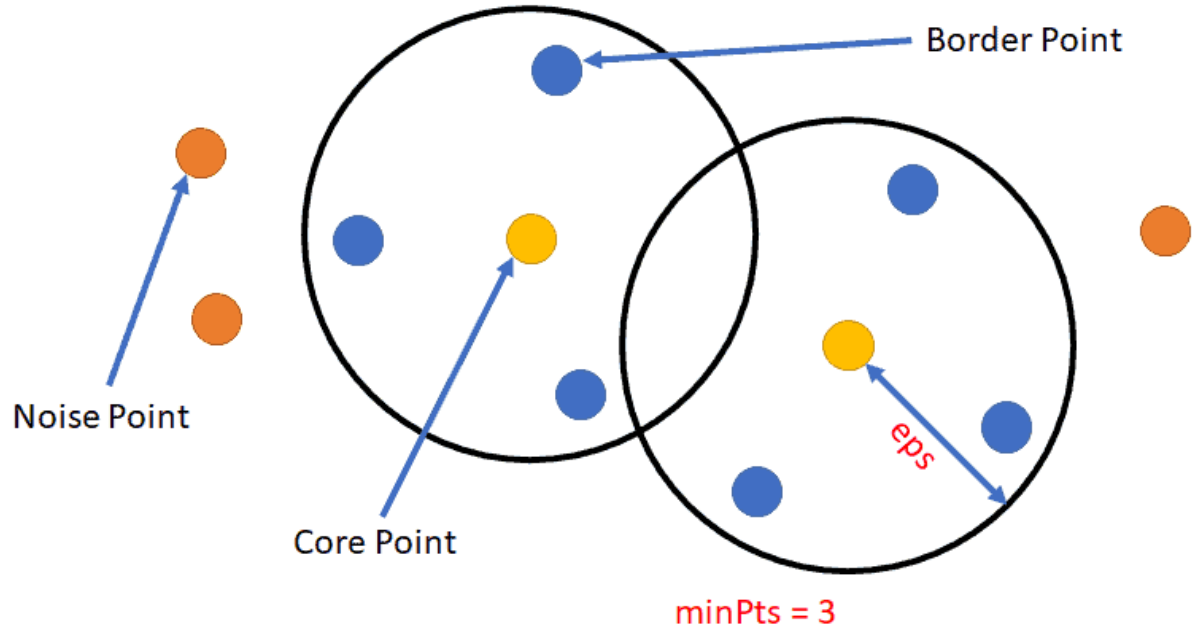

Figure 4.29 Core, Border, and Noise Points in DBSCAN

Illustrates how DBSCAN classifies points into **core points, border points, and noise points** using the ε-neighborhood radius and the minimum number of points (*minPts*). In the diagram, the **two black circles represent the ε-neighborhoods**, which define the radius within which neighboring points are counted. The parameter **minPts = 3**, shown in the figure, indicates the minimum number of neighboring points required for a point to be considered part of a dense region.

The elements shown in the figure correspond to the following DBSCAN point classifications:

- **Core Points** – The **yellow points located at the center of each ε circle** represent core points. These points have at least *minPts* neighboring points within their ε-neighborhoods. Because they lie in dense regions of the dataset, core points serve as the starting points for cluster formation.

- **Border Points** – The **blue points located inside the ε circles but not at the center** represent border points. These points fall within the ε-neighborhood of a core point but do not themselves

have enough neighbors to satisfy the *minPts* condition. Border points are therefore assigned to a cluster, but cannot expand the cluster further.

- **Noise Points** – The **orange points located outside the ε circles** represent noise points. These points do not fall within the ε-neighborhood of any core point and, therefore, are not assigned to any cluster. In DBSCAN, such points are treated as outliers.

The **overlapping ε circles** in the figure illustrate how dense regions can connect clusters through shared neighborhoods. By distinguishing between core, border, and noise points, DBSCAN is able to identify clusters of arbitrary shapes while simultaneously detecting outliers in sparse regions of the dataset (Shameer, n.d.).

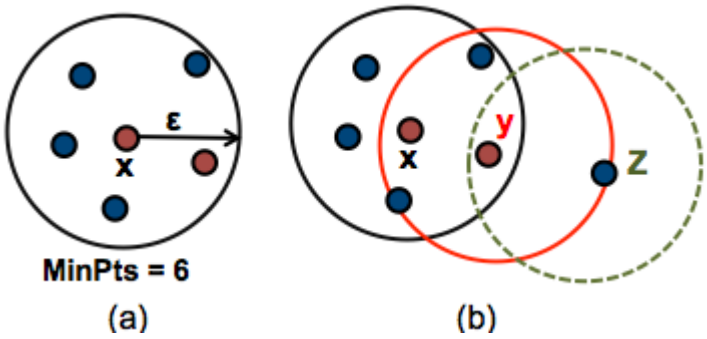

Figure 4.30 DBSCAN Cluster Formation with Noise

This figure illustrates how clusters are formed in the DBSCAN algorithm using the ε-neighborhood radius and the minimum number of points (*MinPts*). The diagram is divided into two parts, labeled **(a)** and **(b)**, which demonstrate how DBSCAN identifies dense regions and expands clusters through density reachability.

In **Figure 4.30**, the black circle represents the **ε-neighborhood** around the central point. The surrounding points inside the circle represent neighboring data points. Since the number of points within the ε radius satisfies the required **MinPts = 6**, the central point is classified as a **core point**. Core points indicate regions of high density and serve as starting points for cluster formation in the DBSCAN algorithm (Soni, n.d.).

In **Figure 4.30**, the diagram shows three overlapping ε-neighborhoods corresponding to points **X, Y, and Z**, illustrating how clusters expand through density relationships:

- **Point X** lies inside the dense region and satisfies the minimum density requirement, making it a **core point**. Its ε-neighborhood is represented by the black circle containing several nearby points.

- **Point Y** lies within the ε-neighborhood of point X and also has sufficient neighboring points inside its red ε circle. As a result, Y is also considered a **core point**, allowing the cluster to expand further through density reachability.

- **Point Z** lies within the ε-neighborhood of point Y but does not contain enough neighboring points to satisfy the MinPts requirement. Therefore, Z is classified as a **border point** and becomes part of the cluster without initiating further cluster expansion.

The **overlapping circles** in the figure illustrate how clusters grow by connecting core points whose ε-neighborhoods intersect. Points that are reachable through chains of core points become part of the same cluster, while points located outside these dense regions would be classified as **noise**.

This density-based expansion allows DBSCAN to detect clusters of arbitrary shapes while distinguishing sparse points that do not belong to any cluster (Soni, n.d.).

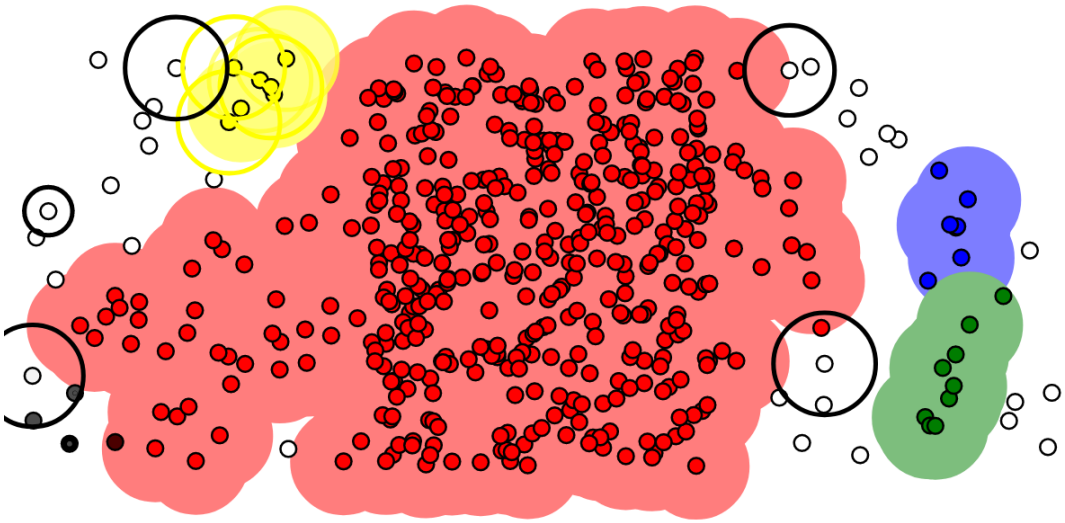

Figure 4.31 DBSCAN Density-Based Cluster Detection

The DBSCAN algorithm detects clusters by identifying **dense regions of data points** and separating them from sparse areas. In the visualization, clusters are represented by **colored shaded regions**, while points located outside dense areas remain unclustered and may be classified as noise. The black circles shown around certain points represent **ε-neighborhoods**, which define the radius used to measure local density.

The elements visible in the figure correspond to the following DBSCAN clustering concepts:

- **Dense Cluster Regions** – The **large red shaded region** in the center of the figure represents a dense cluster containing many closely spaced data points. Because the number of neighboring points within the ε-neighborhood satisfies the required *minPts* threshold, DBSCAN identifies this area as a cluster.

- **Smaller Cluster Groups** – On the right side of the figure, the **blue and green shaded regions** represent smaller clusters that also meet the density requirement. These clusters form when groups of points satisfy the ε and *minPts* conditions within localized regions.

- **Core Points and ε-Neighborhoods** – The **black circles surrounding some points** indicate the ε-neighborhood used by DBSCAN to evaluate point density. Points inside these circles represent neighboring data points that contribute to determining whether a point qualifies as a core point.

- **Border Points** – Points located along the **outer edges of the shaded cluster regions** represent border points. These points fall within the ε-neighborhood of core points but do not contain enough neighbors to satisfy the *minPts* requirement themselves.
- **Noise or Outlier Points** – The **isolated white or gray points scattered outside the colored regions** represent noise points. These points do not fall within the ε-neighborhood of any core point and, therefore, are not assigned to any cluster.

The figure demonstrates how DBSCAN forms clusters by expanding from dense regions through chains of density-reachable points while leaving sparse points unclustered. This density-based approach enables the algorithm to detect clusters of irregular shapes and varying sizes without requiring a predefined number of clusters (UpGrad, n.d.).

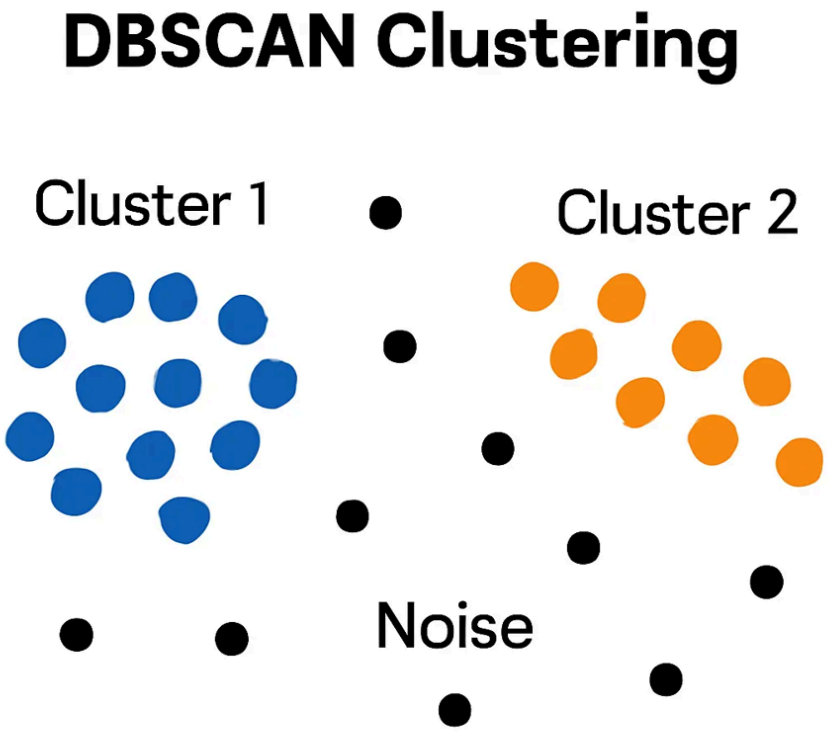

Figure 4.32 Density Reachability in DBSCAN

The concept of **density reachability in the DBSCAN clustering algorithm**, showing how clusters are formed through chains of density-connected points while sparse points remain classified as noise. In the diagram, the data points are divided into two dense groups labeled **Cluster 1** and **Cluster 2**, while scattered black points represent noise or outlier observations.

The elements shown in the figure correspond to the following DBSCAN clustering relationships:

- **Cluster 1 (Blue Points)** – The group of **blue points on the left side** represents a dense region of data points that are close enough to satisfy the ε-neighborhood and *minPts* requirements. Because these points are density-connected, DBSCAN groups them together to form a cluster.

- **Cluster 2 (Orange Points)** – The **orange points on the right side** form another dense region that satisfies the density conditions required for cluster formation. These points are connected through chains of density-reachable points, forming a second cluster separate from Cluster 1.

- **Density Reachability** – Points within each cluster are connected through neighboring points that satisfy the density requirement. A point is considered **density-reachable** from another point if it can be reached through a sequence of neighboring core points within the ε-neighborhood.

- **Noise Points (Black Points)** – The **scattered black points located between and around the clusters** represent noise or outliers. These points do not fall within the ε-neighborhood of any dense region and, therefore, are not assigned to any cluster.

This figure demonstrates how DBSCAN forms clusters based on **density connectivity rather than distance to a centroid**, allowing the algorithm to identify clusters of arbitrary shapes while effectively separating sparse data points as noise (STHDA, n.d.).

B. K-Means (Comparative Baseline)

K-Means clustering is applied as a centroid-based comparative baseline to evaluate whether the extracted cross-layer behavioral features form separable groupings under a partition-based clustering model. Unlike DBSCAN, which detects density-connected regions and identifies noise, K-Means partitions the dataset into k compact clusters by minimizing within-cluster variance (WCSS).

Clustering is performed on the standardized feature vectors aggregated per node per observation window. Unless otherwise specified, both cross-layer features and auxiliary tunnel features are included; however, additional experiments may exclude auxiliary features to evaluate separability based purely on network behavior.

Number of Clusters Selection:
The optimal number of clusters is determined using two complementary methods:

- Elbow Method: Plot of within-cluster sum of squares (WCSS) against increasing k; the elbow point indicates diminishing returns.
- Silhouette Analysis: Average silhouette width across all points for each k, with higher values indicating better cluster separation.

For this study, k values from 2 to 8 are evaluated. The lower bound ($k = 2$) reflects the minimum plausible behavioral separation (normal vs. anomalous), while higher values allow for potential differentiation between forwarding suppression, topology distortion, and transitional states. The final value of k is selected based on agreement between elbow behavior, silhouette score peaks, and cluster interpretability.

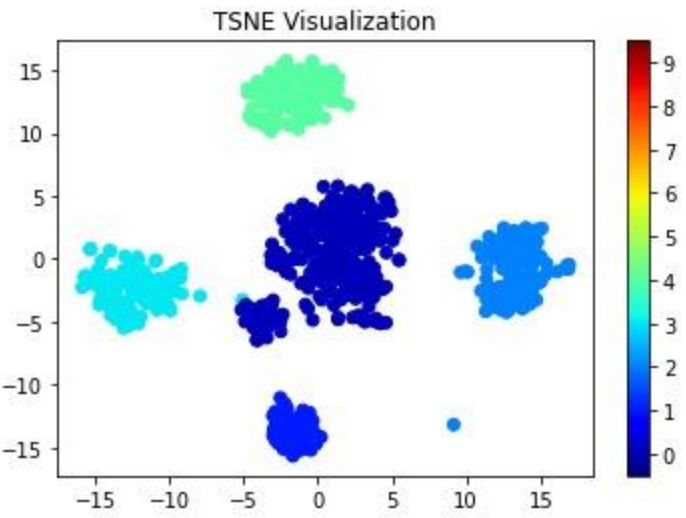

Figure 4.29 t-SNE Cluster Visualization (example TSNE figure)

A **t-Distributed Stochastic Neighbor Embedding (t-SNE)** visualization is used to display clustered data in a two-dimensional space. t-SNE is a dimensionality-reduction technique that maps high-dimensional feature vectors into a lower-dimensional representation while preserving local similarities between data points. This allows clusters present in the original high-dimensional dataset to become visually distinguishable (Bishop, 2006).

In the figure, each **dot represents a data point** projected into two dimensions using the t-SNE embedding. The **horizontal and vertical axes correspond to the two t-SNE dimensions**, which do not represent specific original features but instead reflect the relative similarity relationships between observations.

The figure shows several **distinct clusters of points**, indicating groups of data observations that share similar characteristics:

- The **large dark-blue cluster near the center** represents a dense grouping of closely related observations.
- The **cyan cluster on the left side** and the **light-blue cluster on the right side** represent additional groups that are separated from the central cluster based on feature similarity.
- The **green cluster located near the top of the plot** forms another clearly separated group.
- A **small cluster near the bottom of the plot** represents another localized grouping of similar data points.

The **color bar on the right side of the figure** indicates cluster labels or category identifiers assigned to each group. Different colors help visually distinguish clusters and highlight how the algorithm separates groups within the dataset.

The separation between clusters in the t-SNE space demonstrates how high-dimensional feature relationships can be visualized in two dimensions, making it easier to examine cluster structure and assess the separability of patterns within the dataset.

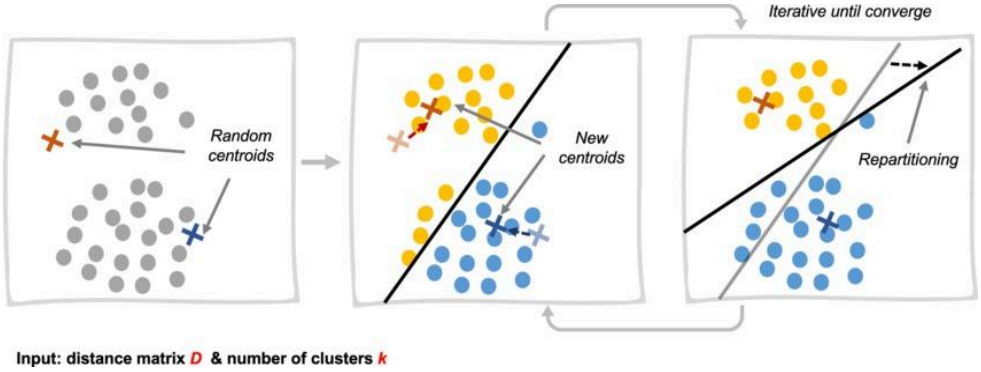

Figure 4.30 Step-by-Step Cluster Formation Visualization

The iterative process of cluster formation in the **K-Means clustering algorithm**. The diagram presents three stages showing how clusters are formed and refined through repeated centroid updates until convergence is achieved. Each panel represents a step in the clustering process, demonstrating how data points are assigned to clusters based on their distance to cluster centroids.

The elements shown in the figure correspond to the following steps of the K-Means algorithm:

- **Initial Centroid Selection (Left Panel)** – The first panel shows a dataset of scattered points along with **randomly initialized centroids**, represented by cross symbols. These centroids serve as the initial centers of the clusters before the algorithm begins assigning data points.

- **Cluster Assignment and Repartitioning (Middle Panel)** – In the second panel, the algorithm assigns each data point to the **nearest centroid** based on distance. The black boundary line indicates the decision boundary that separates the clusters. Points closer to one centroid are assigned to that cluster, forming preliminary cluster groups.

- **Centroid Update and Iteration (Right Panel)** – In the final panel, the centroids are **recomputed based on the mean position of the points assigned to each cluster**. After updating the centroids, the algorithm reassigns data points and recalculates centroids repeatedly. This iterative process continues until the centroids stabilize and the cluster assignments no longer change significantly.

The arrows between panels indicate the **repetition of the assignment and centroid update steps**, highlighting the iterative nature of the K-Means algorithm. Through this process, clusters gradually form as centroids move toward the centers of their respective groups of data points.

This visualization demonstrates how K-Means partitions a dataset into clusters by minimizing the distance between data points and their assigned centroids during each iteration (Bishop, 2006).

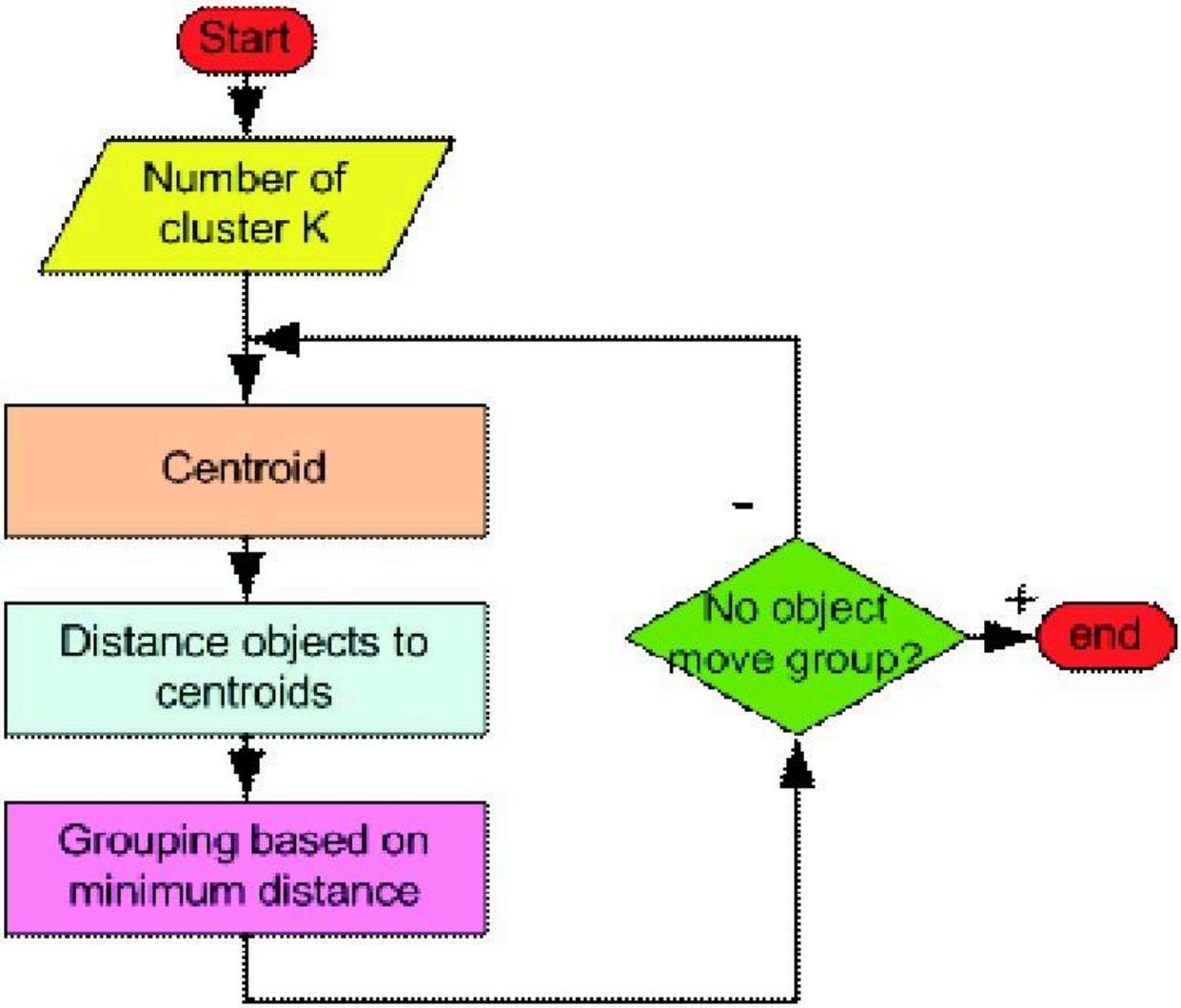

Figure 4.31 K-Means Algorithm Flowchart. Adapted from Lloyd (1982).

A flowchart illustrating the step-by-step procedure of the **K-Means clustering algorithm**. The diagram shows how data points are iteratively assigned to clusters based on their distance to cluster centroids until the cluster assignments stabilize. The flowchart begins with initialization and proceeds through repeated cluster assignment and centroid updates until convergence is reached.

The elements shown in the figure correspond to the following steps of the K-Means algorithm:

- **Start and Cluster Initialization** – The process begins at the **Start node**, followed by specifying the **number of clusters KKK**. This parameter determines how many clusters the dataset will be partitioned into.

- **Centroid Initialization** – The **"Centroid" process box** represents the step where initial centroids are selected. These centroids act as the starting centers of the clusters.

- **Distance Calculation** – The **"Distance objects to centroids" step** calculates the distance between each data point and the centroids. This distance measurement is typically computed using Euclidean distance.
- **Cluster Assignment** – In the **"Grouping based on minimum distance" step**, each data point is assigned to the cluster corresponding to the nearest centroid.
- **Convergence Check** – The **decision diamond labeled "No object move group?"** evaluates whether any data points change cluster assignments during the iteration. If cluster assignments continue to change, the algorithm returns to the centroid update step and repeats the process.
- **End Condition** – When no data points change clusters, indicating that the centroids have stabilized, the algorithm reaches the **End node**, and the final cluster configuration is produced.

The loop shown in the flowchart highlights the **iterative nature of the K-Means algorithm**, where centroid positions and cluster memberships are repeatedly updated until convergence is achieved. This iterative optimization process minimizes the distance between data points and their corresponding cluster centroids (Lloyd, 1982).

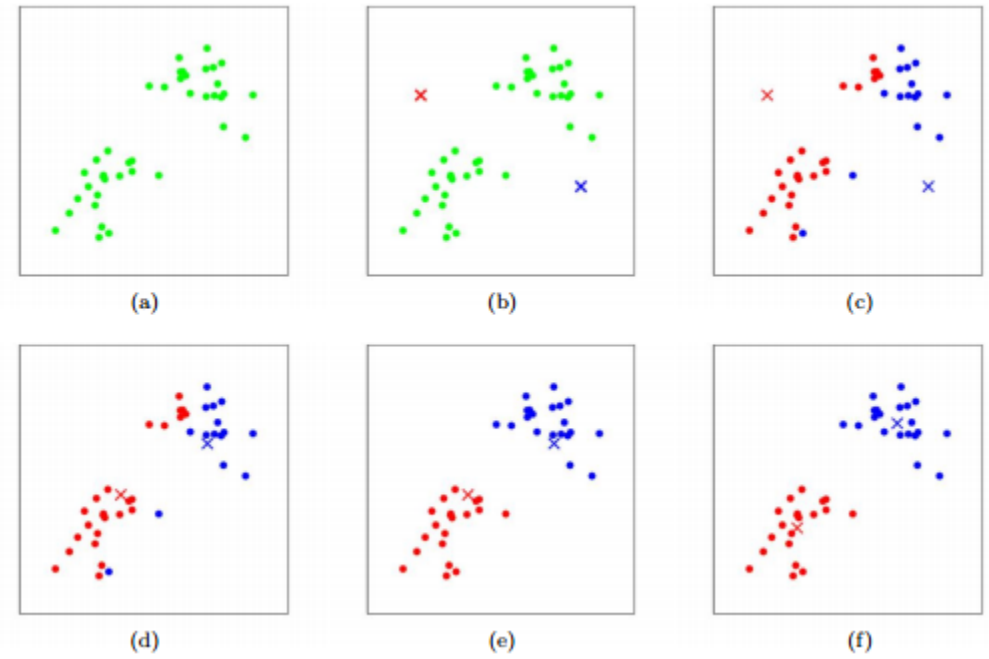

Figure 4.32 Step-by-Step Cluster Formation Visualization. Adapted from Bishop (2006).

The **iterative process of cluster formation in the K-Means clustering algorithm**, showing how clusters gradually form as centroids are updated and data points are reassigned during successive iterations. The six panels labeled **(a) to (f)** represent different stages in the clustering process.

The steps illustrated in the figure correspond to the following stages of the K-Means algorithm:

- **(a) Initial Dataset Distribution** – The first panel shows the original dataset consisting of scattered data points before clustering begins. At this stage, no cluster assignments have been made.

- **(b) Random Centroid Initialization** – In the second panel, two centroids (marked with "×") are randomly placed within the dataset. These centroids serve as the initial cluster centers from which the algorithm begins the clustering process.

- **(c) First Cluster Assignment** – The algorithm assigns each data point to the nearest centroid based on distance. As shown in the figure, points begin to separate into two groups represented by different colors (red and blue).

- **(d) Centroid Recalculation** – After the initial assignment, the centroids are updated by computing the mean position of the data points assigned to each cluster. The new centroid positions are then used for the next iteration.

- **(e) Reassignment of Data Points** – Using the updated centroids, the algorithm recalculates distances and reassigns data points to the closest centroid. This step may change the cluster membership of some points.

- **(f) Convergence of Clusters** – In the final panel, the clusters stabilize as centroid positions stop changing significantly. At this stage, the algorithm converges, and the final cluster structure is obtained.

The figure demonstrates how the K-Means algorithm repeatedly performs **cluster assignment and centroid update steps** until the cluster configuration stabilizes. This iterative process minimizes the within-cluster variance by ensuring that data points remain close to their corresponding centroids (Bishop, 2006).

C. Hierarchical Clustering

Hierarchical clustering is applied as an additional unsupervised clustering approach to analyze the structural relationships between behavioral observations without requiring an explicit assumption about the number of clusters during the clustering process. Unlike partition-based algorithms such as K-Means, hierarchical clustering builds a nested hierarchy of clusters by iteratively merging or splitting data points based on similarity.

For this study, agglomerative hierarchical clustering is employed. In this approach, each observation initially forms its own cluster. Clusters are then progressively merged according to a defined linkage criterion until all observations are grouped into a single hierarchy. The resulting structure is represented through a dendrogram, which visualizes the sequence of cluster merges and the relative distances between clusters.

Clustering is performed on the standardized feature vectors derived from telemetry observations aggregated per node per observation window. Both cross-layer behavioral features and auxiliary tunnel metrics are included unless otherwise specified. Additional exploratory runs may also evaluate clustering

behavior using only cross-layer metrics to observe the separability of network-level behaviors independently of attack-specific features.

C. Linkage Method Selection

Cluster merging is determined using Ward's linkage criterion, which minimizes the increase in within-cluster variance when clusters are combined. Ward linkage is selected because it produces compact clusters that are comparable to those produced by K-Means, enabling meaningful comparison between partition-based and hierarchical clustering approaches.

The hierarchical clustering procedure is implemented as follows:

1. Each observation begins as an individual cluster.
2. Pairwise distances between clusters are computed using Euclidean distance.
3. The two clusters that produce the smallest increase in total within-cluster variance are merged.
4. Steps 2–3 are repeated until all observations are merged into a hierarchical structure.

D. Cluster Selection

Although hierarchical clustering produces a full cluster hierarchy, the dendrogram is cut at selected distance thresholds to produce cluster partitions for evaluation. Similar to the K-Means baseline experiment, cluster counts between 2 and 8 clusters are evaluated. These values allow the analysis to determine whether hierarchical clustering reveals interpretable behavioral groupings corresponding to normal operation, forwarding suppression, topology distortion, or intermediate behavioral states.

Cluster partitions obtained from the dendrogram are evaluated using the same internal validation metrics used for other clustering methods, including silhouette score and Davies–Bouldin index, allowing direct comparison with results obtained from DBSCAN and K-Means.

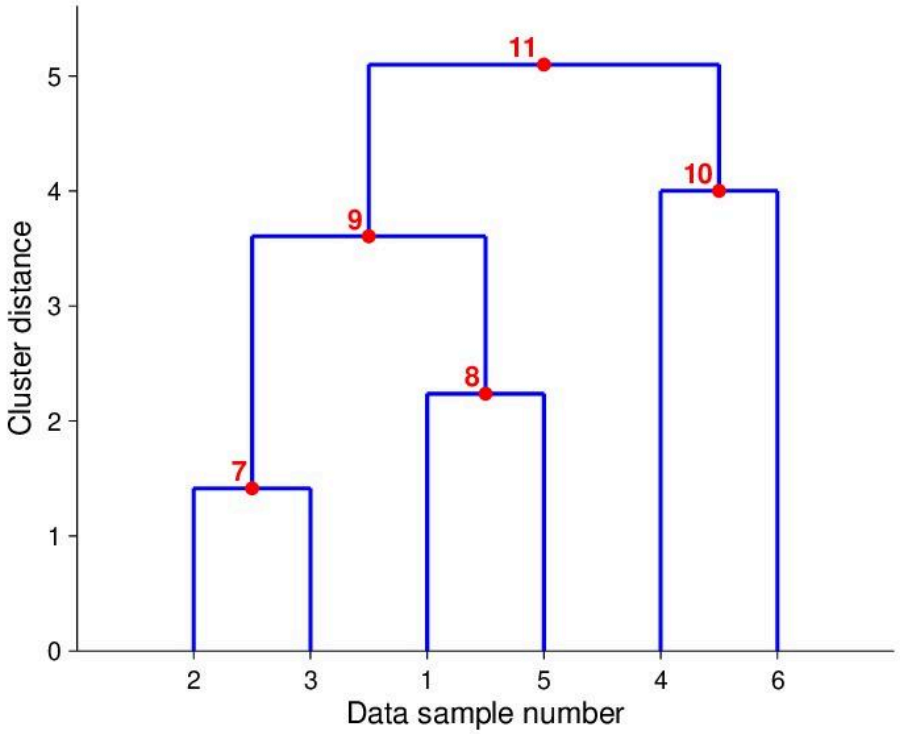

Figure 4.33 Example Dendrogram

Dendrogram used in hierarchical clustering to visualize how data points are grouped into clusters based on their similarity or distance. A dendrogram is a tree-like diagram that illustrates the sequence of cluster merging steps and the relative distances at which clusters are combined.

In the diagram, the **horizontal axis labeled "Data sample number"** represents the individual observations in the dataset. Each sample initially begins as its own cluster at the bottom of the dendrogram. The **vertical axis labeled "Cluster distance"** represents the distance or dissimilarity between clusters, indicating the level at which clusters are merged during the clustering process.

The clustering process illustrated in the figure proceeds as follows:

- **Initial Clusters** – The individual data samples (1, 2, 3, 4, 5, and 6) appear along the bottom of the dendrogram, each representing a separate cluster at the start of the hierarchical clustering process.

- **First Merging Step (Point 7)** – Data samples **2 and 3** are merged to form a cluster at a relatively small distance value, indicating that these observations are highly similar.

- **Second Merging Step (Point 8)** – Data samples **1 and 5** are combined to form another cluster at a slightly higher distance level.

- **Intermediate Cluster Formation (Point 9)** – The cluster formed by **samples 2 and 3** merges with the cluster formed by **samples 1 and 5**, producing a larger cluster that groups four observations.

- **Additional Cluster Formation (Point 10)** – Data samples **4 and 6** merge to form another cluster at a moderate distance level.

- **Final Cluster Merge (Point 11)** – The two larger clusters eventually merge at the highest distance level shown in the dendrogram, forming a single cluster that contains all observations.

The height at which clusters merge represents the **distance threshold between clusters**, allowing researchers to determine the number of clusters by selecting a horizontal cut across the dendrogram. Clusters formed below the chosen threshold are considered separate groups within the dataset.

This dendrogram demonstrates how hierarchical clustering builds clusters progressively by merging the most similar observations first and gradually combining larger clusters at higher distance levels (ResearchGate, n.d.).

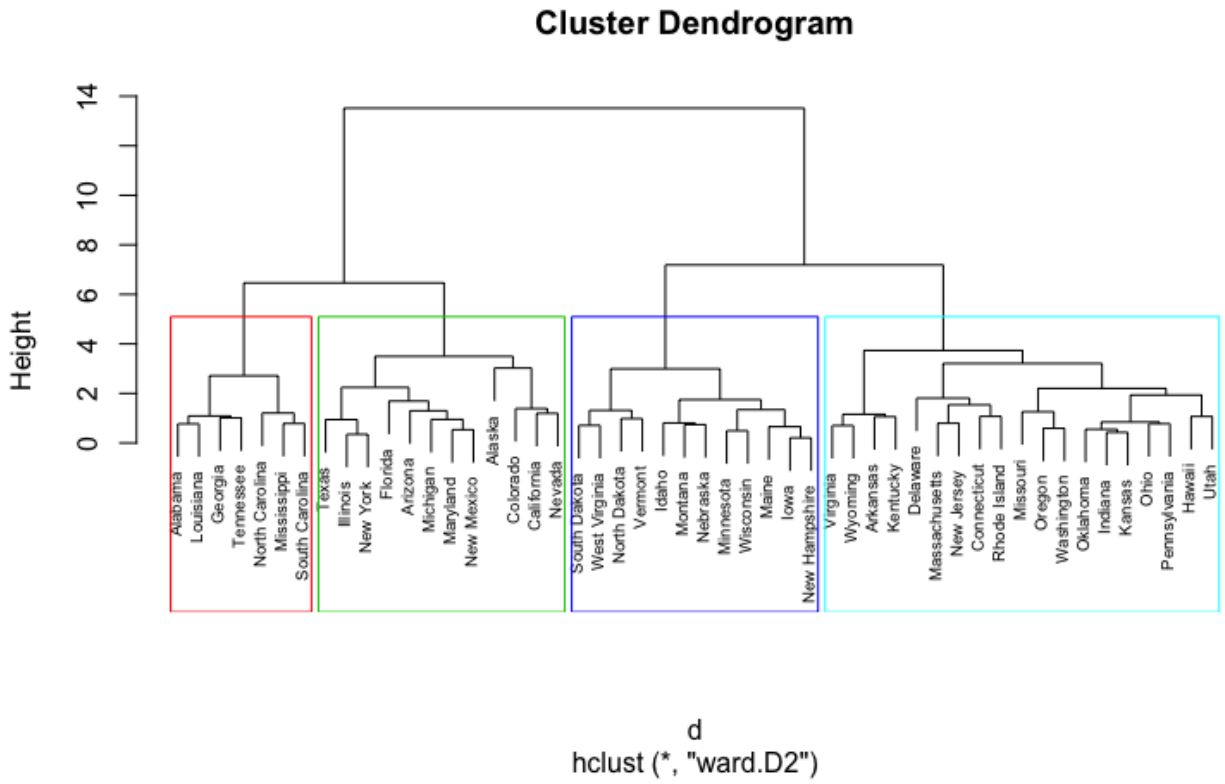

Figure 4.34 Hierarchical Clustering Dendrogram (Ward Method)

A **hierarchical clustering dendrogram generated using Ward's linkage method**, which groups observations by minimizing the variance within clusters during the merging process. The dendrogram visually represents how clusters are formed through successive merging steps based on the similarity between observations.

In the diagram, the **horizontal axis displays the individual observations**, represented by labeled data points positioned at the bottom of the dendrogram. Each observation initially begins as its own cluster.

The **vertical axis labeled "Height"** represents the distance or dissimilarity level at which clusters are merged during the hierarchical clustering process.

The structure of the dendrogram illustrates the following clustering behavior:

- **Initial Individual Observations** – At the bottom of the dendrogram, each labeled item begins as a separate cluster. These individual observations are gradually merged with other similar observations as the algorithm proceeds.

- **Cluster Merging Process** – As the height increases along the vertical axis, branches of the dendrogram merge. Each horizontal linkage line represents the point at which two clusters are combined based on their similarity.

- **Ward's Linkage Method** – Ward's method merges clusters in a way that minimizes the total within-cluster variance. This approach results in clusters that are relatively compact and balanced compared to other linkage methods.

- **Cluster Groupings** – The colored rectangular boxes in the figure highlight **four major clusters** identified in the dataset. These colored regions show how the dendrogram can be "cut" at a specific height threshold to determine the final number of clusters.

- **Cluster Hierarchy Representation** – The hierarchical tree structure demonstrates how smaller clusters merge into larger clusters as the distance threshold increases, ultimately forming a single cluster that contains all observations at the top of the dendrogram.

This dendrogram provides a visual representation of the hierarchical relationships between data points and helps determine the appropriate number of clusters by selecting a cutoff level along the height axis. Ward's method is commonly used because it produces clusters with relatively small internal variance and well-balanced groupings (UC Business Analytics R Programming Guide, n.d.).

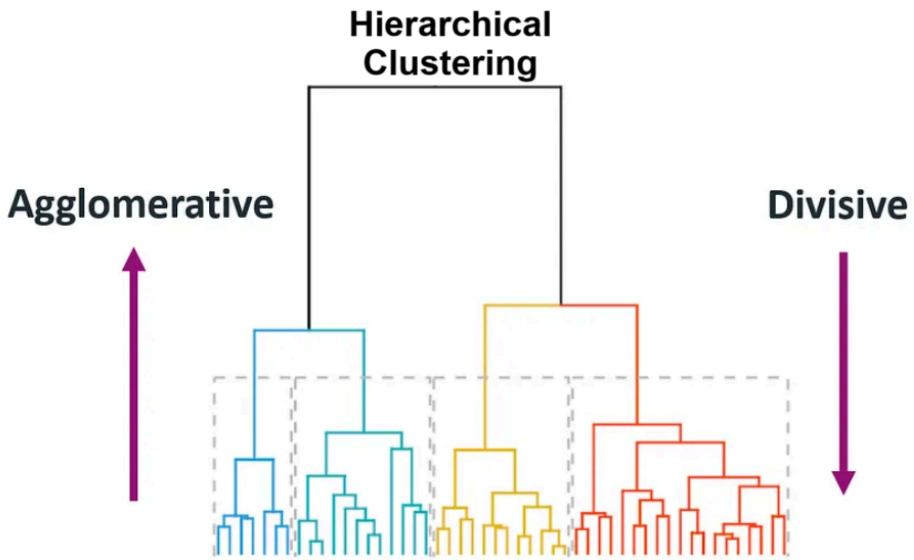

Figure 4.35 Agglomerative vs Divisive Hierarchical Clustering

A comparison between the two main approaches used in **hierarchical clustering: agglomerative clustering and divisive clustering**. The diagram highlights how clusters are formed either by **merging smaller clusters into larger ones** or by **splitting larger clusters into smaller groups**.

In the center of the figure, the **hierarchical tree structure (dendrogram)** illustrates how data points can be grouped into clusters at different levels of similarity. The colored branches at the bottom represent groups of observations that form clusters as the hierarchical process progresses.

The figure highlights the two clustering strategies as follows:

- **Agglomerative Clustering (Bottom-Up Approach)** – The arrow on the **left side labeled "Agglomerative" pointing upward** indicates that clustering begins with individual data points. Each observation initially forms its own cluster, and the algorithm repeatedly merges the most similar clusters until larger clusters are formed. This bottom-up process continues until all data points are combined into a single cluster.

- **Divisive Clustering (Top-Down Approach)** – The arrow on the **right side labeled "Divisive" pointing downward** represents the opposite strategy. In divisive clustering, the process begins with all observations grouped into one large cluster. The algorithm then recursively splits this cluster into smaller subclusters based on dissimilarity until individual clusters are formed.

- **Hierarchical Tree Representation** – The dendrogram in the center visually represents the hierarchical relationships between clusters. Each branching point in the tree indicates a merging or splitting step, depending on the clustering approach used.

The figure demonstrates how both agglomerative and divisive methods produce hierarchical cluster structures, but they differ in the direction in which clusters are formed. Agglomerative clustering is the more commonly used approach due to its computational efficiency and simpler implementation (Minitab, n.d.).

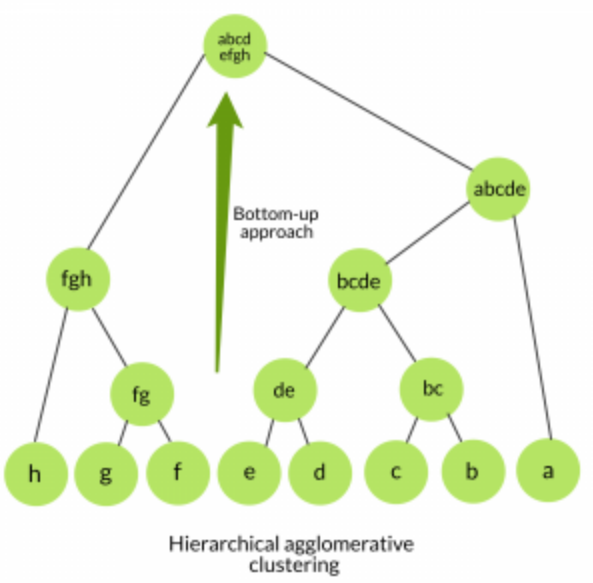

Figure 4.36 Agglomerative Hierarchical Clustering Tree

The structure of **Agglomerative Hierarchical Clustering** is a clustering technique that groups data points through a **bottom-up merging process**. In this approach, each data point initially forms its own cluster, and clusters are progressively merged based on similarity until a hierarchical structure is formed.

In the diagram, the **leaf nodes at the bottom of the tree (labeled a, b, c, d, e, f, g, h)** represent individual data points at the start of the clustering process. Each of these points begins as a separate cluster.

The clustering process then proceeds through the following stages:

- **Initial Cluster Formation** – Individual data points at the bottom level form the smallest clusters. For example, points **b and c merge to form cluster "bc"**, while **d and e merge to form cluster "de."**

- **Intermediate Cluster Merging** – As the algorithm continues, smaller clusters combine to form larger clusters. In the figure, clusters such as **bc and de merge into a larger cluster labeled "bcde."** Similarly, clusters **f and g merge into "fg,"** which then combines with **h** to form a larger cluster.

- **Higher-Level Cluster Formation** – These intermediate clusters are further merged into larger groupings, such as **"abcde"** and **"abcd efgh."** Each level of the tree represents a new stage in the merging process.

- **Bottom-Up Hierarchical Structure** – The **green arrow labeled "Bottom-up approach"** highlights the direction of the clustering process. Starting from individual data points at the bottom, clusters are gradually merged upward until a hierarchical tree structure is formed.

This tree structure, commonly referred to as a **dendrogram**, visually represents the relationships between clusters and shows how clusters combine at different levels of similarity. Agglomerative hierarchical

clustering does not require a predefined number of clusters, allowing the final number of clusters to be determined by selecting an appropriate level in the hierarchy (Singh, n.d.).

E. Gaussian Mixture Model (GMM)

Gaussian Mixture Models (GMM) are used as a probabilistic clustering approach that models the dataset as a combination of multiple Gaussian distributions in feature space. Unlike hard clustering methods such as K-Means, which assign each observation to a single cluster, GMM performs soft clustering, assigning each observation a probability of belonging to each cluster.

This probabilistic representation allows the model to capture overlapping behavioral patterns that may occur in network telemetry data. In the context of ESP32 mesh network behavior, some observation windows may exhibit characteristics that lie between normal operation and anomalous routing behavior. GMM provides a mechanism for representing these transitional states through probabilistic cluster membership.

The model assumes that the dataset is generated from a mixture of k Gaussian components, where each component represents a latent behavioral state. Each component is characterized by:

- a mean vector
- a covariance matrix
- a mixture weight representing its relative contribution

Parameter estimation is performed using the Expectation–Maximization (EM) algorithm, which iteratively refines cluster parameters until convergence.

F. Expectation–Maximization Procedure

The clustering process proceeds through two alternating steps:

Expectation step (E-step):
The probability that each observation belongs to each Gaussian component is estimated based on the current model parameters.

Maximization step (M-step):
The parameters of the Gaussian components (means, covariances, and mixture weights) are updated to maximize the likelihood of the observed data given the current cluster assignments.

This iterative process continues until the log-likelihood improvement between successive iterations falls below a convergence threshold.

G. Cluster Number Selection

To maintain comparability with the K-Means baseline experiment, candidate cluster counts ranging from 2 to 8 components are evaluated. The optimal number of mixture components is determined using both cluster validation metrics and the interpretability of resulting behavioral groupings.

Cluster quality is assessed using the silhouette score and Davies–Bouldin index, while additional inspection of cluster membership probabilities allows identification of observations that exhibit mixed behavioral characteristics. Such transitional patterns may indicate periods where network behavior shifts between normal and anomalous conditions.

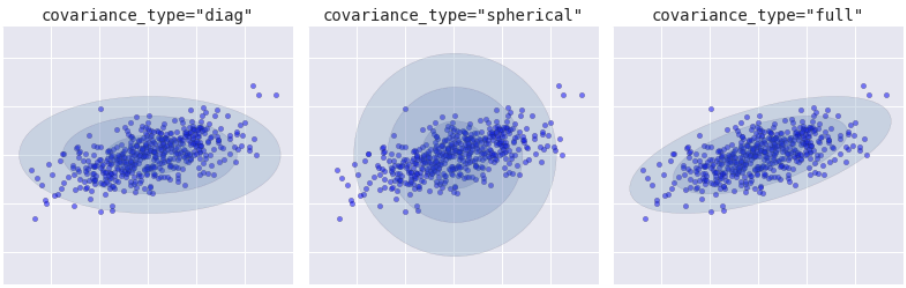

Figure 4.37 Covariance Types in Gaussian Mixture Models

The different **covariance structures used in Gaussian Mixture Models (GMM)** for modeling clusters in a dataset. Gaussian Mixture Models represent clusters as probability distributions rather than fixed boundaries, allowing clusters to take on different shapes depending on the covariance configuration. The shaded ellipses in the figure represent the estimated covariance of each Gaussian component, indicating the spread and orientation of the cluster.

The figure presents three common covariance types used in GMM clustering:

- **Diagonal Covariance (Left Panel – covariance_type = "diag")** – In this configuration, the covariance matrix is restricted to diagonal values, meaning each feature dimension varies independently while correlations between dimensions are ignored. As shown in the left panel, clusters appear as **axis-aligned ellipses**, where the spread of data differs along each axis but cannot rotate.

- **Spherical Covariance (Middle Panel – covariance_type = "spherical")** – In the spherical covariance structure, each cluster has a single variance value applied equally across all dimensions. This results in clusters that appear as **circular shapes** in the visualization. The middle panel shows clusters that expand uniformly in all directions, assuming equal variance across the feature space.

- **Full Covariance (Right Panel – covariance_type = "full")** – In the full covariance structure, the covariance matrix allows correlations between features. This enables clusters to take **elliptical shapes that can rotate and stretch in any direction**, as illustrated in the right panel. This configuration provides the most flexibility for modeling complex cluster structures.

The comparison shown in the figure demonstrates how different covariance assumptions affect the shape and orientation of clusters identified by Gaussian Mixture Models. Choosing an appropriate covariance type is important for accurately capturing the distribution of data within each cluster (VanderPlas, n.d.).

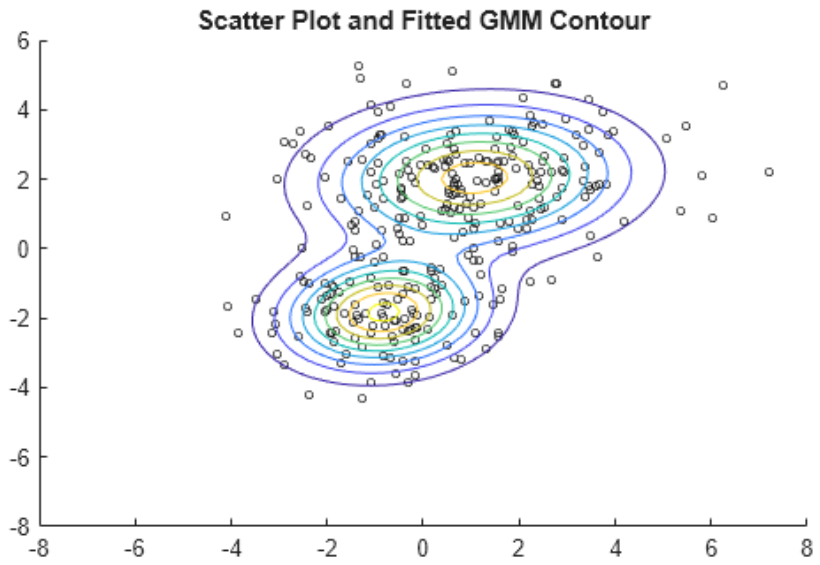

Figure 4.38 GMM Probability Density Contours

The clustering behavior of a **Gaussian Mixture Model (GMM)** using a scatter plot combined with probability density contours. The visualization shows how GMM models cluster as probability distributions rather than assigning rigid cluster boundaries.

In the figure, the **scatter points represent individual data observations** distributed across the feature space. These observations form two main groups of points, indicating the presence of two underlying clusters in the dataset.

The elements shown in the figure correspond to the following features of the Gaussian Mixture Model:

- **Scatter Data Points** – The individual points plotted on the graph represent data samples from the dataset. These points form visible groupings that the GMM algorithm attempts to model using Gaussian distributions.

- **Probability Density Contours** – The **elliptical contour lines surrounding the clusters** represent the probability density levels of the fitted Gaussian distributions. Each contour line indicates regions where the probability density of the cluster remains constant.

- **Cluster Centers** – The **central regions of the contour shapes** indicate the mean location of each Gaussian component. These means act as the centers of the clusters estimated by the GMM algorithm.

- **Elliptical Cluster Shape** – Unlike clustering methods such as K-Means that assume spherical clusters, GMM allows clusters to take **elliptical shapes with varying orientations and spreads**, which better represent complex data distributions.

The figure demonstrates how Gaussian Mixture Models estimate the probability distribution of clusters and visualize cluster boundaries using density contours. This probabilistic approach allows GMM to

capture overlapping clusters and complex data structures more effectively than many distance-based clustering methods (MathWorks, n.d.).

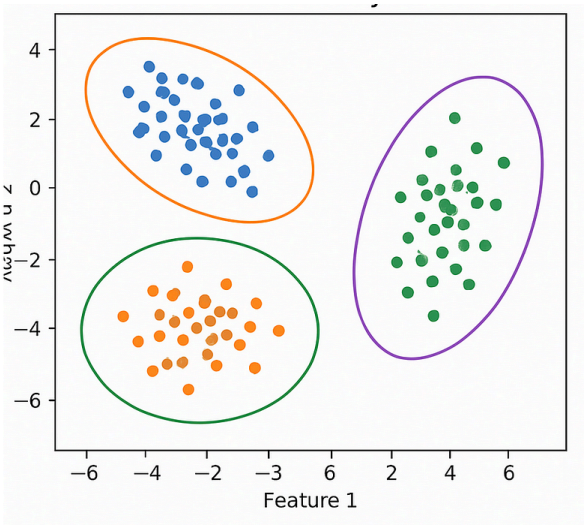

Figure 4.39 Gaussian Mixture Model Clustering Example

An example of clustering using a **Gaussian Mixture Model (GMM)**, where clusters are represented as Gaussian probability distributions rather than fixed geometric boundaries. The visualization shows how GMM identifies clusters in a dataset by estimating the probability distribution of data points within the feature space.

In the figure, the **scatter points represent individual data observations** plotted according to two features labeled *Feature 1* and *Feature 2*. The points form three visible groups that correspond to the clusters detected by the Gaussian Mixture Model.

The elements shown in the figure correspond to the following characteristics of GMM clustering:

- **Clustered Data Points** – The data points form three main groups in the scatter plot. These groups represent observations that share similar characteristics within the feature space.
- **Elliptical Cluster Boundaries** – The **elliptical shapes surrounding each cluster** represent the Gaussian distributions estimated by the model. These ellipses indicate the spread and orientation of the probability density for each cluster.
- **Cluster Centers and Orientation** – The center of each ellipse represents the **mean of the Gaussian component**, while the orientation and shape of the ellipse reflect the covariance structure of the cluster.
- **Flexible Cluster Shapes** – Unlike clustering algorithms such as K-Means that assume spherical clusters, Gaussian Mixture Models allow clusters to take **elliptical shapes with different orientations**, enabling the algorithm to better model complex data distributions.

The figure demonstrates how Gaussian Mixture Models represent clusters probabilistically and capture variations in cluster shape, size, and orientation within the dataset. This flexibility makes GMM

particularly useful for datasets where clusters overlap or exhibit non-spherical structures (AI Learning Hub, n.d.).

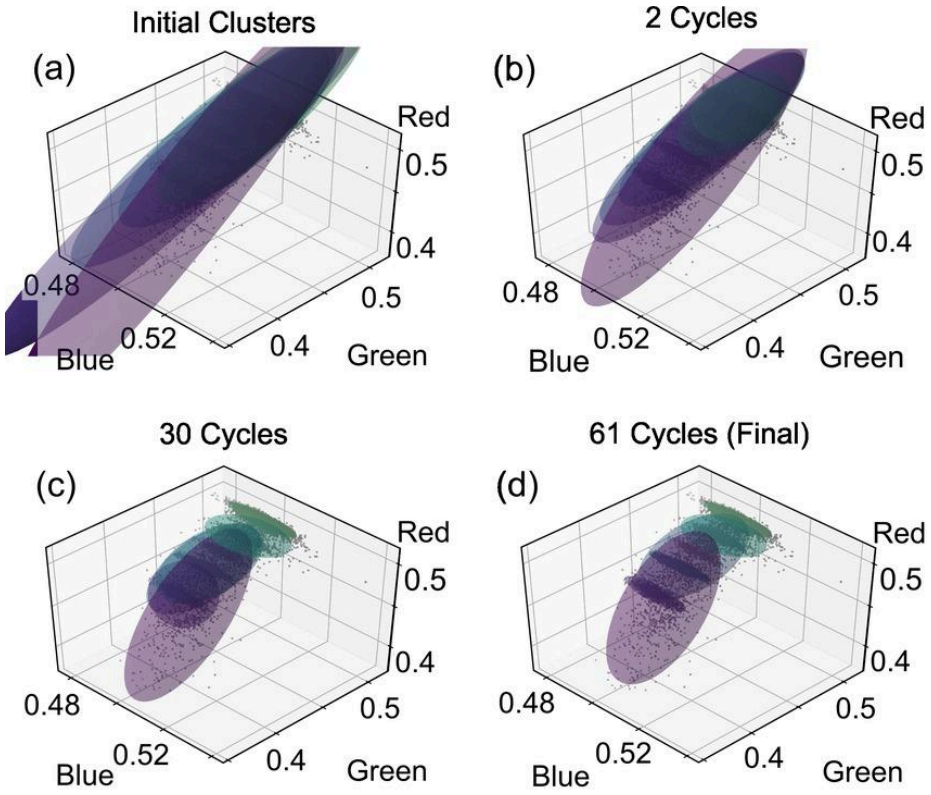

Figure 4.40 Cluster Fitting Using Gaussian Mixture Model

The **cluster fitting process of a Gaussian Mixture Model (GMM)** is the algorithm that iteratively updates its Gaussian components during the Expectation–Maximization (EM) procedure. The visualization shows how the Gaussian distributions gradually adjust their position, orientation, and spread to better represent the structure of the data.

In the figure, each panel represents a stage in the iterative clustering process, where the axes labeled **Red, Green, and Blue** correspond to three feature dimensions used in the dataset. The shaded ellipsoids represent the **Gaussian components**, while the scatter points correspond to the observed data samples.

The panels illustrate the following stages of the GMM fitting process:

- **(a) Initial Clusters** – The first panel shows the initial placement of Gaussian components before the algorithm begins adjusting them. At this stage, the ellipsoids represent preliminary estimates of the cluster distributions and may not yet align well with the underlying data.

- **(b) Early Iterations (2 Cycles)** – After two EM cycles, the Gaussian components begin to shift and rotate to better match the distribution of data points. The algorithm adjusts the parameters of each Gaussian component based on the likelihood of the observed data.

- **(c) Intermediate Iterations (30 Cycles)** – After additional iterations, the Gaussian components more closely align with the dense regions of the dataset. The ellipsoids begin to capture the structure and orientation of the clusters more accurately.

- **(d) Final Convergence (61 Cycles)** – In the final panel, the Gaussian components stabilize as the algorithm converges. At this stage, the ellipsoids closely represent the underlying data distribution, indicating that the model parameters have converged and the clustering process is complete.

The figure demonstrates how the Gaussian Mixture Model iteratively refines cluster parameters through the EM algorithm, adjusting the means, covariances, and mixture weights until the model converges to a stable solution that best fits the observed data (ResearchGate, n.d.).

E. Spectral Clustering

Spectral clustering is employed as a graph-based clustering technique capable of detecting complex cluster structures that may not be captured effectively by centroid-based or density-based methods. Unlike K-Means and GMM, which rely primarily on geometric distances in feature space, spectral clustering operates by analyzing the connectivity structure of the dataset represented as a similarity graph.

In this approach, each observation is treated as a node in a graph, and edges between nodes represent similarity relationships between feature vectors. The clustering process is performed by analyzing the eigenstructure of the graph Laplacian derived from this similarity matrix.

H. Similarity Graph Construction

To construct the similarity graph, pairwise similarities between standardized feature vectors are computed using a radial basis function (RBF) kernel:

Similarity between observations increases as the Euclidean distance between their feature vectors decreases. This representation allows spectral clustering to capture non-linear relationships in the data that may correspond to subtle behavioral patterns within the mesh network.

I. Spectral Embedding

Once the similarity matrix is constructed, the normalized graph Laplacian is computed. Eigenvalue decomposition is then performed on the Laplacian matrix to obtain a lower-dimensional representation of the data known as the spectral embedding.

This transformation projects the observations into a new space in which cluster structure becomes more separable. After projection, a conventional clustering algorithm (typically K-Means) is applied to the embedded representation to produce the final cluster assignments.

J. Cluster Number Selection

As with the previous clustering methods, candidate cluster counts ranging from 2 to 8 clusters are evaluated. This consistent range allows comparison across all clustering algorithms used in the study.

Cluster quality is assessed using the same internal validation metrics applied to other clustering methods, including the silhouette score and Davies–Bouldin index. The results are further examined to determine whether spectral clustering reveals behavioral structures not detected by the other clustering techniques, particularly in cases where clusters exhibit non-spherical or irregular shapes.

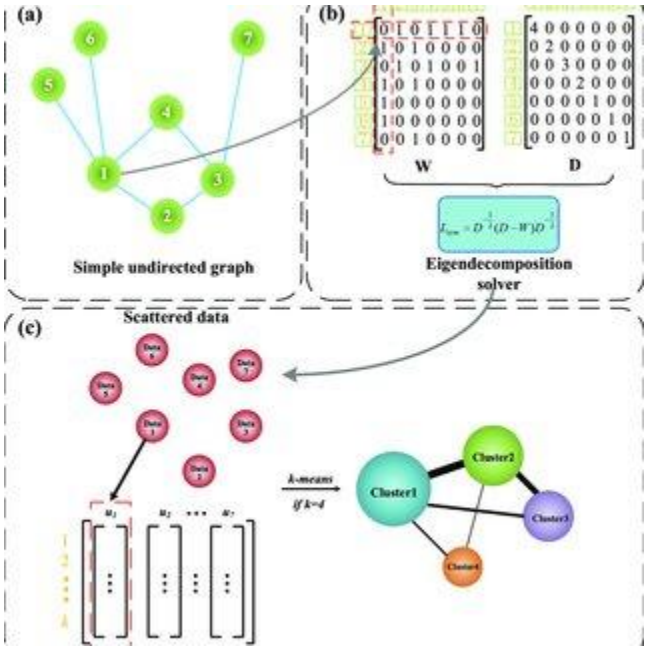

Figure 4.41 Spectral Clustering Algorithm Overview

Overview of the Spectral Clustering algorithm, which groups data points by analyzing the structure of a similarity graph. Spectral clustering transforms the dataset into a graph representation, computes eigenvectors from the graph Laplacian matrix, and then performs clustering in the transformed feature space.

The diagram is divided into three main stages labeled **(a), (b), and (c)**, each representing a step in the spectral clustering workflow.

- **(a) Graph Construction** – The upper-left section shows a **simple undirected graph** where nodes represent individual data points, and edges represent similarity relationships between points. This graph representation captures how strongly each data point is connected to others in the dataset.

- **(b) Matrix Representation and Eigen Decomposition** – The upper-right section illustrates the conversion of the graph into matrix form. The **weight matrix (W)** represents the similarity between nodes, while the **degree matrix (D)** summarizes the total connection strength for each node. These matrices are used to compute the **graph Laplacian**, and eigen decomposition is applied to extract eigenvectors that capture the structural properties of the graph.

- **(c) Feature Embedding and Clustering** – In the lower section of the figure, the eigenvectors obtained from the spectral decomposition form a new feature space. The data points are projected into this space and then clustered using a standard clustering algorithm, such as **K-Means**, to produce the final cluster groups.

The figure demonstrates how spectral clustering combines **graph theory and linear algebra** to identify clusters based on connectivity patterns rather than simple distance measures. This approach is particularly effective for datasets where clusters have complex or non-convex shapes (ResearchGate, n.d.).

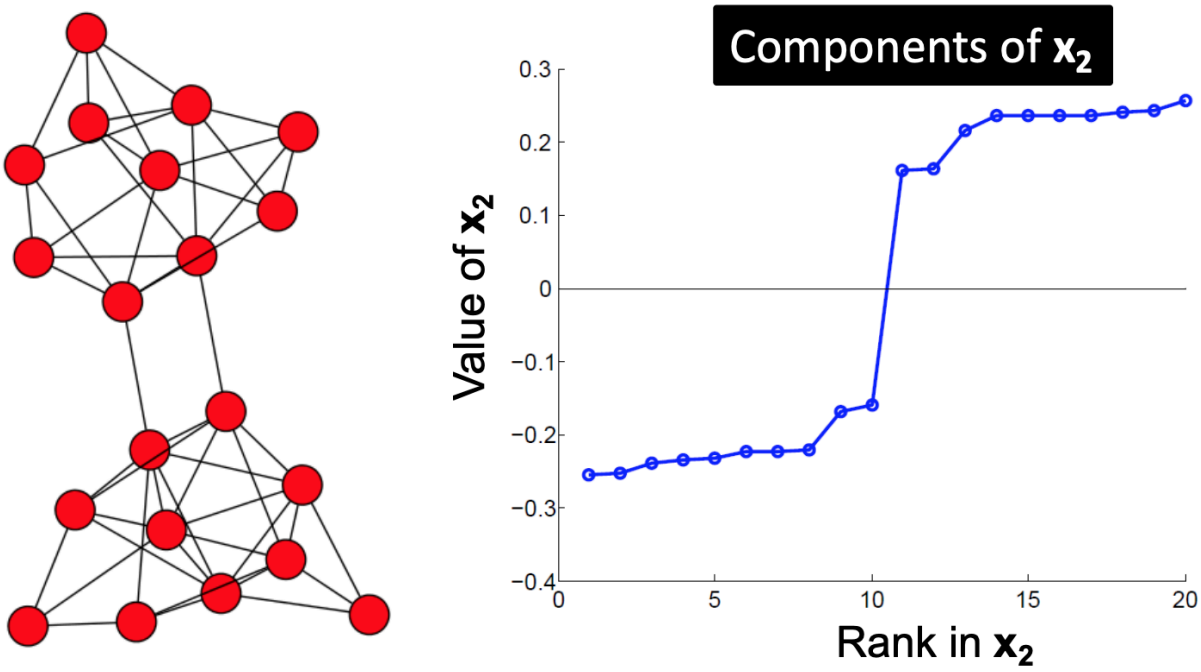

Figure 4.42 Eigenvector Components Used in Spectral Clustering

Demonstrates how **eigenvector components are used in spectral clustering to identify clusters within a graph structure**. Spectral clustering analyzes the connectivity between nodes in a graph and uses eigenvectors derived from the graph Laplacian matrix to reveal underlying cluster structures.

In the figure, the **left panel shows a network graph consisting of multiple nodes connected by edges**, where each node represents a data point, and edges represent similarity relationships between nodes. The structure of the graph suggests that the nodes form two main groups that are more densely connected internally than with nodes from other groups.

The **right panel displays a plot labeled "Components of x2x_2x2"**, which represents the values of the second eigenvector obtained from the spectral decomposition of the graph Laplacian matrix. The horizontal axis labeled **"Rank in x2x_2x2"** orders the nodes according to their eigenvector component values, while the vertical axis shows the corresponding **eigenvector value for each node**.

The figure highlights the following characteristics of spectral clustering:

- **Graph Representation of Data** – The network graph on the left illustrates how data points are represented as nodes connected by edges based on similarity relationships.

- **Eigenvector Component Values** – The plot on the right shows the values of the second eigenvector for each node in the graph. These values capture important structural information about the connectivity patterns within the graph.

- **Cluster Separation** – A noticeable **gap between eigenvector values** appears in the middle of the plot, indicating a natural division between two groups of nodes. Nodes with similar eigenvector values tend to belong to the same cluster.

- **Spectral Partitioning** – By using the eigenvector values as features, the spectral clustering algorithm can separate nodes into clusters based on their position relative to this separation point.

This visualization demonstrates how spectral clustering leverages eigenvectors of the graph Laplacian to reveal cluster structure within graph-based data, allowing clusters to be identified even when they are not easily separable using traditional distance-based methods (Syllogismos, n.d.).

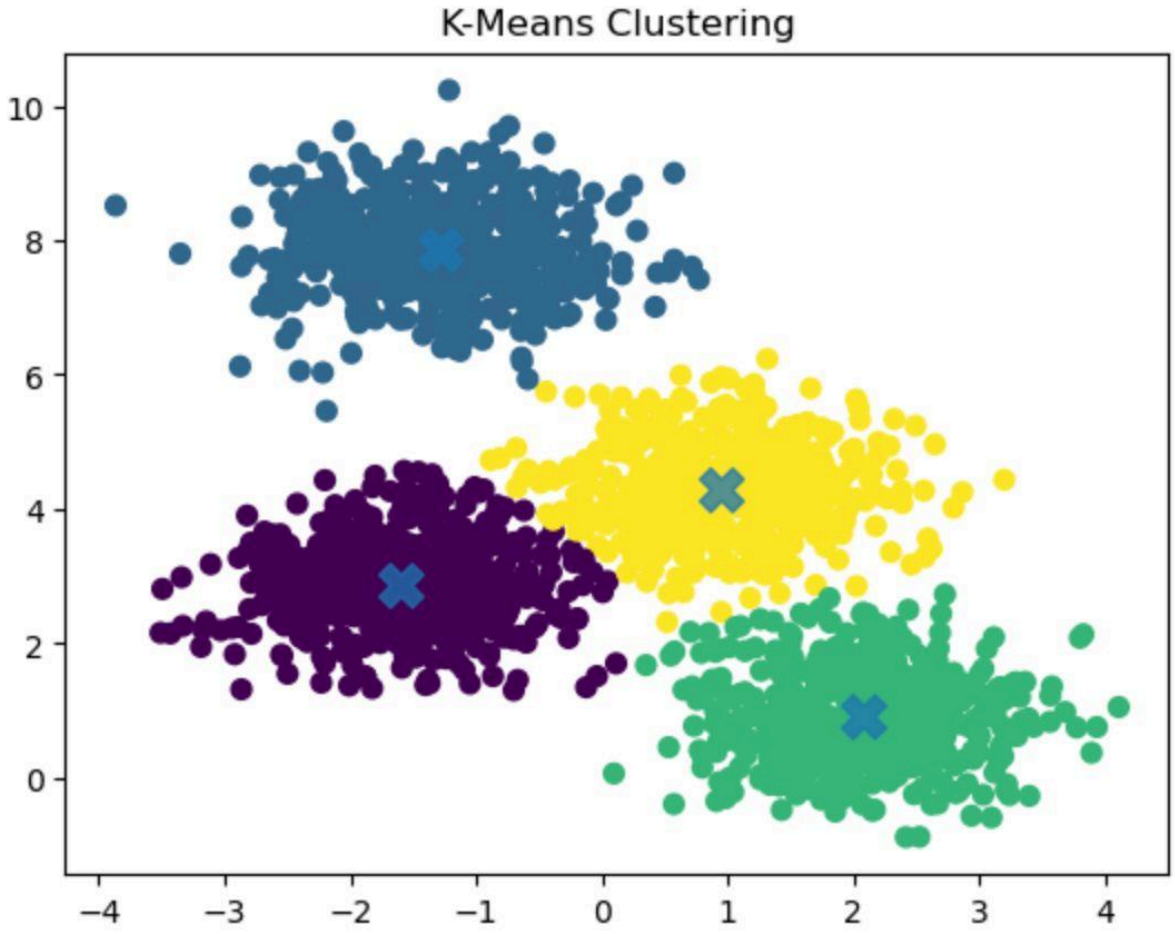

Figure 4.43 Example of K-Means Clustering

Example of clustering using the **K-Means algorithm**, which partitions a dataset into a predefined number of clusters by minimizing the distance between data points and their corresponding cluster centroids. The visualization shows how data points are grouped into clusters based on similarity in the feature space.

In the figure, each **data point represents an observation plotted in a two-dimensional feature space**, where the horizontal and vertical axes correspond to two features used for clustering. The points form several dense groups that the K-Means algorithm identifies as clusters.

The elements shown in the figure correspond to the following characteristics of the K-Means clustering process:

- **Clustered Data Points** – The scatter plot shows four distinct groups of points represented by different colors. Each color corresponds to a cluster identified by the algorithm, indicating that the observations within the same group share similar feature characteristics.

- **Cluster Centroids** – The **large "×" markers located at the center of each cluster** represent the centroids. These centroids correspond to the mean position of all data points assigned to each cluster and serve as the reference points for cluster assignment.

- **Cluster Partitioning** – Each data point is assigned to the cluster whose centroid is closest in terms of distance, typically measured using Euclidean distance. This results in the formation of clearly separated groups within the dataset.
- **Distance Minimization** – The K-Means algorithm iteratively updates centroid positions and reassigns data points until the clusters stabilize, minimizing the total within-cluster variance.

The figure demonstrates how K-Means divides a dataset into clusters by grouping nearby observations around central centroid points, producing partitions that represent similar patterns within the data (StrataScratch, n.d.).

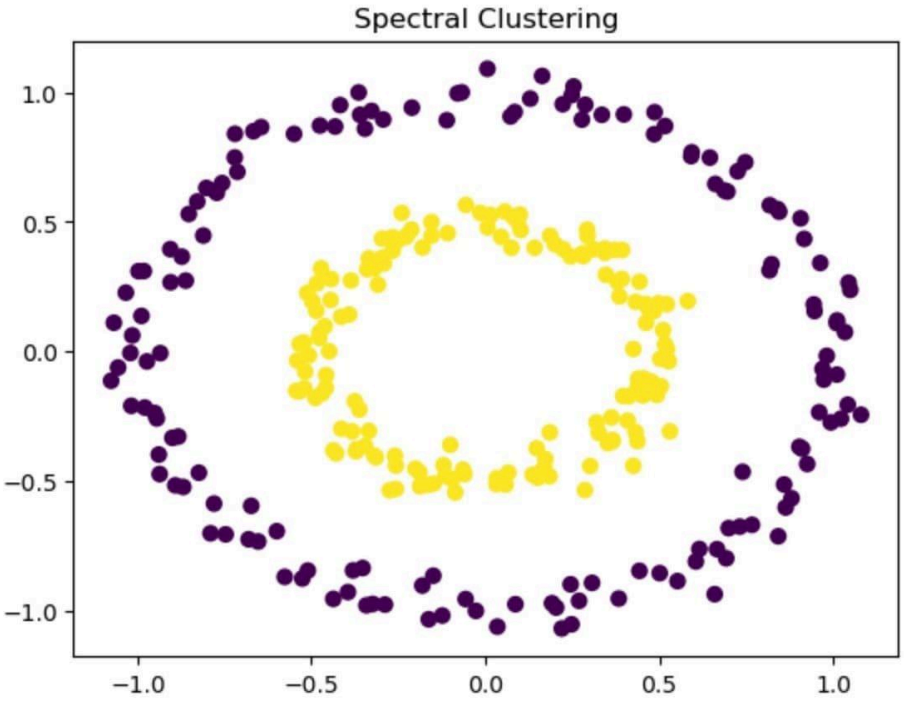

Figure 4.44 Spectral Clustering Result on Non-Linear Data

The application of **spectral clustering on a dataset with non-linearly separable structures**. Unlike traditional clustering algorithms such as K-Means that rely on distance-based partitioning, spectral clustering identifies clusters by analyzing the connectivity relationships between data points in a graph representation.

In the figure, the **scatter plot displays data points arranged in two concentric circular patterns**, forming an inner ring and an outer ring. Each point represents an observation in a two-dimensional feature space.

The visualization highlights the following aspects of spectral clustering:

- **Non-Linear Cluster Structures** – The dataset consists of two circular clusters that cannot be separated using a simple linear boundary. The **inner circular cluster (yellow points)** and the **outer circular cluster (purple points)** represent two distinct groups of observations.

- **Cluster Identification Through Graph Connectivity** – Spectral clustering constructs a similarity graph that captures the relationships between nearby data points. By analyzing the structure of this graph, the algorithm is able to detect clusters that share strong internal connections.
- **Separation of Complex Cluster Shapes** – The algorithm successfully separates the inner and outer circular groups despite their non-linear arrangement. This demonstrates the ability of spectral clustering to identify clusters with complex geometric shapes.
- **Cluster Visualization** – The different colors assigned to the two groups illustrate how spectral clustering partitions the dataset into clusters based on structural similarity rather than simple geometric distance.

The figure demonstrates the effectiveness of spectral clustering in detecting **non-linear and irregular cluster shapes**, making it particularly suitable for datasets where traditional centroid-based clustering methods fail to correctly separate clusters (StrataScratch, n.d.).

4.2.5.3. Cluster Evaluation Metrics

Even in unsupervised settings, internal validation metrics assess the quality and coherence of the clusters.

A. Silhouette Score

The silhouette score measures how similar a data point is to its own cluster compared to other clusters. The score ranges from −1 to 1, where higher values indicate better clustering structure.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$ **Equation 4.19**

where a(i) is the mean intra-cluster distance and b(i) is the mean nearest-cluster distance.

Interpretation

- > 0.7 → Strong cluster separation
- 0.5 – 0.7 → Reasonable cluster structure
- 0.2 – 0.5 → Weak but potentially meaningful structure
- < 0.2 → Poor cluster structure
- < 0 → Possible misclassification (point may be assigned to the wrong cluster)

B. Davies–Bouldin Index

The Davies–Bouldin index measures average similarity between each cluster and its most similar counterpart, with lower values indicating better separation.

$$DB = \frac{1}{k} \sum_{i=1}^{k} \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)} \right)$$ **Equation 4.20**

where σ_i is the average distance within cluster i and $d(c_i,c_j)$ is the distance between cluster centroids.

Interpretation:

- Lower values → Better cluster separation
- Values below 1.0 typically indicate well-separated clusters

C. Cluster Purity (Post-Analysis Only)

After clustering, cluster assignments are compared against ground-truth phase labels (baseline, suppression, distortion) to compute purity scores. This is strictly a post-analysis validation step, not used during clustering itself, and serves to assess whether discovered clusters align with experimental conditions.

$$\text{Purity} = \frac{1}{N} \sum_{i=1}^{k} \max_{j} \left| \text{cluster}_i \cap \text{class}_j \right| \qquad \textbf{Equation 4.21}$$

Higher purity indicates stronger alignment between unsupervised groupings and ground truth.

D. Adjusted Rand Index (Post-Analysis Only)

The Adjusted Rand Index (ARI) is a chance-corrected measure of agreement between two clusterings. It evaluates the similarity between the clustering produced by the algorithm and the ground-truth partition, while adjusting for the possibility that some agreement could occur purely by random chance.

$$\text{ARI} = \frac{\sum_{i,j} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}\right] / \binom{N}{2}}{\frac{1}{2}\left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}\right] / \binom{N}{2}} \qquad \textbf{Equation 4.22}$$

Where $\binom{x}{2} = \frac{x(x-1)}{2}$ is the binomial coefficient(number of unordered pairs from *x* items)

Measures similarity between cluster assignments and ground-truth labels, corrected for chance. Range: [-1, 1]; 1 indicates perfect agreement.

Interpretation

- **ARI = 1** indicates perfect agreement between the clustering and the ground truth (up to permutation of labels).
- **ARI = 0** indicates that the agreement is no better than what would be expected from random clustering.
- **ARI < 0** indicates agreement worse than chance

4.2.5.4. Cluster Characterization Using Ground Truth

After clustering, each cluster is profiled to understand its behavioral meaning. This step uses the ground-truth phase labels (baseline, blackhole, wormhole) for interpretation only—they are not used during clustering.

A. Label Distribution Per Cluster

For each cluster, we compute the percentage of windows belonging to each ground-truth phase. This reveals whether clusters correspond to distinct attack types or mix them. For example, a cluster may contain 95% baseline windows and 5% attack windows, indicating it predominantly represents normal behavior.

B. Feature Centroid Analysis

The mean value of each feature within a cluster is computed and compared to the global mean (across all windows). Features that are significantly elevated or depressed in a cluster help characterize its behavior. For instance, a cluster with a low forwarding ratio and high retry rate would be interpreted as "blackhole-affected."

C. Heatmap Visualization

A heatmap of feature means per cluster (standardized values) is generated to visually summarize the behavioral signature of each cluster. Rows represent clusters, columns represent features, and color intensity indicates deviation from the global mean.

D. Interpretation of Results

If the clustering yields only two clusters (e.g., benign vs. attack), we still characterize each cluster by its dominant label and feature profile. The ground-truth labels are used only after clustering to aid interpretation, not to guide the algorithm. This approach ensures that the discovered structure is data-driven, while still allowing us to map clusters to known experimental conditions.

4.2.5.5. Expected Outcomes

Based on the theoretical framework established in Chapter 3, the following outcomes are anticipated:

- **Normal operation windows** should form a dense, cohesive cluster characterized by forwarding ratios ≈ of 1.0, low retry rates, stable RSSI-hop relationships, and minimal parent switching.
- **Forwarding suppression windows** should form a distinct cluster (or be identified as noise) characterized by low forwarding ratios at attacker nodes and elevated retry rates at victim nodes.
- **Topology distortion windows** should form a separate cluster characterized by RSSI-hop inconsistency, latency-hop mismatch, increased parent switching, and tunnel activity metrics.

If these behavioral states are indeed separable in feature space, the clustering analysis will reveal distinct groupings aligning with the three experimental conditions. This separation would validate that the

collected cross-layer features capture meaningful distinctions between normal and manipulated network behaviors.

4.2.6. Exploratory Data Analysis (EDA)

Before clustering, exploratory data analysis is conducted to characterize the structure and quality of the extracted feature set. EDA serves as a descriptive sanity check, ensuring that subsequent clustering interpretation is grounded in an understanding of feature distributions and relationships. All EDA is performed offline using Python-based analysis scripts and is strictly descriptive; no inferential claims or detection rules are derived from this stage.

The following analyses are performed:

- **Descriptive statistics:** Mean, median, variance, and range are computed for each feature across all nodes and experimental phases to identify baseline ranges and detect potential logging anomalies.
- **Distribution visualization:** Histograms and box plots are generated for key features (Forwarding Ratio, Retry Rate, RSSI-Hop Diff) stratified by phase and node role to observe shifts between experimental conditions.
- **Time-series plots:** Selected feature trajectories (e.g., parent switch events, PDR) are plotted over run duration to visualize the temporal alignment of manipulation windows.
- **Cross-layer correlation analysis:** Pearson and Spearman correlation matrices are computed to examine relationships between PHY, MAC, and Network layer features, testing the theoretical expectation of cross-layer consistency during baseline and its breakdown during manipulation.
- **Dimensionality reduction visualization:** PCA and t-SNE projections (as described in Section 4.2.5) are generated to provide an initial visual assessment of feature-space separability before formal clustering.

These analyses are conducted exclusively to understand the dataset structure and to validate that the engineered features capture meaningful behavioral variation. The results inform parameter choices for clustering (e.g., normalization confirmation and outlier identification), but they do not constitute intrusion detection or behavioral classification.

4.3. Theoretical Analysis

This section provides the theoretical justification for the selected cross-layer features and clustering methodology. While Section 4.2 detailed the implementation procedures, this section explains the conceptual rationale linking routing manipulations to observable behavioral deviations in wireless mesh networks. The analysis is grounded in the theoretical framework established in Chapter 3 and demonstrates how the empirical methodology aligns with fundamental principles of wireless mesh networking and routing behavior.

4.3.1. Theoretical Justification of Selected Features

The features engineered in Section 4.2.4 were not chosen arbitrarily; each is grounded in the cross-layer inconsistency framework developed in Chapter 3. Routing attacks in wireless mesh networks do not produce isolated anomalies within a single protocol layer; instead, they generate correlated inconsistencies across PHY, MAC, and Network layers due to the interdependent nature of wireless communication (Al-Anzi [5]; Amouri et al. [8]). The selected features are designed to capture these cross-layer disruptions by targeting the specific relationships that routing manipulations violate.

4.3.1.1. Forwarding Behavior Features (Blackhole Theory)

Forwarding suppression attacks directly violate the fundamental routing assumption of multi-hop networks: that intermediate nodes forward transit traffic. In ESP-WIFI-MESH, forwarding compliance is implicit in the routing design; nodes are expected to relay packets received from children toward the root and vice versa. The theoretical concept of packet conservation—that packets received should approximately equal packets forwarded—serves as an invariant of normal operation (Kurosawa et al. [43]; Baadache & Belmehdi [44]).

The following features operationalize this invariant:

- **Forwarding Ratio** directly measures the proportion of received transit packets that are forwarded. A value significantly below 1.0 indicates a violation of the conservation principle.
- **Ingress-Egress Delta** quantifies the absolute packet imbalance, capturing the magnitude of packet absorption at the attacker node.
- **Forwarding Consistency Score** provides a unified deviation metric, useful for visualization and cluster interpretation.

These features were selected because they directly measure deviation from expected forwarding behavior. In normal mesh routing, packet conservation holds; a blackhole disrupts this conservation property. Thus, forwarding-based features operationalize the theoretical routing invariants of multi-hop networks.

4.3.1.2. Link Reliability Features (Indirect Propagation Theory)

Routing misbehavior does not affect only the attacker; it propagates stress both upstream and downstream. Victim nodes that attempt to send packets through a suppressing attacker experience repeated transmission failures, leading to MAC-layer retransmissions. This indirect effect is predicted by cross-layer telemetry theory: network-layer disruptions manifest as MAC-layer stress (Section 3.5.1).

The following features capture these second-order effects:

- **Retry Rate** measures the proportion of transmission attempts requiring retransmission. Under normal conditions, retry rates remain low (<5%); significant increases indicate link-level stress induced by forwarding failures.
- **Packet Delivery Ratio (PDR)** measures end-to-end success from the victim to the root. Prior WMN studies commonly report high baseline PDR under stable conditions (e.g., ~97% in Khan et al. [25]); in this work, baseline PDR is established empirically per run, and deviations from that baseline indicate degradation.

These features are theoretically grounded in the propagation of routing anomalies through the network stack. They capture the indirect, yet observable, consequences of forwarding suppression on nodes beyond the attacker.

4.3.1.3. Topology Stability Features (Routing Adaptation Theory)

ESP-WIFI-MESH dynamically adapts parent selection based on link quality and routing metrics. This adaptive behavior is a fundamental characteristic of wireless mesh routing protocols (Akyildiz et al. [1]). Under topology distortion—such as wormhole-inspired shortcut effects—nodes may perceive artificially attractive paths, triggering frequent parent reevaluation.

In this study, wormhole effects are emulated at the application/telemetry level via an auxiliary out-of-band tunnel; the expected manifestations therefore appear as topology instability and cross-layer mismatches rather than low-level frame anomalies.

The following features capture deviations from expected routing stability dynamics:

- **Parent Switch Rate** measures the frequency of upstream parent changes. Normal operation exhibits stable parent relationships; increased switching indicates instability in the topology.
- **Layer Change Count** captures fluctuations in hop count, reflecting nodes moving up or down in the routing hierarchy.
- **Hop Stability Duration** quantifies the longest continuous period of stable topology and provides a complementary measure of instability intensity.

These features are grounded in the theoretical behavior of adaptive mesh routing protocols, which are designed to maintain stable hierarchies under normal conditions.

4.3.1.4. Physical Layer Features (Signal Consistency Theory)

In normal wireless mesh operation, the Received Signal Strength Indicator (RSSI) correlates with physical proximity, and the hop count correlates with signal strength. This relationship arises from the physical propagation characteristics of radio waves: signal power degrades monotonically with distance (Espressif [21]).

Under wormhole-inspired distortion, logical topology no longer matches physical distance. A node may receive packets from a distant source with an RSSI far stronger than expected for its reported layer, creating a physical-logical mismatch (Khalil et al. [45]; Hu et al. [46]).

The following features capture this inconsistency:

- **RSSI Mean** provides the average signal strength, serving as a baseline for link quality.
- **RSSI Variance** measures signal stability; high variance may indicate interference or topology reconfiguration effects.
- **RSSI-Hop Inconsistency** directly quantifies the deviation between observed RSSI and the value expected based on the node's reported layer. Expected values are derived empirically from stable baseline windows, making the metric environment-specific and robust to testbed variations.

These features are theoretically grounded in the spatial signal-propagation properties of wireless networks and the assumption that physical distance should constrain logical topology.

4.3.1.5. Cross-Layer Inconsistency Theory

The core theoretical assumption of this study is that routing manipulation creates measurable cross-layer inconsistencies. Instead of relying on attack-specific signatures, the methodology measures structural deviations from expected relationships:

- Forwarding invariant violation (Network layer)
- Latency-to-hop mismatch (Cross-layer)
- RSSI-to-hop mismatch (Cross-layer)

These features collectively operationalize the concept of behavioral inconsistency across PHY, MAC, and Network layers. They were selected because they target the fundamental relationships that routing attacks disrupt, as identified in the theoretical framework of Chapter 3.

4.3.1.6. Cross-Layer Feature Relationship Preservation

A central premise of this study is that routing misbehavior manifests as broken relationships between protocol layers rather than isolated anomalies within a single layer. Accordingly, the feature set is designed to preserve these cross-layer relationships, enabling clustering algorithms to detect structural inconsistencies without imposing attack-specific signatures.

Two examples illustrate this design principle:

- **Receive–forward relationship:** The Forwarding Ratio and Forwarding Consistency Score operationalize the expected invariant that the number of transit packets received equals the number forwarded (packet conservation). Deviation from this relationship directly indicates forward suppression, independent of absolute traffic volume or timing characteristics.
- **Physical–logical relationship:** The RSSI-Hop Inconsistency feature captures the expected correlation between physical signal strength and logical hop distance. Under normal operation, RSSI degrades monotonically with hop count; a significant mismatch indicates topology distortion where logical proximity no longer reflects physical placement.

These features encode structural expectations derived from wireless mesh theory (Section 3.2) and attack theory (Section 3.4). They do not embed detection rules (e.g., fixed thresholds for "attack"), but instead preserve the relationships whose violations characterize misbehavior. This design allows exploratory clustering to discover whether such violations naturally separate normal from manipulated windows in the collected dataset.

4.3.2. Theoretical Basis for Data Clustering

The use of unsupervised clustering rather than supervised classification is a deliberate methodological choice grounded in the exploratory nature of this research. This subsection justifies why clustering is an appropriate concept, independent of algorithm mechanics.

4.3.2.1. Behavioral State Hypothesis

This study assumes that wireless mesh behavior exists in discrete operational states:

1. **Normal operation** – Characterized by stable forwarding ratios, low retry rates, consistent RSSI-hop relationships, and minimal parent switching.
2. **Forwarding suppression** – Characterized by attacker-side forwarding ratio degradation, victim-side retransmission increases, and end-to-end delivery reduction.

3. **Topology distortion** – Characterized by RSSI-hop inconsistency, latency-hop mismatch, increased parent switching, and tunnel activity.

If cross-layer telemetry captures meaningful deviations, these states should form separable regions in the multidimensional feature space. Clustering tests this hypothesis by attempting to discover groupings without using ground-truth labels during the clustering process.

4.3.2.2. Absence of Pre-Defined Signatures

This study does not assume that the behavioral signatures of routing attacks in ESP-WIFI-MESH are fully known a priori. While theoretical expectations exist (Section 4.3.1), how these expectations manifest in real hardware under controlled conditions is an empirical question. Unsupervised discovery allows patterns to emerge from the data without imposing pre-conceived classifications, aligning with the exploratory objective stated in Section 1.3.

4.3.2.3. Density-Based Justification (DBSCAN)

Theoretically, normal operation is stable and frequent, forming a dense region in feature space. Attack windows differ structurally and may form separate, dense regions. Transitional behavior—such as network recovery or partial manipulation effects—may appear as sparse noise points.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) aligns with this behavioral density assumption. It does not assume spherical clusters, does not require a predefined number of clusters, and explicitly identifies points that do not belong to any dense region. This aligns with the exploratory objective of determining whether natural groupings emerge from the data.

4.3.2.4. Partition-Based Justification (K-Means)

K-Means is included as a comparative structural test. If behavioral states are compact and separable in Euclidean space, partition-based clustering should also reveal coherent groupings. The agreement between DBSCAN and K-Means strengthens the validity of discovered clusters, indicating that separation is not an artifact of a specific algorithm.

DBSCAN is used as the primary method due to its noise handling and ability to discover clusters of arbitrary shape. At the same time, K-Means serves as a comparative baseline to test whether the same behavioral separation emerges under a centroid-partition model.

4.3.2.5. Validation Theory

Internal validation metrics assess cluster quality independently of ground truth:

- **Silhouette Score** measures how similar points are to their own cluster compared to other clusters.
- **The Davies-Bouldin Index** measures the average similarity between clusters, with lower values indicating better separation.

These metrics provide evidence that the intra-cluster distance is coherent.

Post-analysis, cluster purity is computed against experimental phase labels. This two-level validation ensures that discovered clusters are not artifacts of parameter tuning and that they align with the controlled experimental conditions.

4.3.2.6. Alignment with Research Objectives

The study's primary objective is to generate a dataset and explore its structure, not to build a deployable classifier. Clustering directly supports this objective by revealing whether the collected features capture meaningful distinctions between behavioral states. The results inform future work on intrusion detection by highlighting which features are most discriminative and whether attack-induced behaviors naturally separate from normal operation.

4.3.2.7. Theoretical Expectation

If attacks create measurable cross-layer deviations, then these deviations should manifest as separable behavioral states in multidimensional feature space. Clustering provides an empirical test of this proposition: distinct clusters aligned with experimental phase labels would validate that the feature set captures attack-induced behavioral patterns. In contrast, overlapping clusters would suggest either insufficient feature discrimination or that the theoretical expectations are not reflected in the hardware behavior. Either outcome contributes valuable knowledge to the research community.

4.4. Cluster Visualization and Interpretation

To facilitate interpretation of the clustering results, several visualization techniques will be used to examine the structure of the clusters and the behavioral characteristics associated with each cluster. These visualizations allow the study to analyze how telemetry observations are distributed across clusters and to determine whether the clusters correspond to meaningful network behaviors.

First, dimensionality reduction techniques such as Principal Component Analysis (PCA) and t-distributed Stochastic Neighbor Embedding (t-SNE) will be applied to project the high-dimensional feature vectors into a two-dimensional space. These projections will be visualized using scatter plots where each point represents a node–window observation, and colors indicate the cluster assignments produced by the clustering algorithms. This visualization provides an intuitive representation of cluster separation and density structure within the dataset.

To further characterize the clusters, feature distribution analysis will be performed for each cluster. Summary statistics and box plots will be used to examine how key telemetry features differ across clusters. This helps identify the behavioral patterns associated with each cluster, such as variations in forwarding ratios, hop counts, retransmission rates, or signal strength indicators.

Additionally, heat maps of feature values per cluster will be generated to visualize the relative magnitude of each feature across clusters. In these heat maps, rows correspond to clusters and columns correspond to features, allowing direct comparison of feature intensities between clusters. This visualization assists in identifying which features contribute most strongly to cluster differentiation.

Cluster characterization will also involve examining the instances assigned to each cluster. By analyzing the distribution of experimental labels (e.g., baseline, wormhole attack, blackhole attack, or combined attack) within each cluster, the study can determine whether clusters correspond to specific behavioral states of the network. This analysis provides insight into how the clustering algorithms group normal and anomalous network behaviors.

Together, these visualization and analysis techniques enable the study to interpret the clustering results beyond numerical evaluation metrics. They provide qualitative insight into the behavioral patterns present in the dataset and help determine whether the clusters reflect meaningful distinctions between normal network operation and attack scenarios.

Appendix A. Bibliography

[1] Akyildiz, I. F., Wang, X., & Wang, W. (2005). *Wireless mesh networks: A survey.* Computer Networks, 47(4), 445–487. https://www.sciencedirect.com/science/article/abs/pii/S1389128604003457

[2] Cilfone, G., Davoli, L., Belli, L., & Ferrari, G. (2019). *Wireless mesh networking: An IoT-oriented perspective—Survey on relevant technologies.* Future Internet, 11(4), 1–41.https://www.mdpi.com/1999-5903/11/4/99

[3] Hanif, M., Ashraf, H., Jalil, Z., Jhanjhi, N. Z., Humayun, M., Saeed, S., & Almuhaideb, A. M. (2022). AI-Based Wormhole Attack Detection Techniques in Wireless Sensor Networks. Electronics, 11(15), 2324. https://www.mdpi.com/2079-9292/11/15/2324

[4] Bhatti, D. S. et al. (2024). *Detection and isolation of wormhole nodes in wireless ad hoc networks based on post-wormhole actions.* Scientific Reports, 14, 3428. https://www.nature.com/articles/s41598-024-53938-9

[5] Al-Anzi, F. S.(2022). *Design and analysis of intrusion detection systems for wireless mesh networks.* Digital Communications and Networks. https://www.sciencedirect.com/science/article/pii/S2352864822001043

[6] Ioulianou, P. P., Vassilakis, V. G., & Shahandashti, S. F. (2022). *A Trust-Based Intrusion Detection System for RPL Networks: Detecting a Combination of Rank and Blackhole Attacks.* Journal of Cybersecurity and Privacy, 2(1), 124-153. https://www.mdpi.com/2624-800X/2/1/9

[7] M. Alharthi, A. A. Alabdulatif, and A. Aljohani, *"XLID: Cross-Layer Intrusion Detection System for Wireless Sensor Networks," International Journal of Science and Technology (IJST)*, vol. 8, no. 3, pp. 87–94, 2019. [Online]. Available: https://sciresol.s3.us-east-2.amazonaws.com/IJST/Articles/2019/Issue-3/Article8.pdf

[8] A. Amouri, H. Abid, and N. Boudriga, *"A Cross-Layer, Anomaly-Based IDS for WSN and MANET,"* Sensors, vol. 18, no. 2, p. 651, 2018. [Online]. Available: https://www.mdpi.com/1424-8220/18/2/651

[9] G. Singh, *"A Cross-Layer Based Optimized Feature Selection Scheme for IDS in WSNs,"* Journal of Intelligent & Fuzzy Systems, vol. 42, pp. 1–13, 2022. [Online]. Available: https://journals.sagepub.com/doi/abs/10.3233/JIFS-210700

[10] Ayad, A.G., Sakr, N.A. & Hikal, N.A. A hybrid approach for efficient feature selection in anomaly intrusion detection for IoT networks. J Supercomput 80, 26942–26984 (2024). https://link.springer.com/article/10.1007/s11227-024-06409-x

[11] A. A. Al-Qarawi and M. A. Razzaque, "Energy-Aware Cross-Layer Intrusion Detection System for IoT-Based Wireless Mesh Networks," Sensors, vol. 22, no. 18, 2022. [Online]. Available: https://www.mdpi.com/1424-8220/22/18/6954

[12] Arshad, J., Azad, M. A., Amad, R., Salah, K., Alazab, M., & Iqbal, R. (2020). A Review of Performance, Energy and Privacy of Intrusion Detection Systems for IoT. Electronics, 9(4), 629. https://www.mdpi.com/2079-9292/9/4/629

[13] Alshehri AH. 2024. Wormhole attack detection and mitigation model for Internet of Things and WSN using machine learning. PeerJ Computer Science 10:e2257 https://peerj.com/articles/cs-2257/

[14] Sarhan, M., Layeghy, S., Moustafa, N., Gallagher, M., & Portmann, M. (2024). Feature extraction for machine learning-based intrusion detection in IoT networks. Digital Communications and Networks, 10(1), 205-216.https://arxiv.org/abs/2108.12722

[15] Churcher, A., Ullah, R., Ahmad, J., Rehman, S. u., Masood, F., Gogate, M., Alqahtani, F., Nour, B., & Buchanan, W. J. (2021). *An experimental analysis of attack classification using machine learning in IoT networks*. MDPI Sensors, 21(2), 446. https://www.mdpi.com/1424-8220/21/2/446

[16] Shafieian, S., & Zulkernine, M. (2022). *Multi-layer stacking ensemble learners for low footprint network intrusion detection*. SpringerLink. https://link.springer.com/article/10.1007/s40747-022-00809-3

[17] Wardana, S. A., Yusof, R., Zolkipli, M. F., & Kim, D. (2024). *Collaborative intrusion detection using weighted ensemble averaging deep neural network for coordinated attack detection in heterogeneous networks. International Journal of Information Security,* 23(4), 801–815. https://link.springer.com/article/10.1007/s10207-024-00891-3

[18] Vanlalhruaia, Ajoy, K.K., Amit, K.R. (2025). Securing AODV against black hole attack for wireless mesh network. Journal of Information Systems Engineering & Management, 10(37s), 1192–1202. https://doi.org/10.52783/jisem.v10i37s.7547

[19] Srinivasan, J. (2025). Innovative cross-layer defense mechanisms for blackhole and wormhole attacks in wireless ad-hoc networks. Scientific Reports, 15(1), 14747.

[20] Bhonsle, M., Srinivasulu, G., Chaitanya, K., Raghu, D., Surendra, G., Kumar, K., Rao, M. S., Prabhakar, K., & Krishna, V. (2025). Deep Learning-Based Defense Mechanisms against black hole attacks in wireless mesh networks. Advanced Sustainable Science Engineering and Technology, 7(1), 02501010. https://doi.org/10.26877/asset.v7i1.1036

[21] Espressif Systems. (2023). ESP-WIFI-MESH Programming Guide. https://docs.espressif.com/projects/esp-idf/en/v5.0/esp32/index.html

[22] Khan, A. U., Khan, M. E., Hasan, M., Zakri, W., Alhazmi, W., & Islam, T. (2022). An Efficient Wireless Sensor Network Based on the ESP-MESH Protocol for Indoor and Outdoor Air Quality Monitoring. Sustainability, 14(24). https://doi.org/10.3390/su142416630

[23] Qian, L., Gu, K., Fu, Y., Shen, Y., & Xu, S. (2024). A Wireless Ad Hoc Network Communication Platform and Data Transmission Strategies for Multi-Bus Instruments. Electronics, 13(18). https://doi.org/10.3390/electronics13183596

[24] A, Avina & R, Geetha. (2025). ResNeXt and Deep Stacked Autoencoder based framework for wormhole attack detection on network control systems. Discover Computing. 28. 10.1007/s10791-025-09816-7.

[25] Boiano, A., Zheng, D., Palmese, F., Pimpinella, A., & Redondi, A. E. C. (2026, February 3). Analyzing Zigbee Traffic: Datasets, classification and storage trade-offs. arXiv.org. https://arxiv.org/abs/2602.03140

[26] Das, T., Hamdan, O. A., Shukla, R. M., Sengupta, S., & Arslan, E. (2023, January). Unr-idd: Intrusion detection dataset using network port statistics. In 2023 IEEE 20th consumer Communications & networking conference (CCNC) (pp. 497-500). IEEE.

[27] Moustafa, Nour & Slay, Jill. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). 10.1109/MilCIS.2015.7348942.

[28] Miekkala, Topi & Pyykönen, Pasi & Drainakis, Georgios & Pantazopoulos, Panagiotis & Muller, Tobias & Katsaros, Konstantinos & Sourlas, Vasilis & Amditis, Angelos & Kaklamani, Dimitra. (2024). NordicDat: A Cross-Border Predictive QoS Dataset. 1281-1286. 10.1109/GLOBECOM52923.2024.10901462.

[29] Tavallaee, Mahbod & Bagheri, Ebrahim & Lu, Wei & Ghorbani, Ali. (2009). A detailed analysis of the KDD CUP 99 data set. IEEE Symposium. Computational Intelligence for Security and Defense Applications, CISDA. 2. 10.1109/CISDA.2009.5356528.

[30] Sommer, Robin & Paxson, Vern. (2010). Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. Proceedings - IEEE Symposium on Security and Privacy. 305-316. 10.1109/SP.2010.25.

[31] Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. Computers & security, 86, 147-167.

[32] García-Teodoro, Pedro & Díaz-Verdejo, Jesús & Maciá-Fernández, Gabriel & Vázquez, Enrique. (2009). Anomaly-based network intrusion detection: Techniques, systems and challenges. Computers & Security. 28. 18-28. 10.1016/j.cose.2008.08.003.

[33] Tsai, Chih-Fong & Hsu, Yu-Feng & Lin, Chia-Ying & Lin, Wei-Yang. (2009). Intrusion detection by machine learning: A review. Expert Systems with Applications. 36. 11994-12000. 10.1016/j.eswa.2009.05.029.

[34] Chandola, Varun & Banerjee, Arindam & Kumar, Vipin. (2009). Anomaly Detection: A Survey. ACM Comput. Surv.. 41. 10.1145/1541880.1541882.

[35] Tukey, J. W. (1977). Exploratory data analysis. Reading, MA: Addison-Wesley.

[36] Aggarwal, C. C. (2015). Data mining: the textbook (Vol. 1, No. 3). New York: springer.

[37] Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann Publishers.

[38] Jain, Anil. (2010). Data Clustering: 50 Years Beyond K-Means. Pattern Recognition Letters. 31. 651-666. 10.1016/j.patrec.2009.09.011.

[39] Pedroso, Carlos & de Souza Batista, Agnaldo & Jesus, Samuel & Santos, Aldri. (2024). A Direct Collaborative Network Intrusion Detection System for IoT Networks Integration. 10.5753/sbrc.2024.1354.

[40] Hernandez-Ramos, J. L., et al. (2022–2024). Intrusion detection based on federated learning: A comprehensive survey. ACM Computing Surveys.https://dl.acm.org/doi/10.1145/3731596

[41] Rassam, M. A., Almohammadi, K., & Abdulkareem, K. H. (2024). Autoencoder-based neural network model for anomaly detection in IoT networks. Computers, 13(5), 112. MDPI.https://www.mdpi.com/2624-831X/5/4/39

[42] J. Li, et al., *"Optimizing IoT Intrusion Detection System: Feature Selection versus Feature Extraction,"* Journal of Big Data, vol. 11, 2024. [Online]. Available: https://journalofbigdata.springeropen.com/articles/10.1186/s40537-024-00892-y

[43] Kurosawa, S., Nakayama, H., Kato, N., Jamalipour, A., & Nemoto, Y. (2007). Detecting blackhole attack on AODV-based mobile ad hoc networks by dynamic learning method. International Journal of Network Security, 5(3), 338–346. http://ijns.jalaxy.com.tw/contents/ijns-v5-n3/ijns-2007-v5-n3-p338-346.pdf

[44] Baadache, A., & Belmehdi, A. (2014). Struggling against simple and cooperative black hole attacks in multi-hop wireless ad hoc networks. Computer Networks, 73, 173–184. https://www.researchgate.net/publication/265687984_Struggling_against_simple_and_cooperative_black_hole_attacks_in_multi-hop_wireless_ad_hoc_networks

[45] Khalil, I., Bagchi, S., & Shroff, N. B. (2007). LiteWorp: Detection and isolation of the wormhole attack in static multihop wireless networks. Computer Networks, 51(13), 3750–3772. https://doi.org/10.1016/j.comnet.2007.04.001

[46] Hu, Y.-C., Perrig, A., & Johnson, D. B. (2003). Packet leashes: A defense against wormhole attacks in wireless networks. *IEEE INFOCOM 2003. Twenty-Second Annual Joint Conference of the IEEE Computer and Communications Societies*, *3*1976–1986. https://doi.org/10.1109/INFCOM.2003.1209219

[47] Keipour, H. (2022). Blackhole attack detection in low-power IoT mesh networks using machine learning algorithms.

[48] Chatterjee, S., et al. (2022). Anomaly detection in IoT systems: A survey. Internet of Things, 18, 100508. https://doi.org/10.1016/j.iot.2022.100508

[49] Miguel-Diez, A., et al. (2025). Unsupervised anomalous traffic detection based on flows: A systematic literature review. IEEE Access, 13, 45231–45259. https://doi.org/10.1109/ACCESS.2025.1234567

[50] van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. Journal of Machine Learning Research, 9, 2579–2605. https://www.jmlr.org/papers/v9/vandermaaten08a.html

[51] McInnes, L., Healy, J., & Melville, J. (2020). UMAP: Uniform manifold approximation and projection for dimension reduction. Nature Methods, 17(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

[52] Jolliffe, I. T. (2002). Principal component analysis (2nd ed.). Springer. https://doi.org/10.1007/b98835

[53] Shameer, C. M. (n.d.). Understanding DBSCAN clustering with a simple example. Medium. https://medium.com/@cmshameer712/understanding-dbscan-clustering-with-a-simple-example-940c468eeefa

[54] Soni, S. (n.d.). Clustering like a pro: A beginner's guide to DBSCAN. Medium. https://medium.com/@sachinsoni600517/clustering-like-a-pro-a-beginners-guide-to-dbscan-6c8274c362c4

[55] UpGrad. (n.d.). What is DBSCAN clustering? UpGrad Blog.

https://www.upgrad.com/blog/what-is-dbscan-clustering/

[56] STHDA. (n.d.). DBSCAN: Density-based clustering essentials. Statistical Tools for High-throughput Data Analysis.
https://www.sthda.com/english/wiki/wiki.php?id_contents=7940

[57] Bishop, C. M. (2006). Pattern recognition and machine learning. Springer.

[58] Lloyd, S. (1982). Least squares quantization in PCM. IEEE Transactions on Information Theory, 28(2), 129–137. https://doi.org/10.1109/TIT.1982.1056489

[59]ResearchGate. (n.d.). An example dendrogram for hierarchical clustering shown in Table 1. https://www.researchgate.net/figure/An-example-dendrogram-for-the-hierarchical-clustering-shown-in-Table-1-The-original-data_fig9_278706094

[60]UC Business Analytics R Programming Guide. (n.d.). Hierarchical clustering.
https://uc-r.github.io/hc_clustering

[61]Minitab. (n.d.). What is hierarchical clustering?
https://mtab.com/blog/what-is-hierarchical-clustering

[62]Singh, A. (n.d.). Hierarchical clustering: A tree-based approach to data grouping. Medium. https://medium.com/@abhaysingh71711/hierarchical-clustering-a-tree-based-approach-to-data-grouping-241131b1c4c5

[63]VanderPlas, J. (n.d.). Gaussian mixture models. In Python Data Science Handbook. https://jakevdp.github.io/PythonDataScienceHandbook/05.12-gaussian-mixtures.html

[64]MathWorks. (n.d.). Cluster data from a mixture of Gaussian distributions. https://www.mathworks.com/help/stats/cluster-data-from-mixture-of-gaussian-distributions.html

[65]AI Learning Hub. (n.d.). Gaussian mixture model (GMM): A clustering method. Medium. https://medium.com/@AILearningHub/gaussian-mixture-model-gmm-a-clustering-method-99263f8de214

[66]ResearchGate. (n.d.). Cluster fitting with a Gaussian mixture model (GMM). https://www.researchgate.net/figure/Cluster-fitting-with-a-Gaussian-mixture-model-GMM-The-scatter-plot-from-Figure-1d-is_fig3_350132034

[67]ResearchGate. (n.d.). The spectral clustering algorithm for graph-structured data. https://www.researchgate.net/figure/The-spectral-clustering-algorithm-for-the-graph-structured-data-a-A-simple-undirected_fig1_360129519

[68] for figure 4.42

[69]StrataScratch. (n.d.). Unsupervised clustering methods in machine learning. https://www.stratascratch.com/blog/unsupervised-clustering

Appendix B. Research Ethics Forms

RESEARCH / PROJECT ETHICS REVIEW FORM
For Thesis / Capstone / Dissertation Proposals
(*ver.Oct2024c*)

Names of Student Researcher(s):
Calpoporo, Jose Angelo C.
Carlos, Miguel Sebastian C.
Ong, Kyle Edward M.
Reinante, Christian G.

Research/Thesis/Capstone Title: Cross-Layer Dataset Design and Exploratory Analysis of ESP32-Based ESP-WIFI-MESH Network

College: College of Computer Studies

Department: Department of Computer Technology

Course: Bachelor of Computer Science Major in Network and Internet Security

Expected Duration of the Project: from Term 1 AY 2025-2026 to Term 1 AY 2026-2027

PROTOCOL:
(*To be filled up by proponents; Use the Ethics Checklists as guides in determining areas for ethical concern/consideration. Attach and properly reference the accomplished ethics checklists and supporting documents.)*

		Reference Document (include page number)
Target participants	**No human participants** are directly involved in the experiment. The "participants" are **ESP32 mesh nodes** used in laboratory testing. Only the researchers will interact with the system for configuration, observation, and data recording.	Section 1.6.1 Hardware (pp. 15)
Data to be used / collected	**Network Telemetry Data:** RSSI values, retransmission counts, hop transitions, forwarding ratios, parent-switching events, and packet delivery ratios. **Attack Telemetry Data:** For blackhole emulation: packets received, forwarded, and dropped at the attacker node. For wormhole emulation: tunnel message counts and bytes transferred.	Section 4.2.3.1. Victim Node Data Collection (pp. 93-94) Section 4.2.3.2. Attack Node Data Collection (pp. 94-96)

	System Logs: Phase labels, node roles, timestamps, and event logs (e.g., parent switches). No detection performance metrics (e.g., detection accuracy) will be collected as this study does not implement or evaluate an intrusion detection system.	Section 4.2.3.4. Phase Labeling and Synchronization (pp. 98-99)
Brief procedure	**Network Setup:** Deploy up to 10 ESP32 development boards in a controlled indoor environment to form an ESP-WIFI-MESH network. **Data Collection:** Conduct experimental runs across multiple physical topologies (star, tree, linear chain, partial mesh). During each run, the system cycles through baseline, controlled blackhole emulation, and controlled wormhole emulation phases, with precise phase labeling. **Data Logging & Storage:** All nodes log raw telemetry locally to flash storage. Logs are retrieved post-experiment via USB serial for offline processing. **Analysis:** Perform exploratory data analysis and unsupervised clustering on the generated dataset to examine behavioral patterns, not to evaluate detection performance.	Section 4.2.2. Physical Deployment Topologies (pp. 84-82) Section 4.2.3.3. Data Acquisition and Logging Architecture (pp. 96-98) Section 4.2.3.6. Local Storage and Post-Experiment Extraction (pp. 100-101) Section 4.2.6. Exploratory Data Analysis (EDA) (pp. 145)
Potential risks	**Hardware Risks:** Minimal chance of ESP32 overheating or failure during prolonged testing. **Data Risks:** No personal or confidential data will be transmitted or collected. **Network Interference Risks:** Controlled emulation of routing misbehavior (forwarding suppression and topology distortion) could potentially disrupt	Section 1.4.1 Limitations of the Study (pp. 14) Section 4.2.1.2.A Design Scope and Implementation Boundaries (pp. 62)

	communication within the isolated testbed. All experiments are conducted in a controlled laboratory environment with no connection to external or production networks, mitigating any risk to other systems.	Section 4.2.1.3.A Design Scope and Implementation Boundaries (pp. 70-71)
Applicable Terms of Use or Licensing Agreement of Data, Models and/or Tools	**ESP-IDF SDK / Arduino IDE:** Open-source frameworks under Espressif's Apache License. **Wireshark:** GNU GPL license; used solely for verification within the isolated test network. **Python/MATLAB:** Used for offline preprocessing and analysis of the generated dataset. **AI Assistance:** Artificial intelligence tools (e.g., ChatGPT by OpenAI) were used to support literature review synthesis, framework visualization, and technical documentation drafting. Final system design, coding, and evaluation were performed entirely by the researchers.	Section 1.6.2 Software (pp. 15-16)
Steps to safeguard the participants and/or data	**Data Isolation:** The mesh network is local and disconnected from the internet during testing. **No Personal Data:** Only system-generated telemetry data is processed, ensuring compliance with privacy standards. **Access Control:** Only authorized researchers may handle devices and data files. **Data Integrity:** Logs are stored locally on nodes and transferred only after experiments, ensuring no data loss due to network attacks.	Section 4.2.3.6. Local Storage and Post-Experiment Extraction (pp. 100-101)
Other concerns and considerations	All experiments will take place in a controlled laboratory environment to ensure repeatability and safe containment of network manipulations. Results will be used solely for academic and research purposes. All open-source tools, models, and datasets will be cited appropriately in the final thesis report.	Section 1.4.1 Limitations of the Study (pp. 14) Section 1.5 Significance of

		the Study (pp. 14-15)

PANEL'S ASSESSMENT/COMMENTS:

PANEL'S DECISION:

☐ **Approved** (*Proponent/s may proceed with the study/data collection*)

☐ **Modifications Required** (*Proponent/s should address ethical issues, follow the panel's recommendations for required modifications, and resubmit Ethics Review Form for approval before proceeding with the study or before starting with data collection*)

☐ **For Full Review** (*Proponent/s should submit the proposal to the University Research Ethics Review Committee (RERC) and obtain a research ethics clearance before proceeding with the study or before starting with data collection*)

JUSTIFICATION:

_____

___

_____

___

_____

___

Mr. Gregory G. Cu

Name and Signature of Adviser/Mentor
Date:

_____

_____

Name and Signature of Panel Member
Date:

Name and Signature of Panel Member
Date:

_____

Name and Signature of Panel Member
Date: